Optimal Planning Strategies for the Board Game "Ricochet Robots"

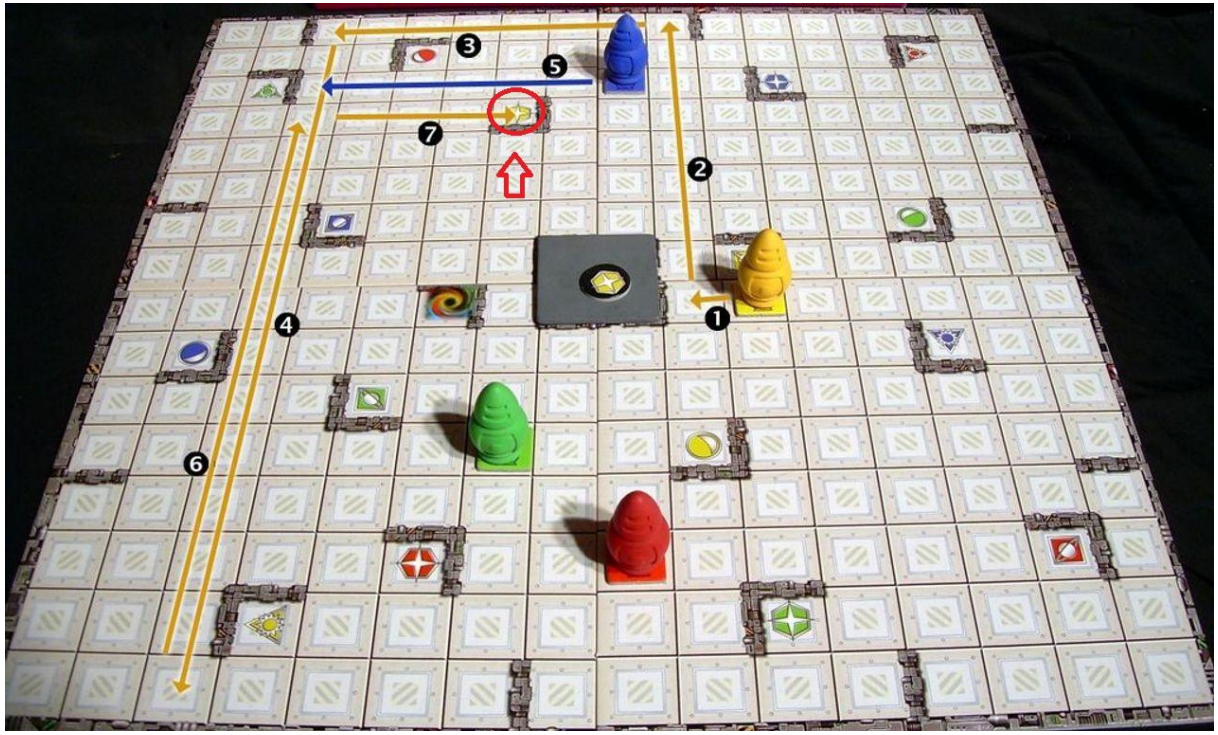


SAPIENZA
UNIVERSITÀ DI ROMA

course: Planning and Reasoning A.Y. 2025/2026

project by: Samuele Proietti 1807094 Luca del signore 2096442

What is Ricochet Robots?



https://images.app.goo.gl/naogfo9t32YJEaAm8

- Robots slide until an obstacle blocks their movement
- Each slide counts as one action
- The player that finds a path to the goal with fewer actions within a time interval takes the point
- Other robots can be used to create dynamic obstacles

Extensions of the Domain

These properties were added in order to have an incremental difficulty in the development of the project:

1. **Multiple goal-positions** in a sequential order
2. **Battery Charge**: Each robot has a limited number of actions that it can perform (managed through a **fluent-counter**). They completely recharge by visiting a charging station

PDDL

Problems

1. A standard game round (as the original game) with a single goal-position and unlimited energy
2. ~~A game round with a single goal-position and limited energy~~
   - ➢ A game round with multiple goal-positions and unlimited energy
3. A game round with multiple goal-positions and limited energy

Approaches Used



Approaches Used

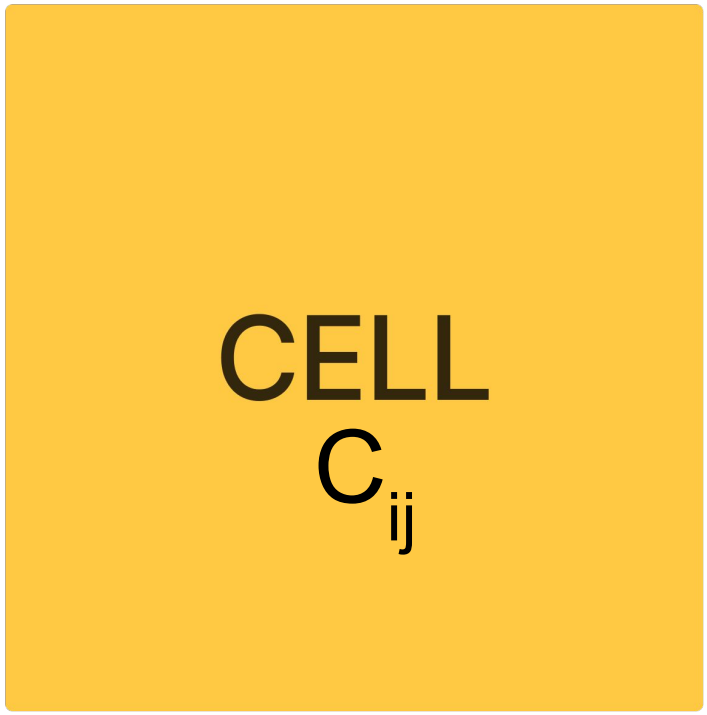
Planners: FastDownward and ENHSP NEW
Algorithm: A* and Greedy Best First Search NEW
Heuristics: h^{max} and h^{add}

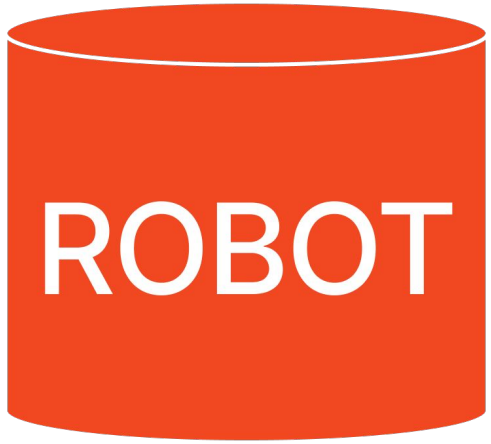




PDDL Modelling

Types deriving from “object”





Cell name convention: i-th row and j-th column

WALL PREDICATES

wall_to_the_top_of ?c - cell



wall_to_the_left_of ?c - cell

wall_to_the_right_of ?c - cell

wall_to_the_bottom_of ?c - cell

WALL PREDICATES

wall_to_the_top_of ?c - cell



wall_to_the_left_of ?c - cell

wall_to_the_right_of ?c - cell

wall_to_the_bottom_of ?c - cell

WALL PREDICATES

wall_to_the_top_of ?c - cell



wall_to_the_left_of ?c - cell

wall_to_the_right_of ?c - cell

wall_to_the_bottom_of ?c - cell

WALL PREDICATES

wall_to_the_top_of ?c - cell



wall_to_the_left_of ?c - cell

wall_to_the_right_of ?c - cell

wall_to_the_bottom_of ?c - cell

ADJACENCY PREDICATES

cell_adj_to_the_top_of ?c1 – cell ?c2 - cell



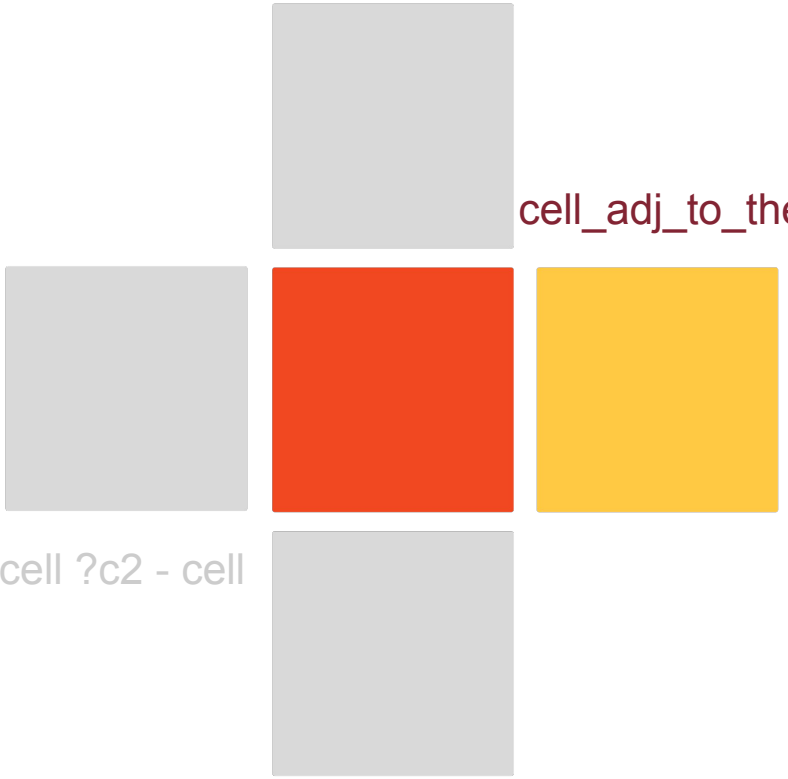
cell_adj_to_the_right_of ?c1 – cell ?c2 - cell

cell_adj_to_the_left_of ?c1 – cell ?c2 - cell

cell_adj_to_the_bottom_of ?c1 – cell ?c2 - cell

ADJACENCY PREDICATES

cell_adj_to_the_top_of ?c1 – cell ?c2 - cell



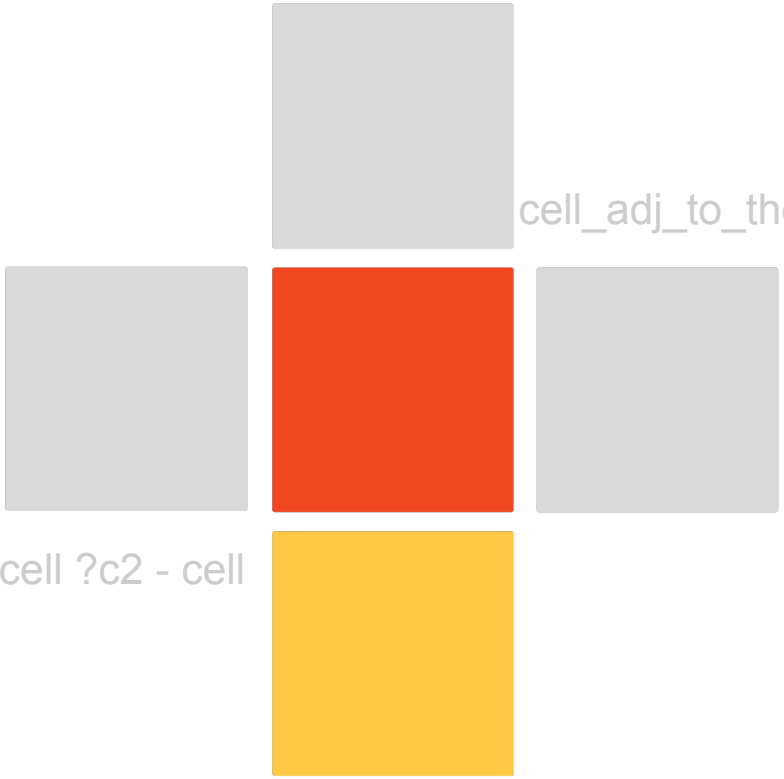
cell_adj_to_the_right_of ?c1 – cell ?c2 - cell

cell_adj_to_the_left_of ?c1 – cell ?c2 - cell

cell_adj_to_the_bottom_of ?c1 – cell ?c2 - cell

ADJACENCY PREDICATES

cell_adj_to_the_top_of ?c1 – cell ?c2 - cell



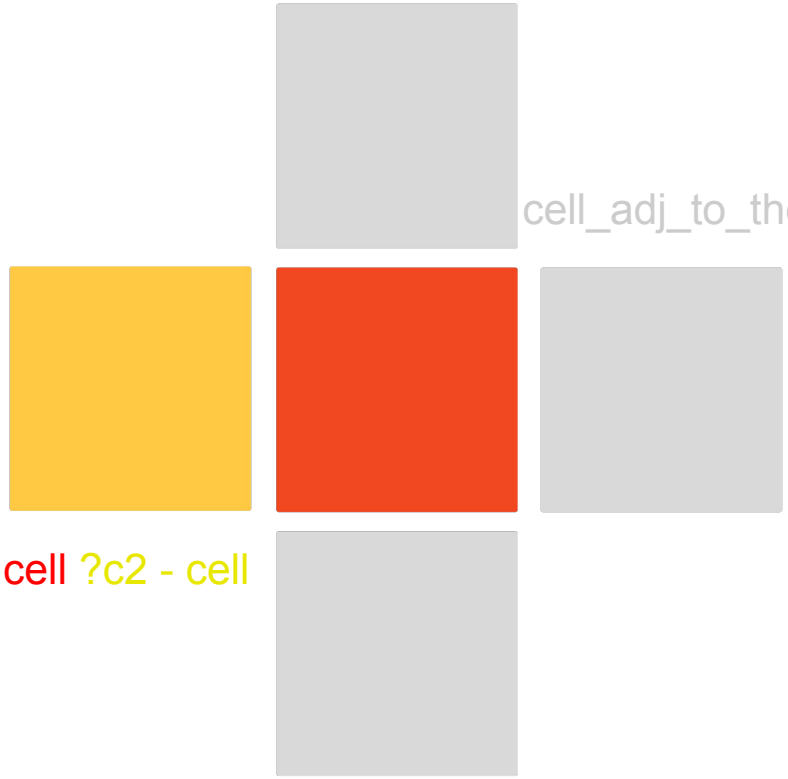
cell_adj_to_the_right_of ?c1 – cell ?c2 - cell

cell_adj_to_the_left_of ?c1 – cell ?c2 - cell

cell_adj_to_the_bottom_of ?c1 – cell ?c2 - cell

ADJACENCY PREDICATES

cell_adj_to_the_top_of ?c1 – cell ?c2 - cell



cell_adj_to_the_right_of ?c1 – cell ?c2 - cell

cell_adj_to_the_left_of ?c1 – cell ?c2 - cell

cell_adj_to_the_bottom_of ?c1 – cell ?c2 - cell

ROBOT PREDICATES

robot_at ?r – robot ?c - cell

IMPLIES

occupied ?c - cell



Directional Cell Relations (Row & Column)

For each cell, it is necessary to register which cells of the grid are:

- on the right side in the same row;
- on the left side in the same row;
- on the top side in the same column;
- on the bottom side in the same column;

Directional Cell Relations PREDICATES

(cell_aligned_to_the_right_of ?c1 - cell ?c2 - cell)
(cell_aligned_to_the_left_of ?c1 - cell ?c2 - cell)
(cell_aligned_to_the_top_of ?c1 - cell ?c2 - cell)
(cell_aligned_to_the_bottom_of ?c1 - cell ?c2 - cell)



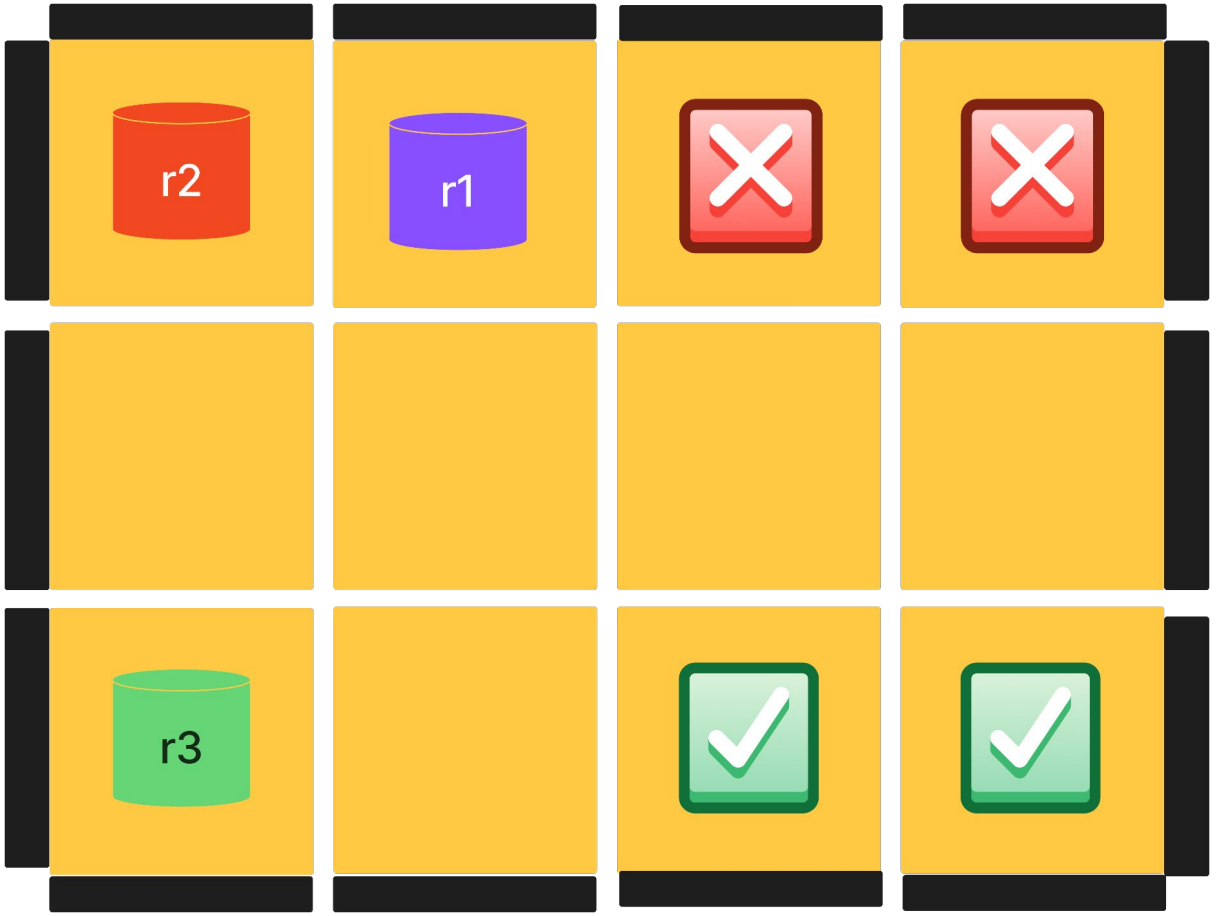
These relationships allow computing robots' line of sight by retrieving all cells in the same row/column in a given direction (see later for more details)

Line of sight

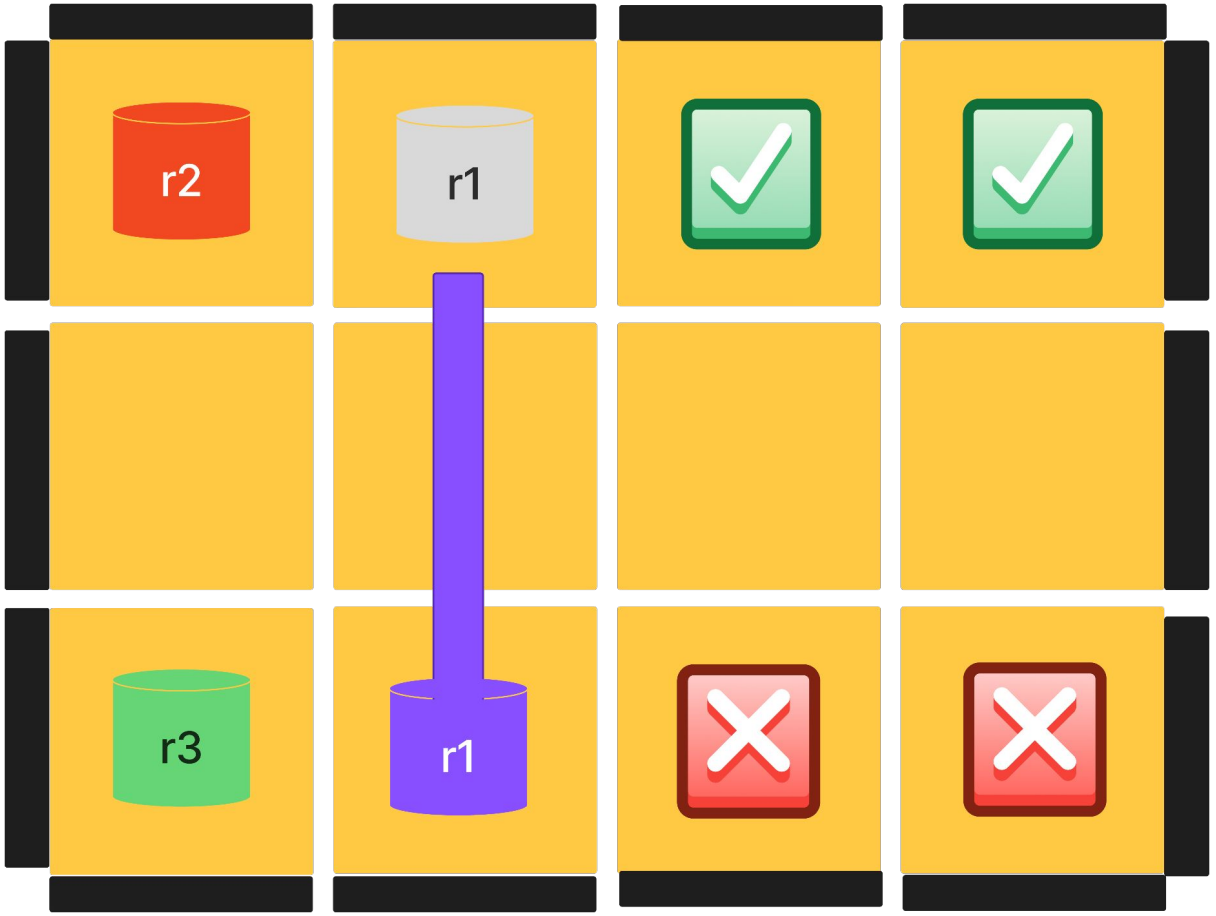
Each robot must determine which cell can see and pass through without colliding.

It would be impractical to use a normal predicate (free_line_row/column ?r - robot ?to - cell) because, after a sliding action of another robot, global changes of these predicates can occur around the grid

Not only a local change



Not only a local change



Two ways to solve this

:Derived-Predicates
recomputed after each action. They verify if a robot, given the new configuration of the grid, can move (or not move) through/to a cell

Pros: Fidelity to the original idea of model

Cons: More computational time

Atomic-step slide actions:
once the planner chooses a direction, we force it to repeat the same one-step action until it encounters an obstacle in the adjacent cell.

Pros: Simple to implement

Cons: Ad-hoc start and stop flag-predicates. It diverges from the original concept of sliding

Two ways to solve this

:Derived-Predicates
recomputed after each action. They verify if a robot, given the new configuration of the grid, can move (or not move) through/to a cell

Pros: Fidelity to the original idea of model

Cons: More computational time

Atomic-step slide actions: once the planner chooses a direction, we force it to repeat the same one-step action until it encounters an obstacle in the adjacent cell.

Pros: Simple to implement

Cons: Ad-hoc start and stop flag-predicates. It diverges from the original concept of sliding

Derived Predicates

(free_line_row ?r - robot ?to - cell)
(free_line_column ?r - robot ?to - cell)



Derived predicates to be recomputed after each action: number of calls of $O(mn)$ for each derived predicate, where m is the number of robots and n the cells.
The time complexity become $O(mn^2)$ being present a forall condition in the derived predicate

free_line_row ?from ?to

```
(:derived
  (free_line_row ?r - robot ?to - cell)
  (exists
    (?from - cell)
    (and
      (or ;"?to" is in the same row of "?from"
        (cell_aligned_to_the_right_of ?from ?to)
        (cell_aligned_to_the_left_of ?from ?to))
      (not (= ?from ?to))
      (robot_at ?r ?from)) ;"?r" is located in "?from"
      (forall
        (?c - cell)
        (or
          (= ?c ?from)
          (= ?c ?to)
          (not (or
                 (and
                   (cell_aligned_to_the_right_of ?from ?c)
                   (not(cell_aligned_to_the_right_of ?to ?c)))
                 (and
                   (cell_aligned_to_the_left_of ?from ?c)
                   (not(cell_aligned_to_the_left_of ?to ?c))))))
          (and
            (not (occupied ?c))
            (not (wall_to_the_left_of  ?c))
            (not (wall_to_the_right_of  ?c)))
        )
      )
))
```

free_line_row “forall” condition

```
(forall
      (?c - cell)
      (or
        (= ?c ?from)
        (= ?c ?to)
        (not (or
               (and
                 (cell_aligned_to_the_right_of ?from ?c)
                 (not(cell_aligned_to_the_right_of ?to ?c)))
               (and
                 (cell_aligned_to_the_left_of ?from ?c)
                 (not(cell_aligned_to_the_left_of ?to ?c))))))
        (and
          (not (occupied ?c))
          (not (wall_to_the_left_of  ?c))
          (not (wall_to_the_right_of  ?c))
        )))
```

=

$$\forall c \left(\left((\text{RightOf}(from, c) \wedge \neg \text{RightOf}(to, c)) \vee (\text{LeftOf}(from, c) \wedge \neg \text{LeftOf}(to, c)) \right) \wedge (c \neq from) \wedge (c \neq to) \rightarrow \left(\neg \text{Occupied}(c) \wedge \neg \text{WallLeft}(c) \wedge \neg \text{WallRight}(c) \right) \right)$$

All cells between “?from” and “?to” in the same row must be free from obstacles and side walls (right and left side) to allow the robot “?r” to see the target cell “?to”

free_line_column ?from ?to

```
(:derived
  (free_line_column ?r - robot ?to - cell)
  (exists
    (?from - cell)
    (and
      (or ;"?to" is in the same column of "?from"
        (cell_aligned_to_the_top_of ?from ?to)
        (cell_aligned_to_the_bottom_of ?from ?to))
      (not (= ?from ?to))
      (robot_at ?r ?from)) ;"?r" is located in "?from"
      (forall
        (?c - cell)
        (or
          (= ?c ?from)
          (= ?c ?to)
          (not (or
                 (and
                   (cell_aligned_to_the_bottom_of ?from ?c)
                   (not(cell_aligned_to_the_bottom_of ?to ?c)))
                 (and
                   (cell_aligned_to_the_top_of ?from ?c)
                   (not(cell_aligned_to_the_top_of ?to ?c))))))
          (and
            (not (occupied ?c))
            (not (wall_to_the_top_of  ?c))
            (not (wall_to_the_bottom_of  ?c)))
        )
      )
))
```

ENHSP and Derived predicates

ENHSP does not support :derived-predicates, thus, as opposed to FastDownward, their inner computation must be explicitly written in each action's preconditions (accordingly to the direction).

This twin-symmetric domain will be used to compare ENHSP against FastDownward (with derived predicates) on the same goal formulas

ACTIONS

slide_right ?r ?from ?to - 1

```
(:action slide_right
  :parameters(?r - robot ?from - cell ?to - cell)
  :precondition(and
    (or
      (wall_to_the_right_of ?to);there is a wall or an obst-
      (exists ;-right_adj to “?to” that can stop the move
        (?c - cell)
        (and
          (occupied ?c)
          (cell_adj_to_the_right_of ?to ?c)
       )))
    (cell_aligned_to_the_right_of ?from ?to);in the same row
    (not(wall_to_the_right_of ?from))
    (not(occupied ?to))
    (free_line_row ?r ?to);line of sight until “?to”
    (robot_at ?r ?from)
  )
```

slide_left ?r ?from ?to - 1

```
(:action slide_left
  :parameters(?r - robot ?from - cell ?to - cell)
  :precondition(and
    (or
      (wall_to_the_left_of ?to)
      (exists
        (?c - cell)
        (and
          (occupied ?c)
          (cell_adj_to_the_left_of ?to ?c)
      )))
    (cell_aligned_to_the_left_of ?from ?to)
    (not(wall_to_the_left_of ?from))
    (not(occupied ?to))
    (free_line_row ?r ?to)
    (robot_at ?r ?from)
  )
```

slide_up ?r ?from ?to - 1

```
(:action slide_up
  :parameters(?r - robot ?from - cell ?to - cell)
  :precondition(and
    (or
      (wall_to_the_top_of ?to)
      (exists
        (?c - cell)
        (and
          (occupied ?c)
          (cell_adj_to_the_top_of ?to ?c)
      )))
    (cell_aligned_to_the_top_of ?from ?to)
    (not(wall_to_the_top_of ?from))
    (not(occupied ?to))
    (free_line_column ?r ?to)
    (robot_at ?r ?from)
  )
```

slide_down ?r ?from ?to - 1

```
(:action slide_down
  :parameters(?r - robot ?from - cell ?to - cell)
  :precondition(and
    (or
      (wall_to_the_bottom_of ?to)
      (exists
        (?c - cell)
        (and
          (occupied ?c)
          (cell_adj_to_the_bottom_of ?to ?c)
      )))
    (cell_aligned_to_the_bottom_of ?from ?to)
    (not(wall_to_the_bottom_of ?from))
    (not(occupied ?to))
    (free_line_column ?r ?to)
    (robot_at ?r ?from)
  )
```

slide_* ?r ?from ?to - 2

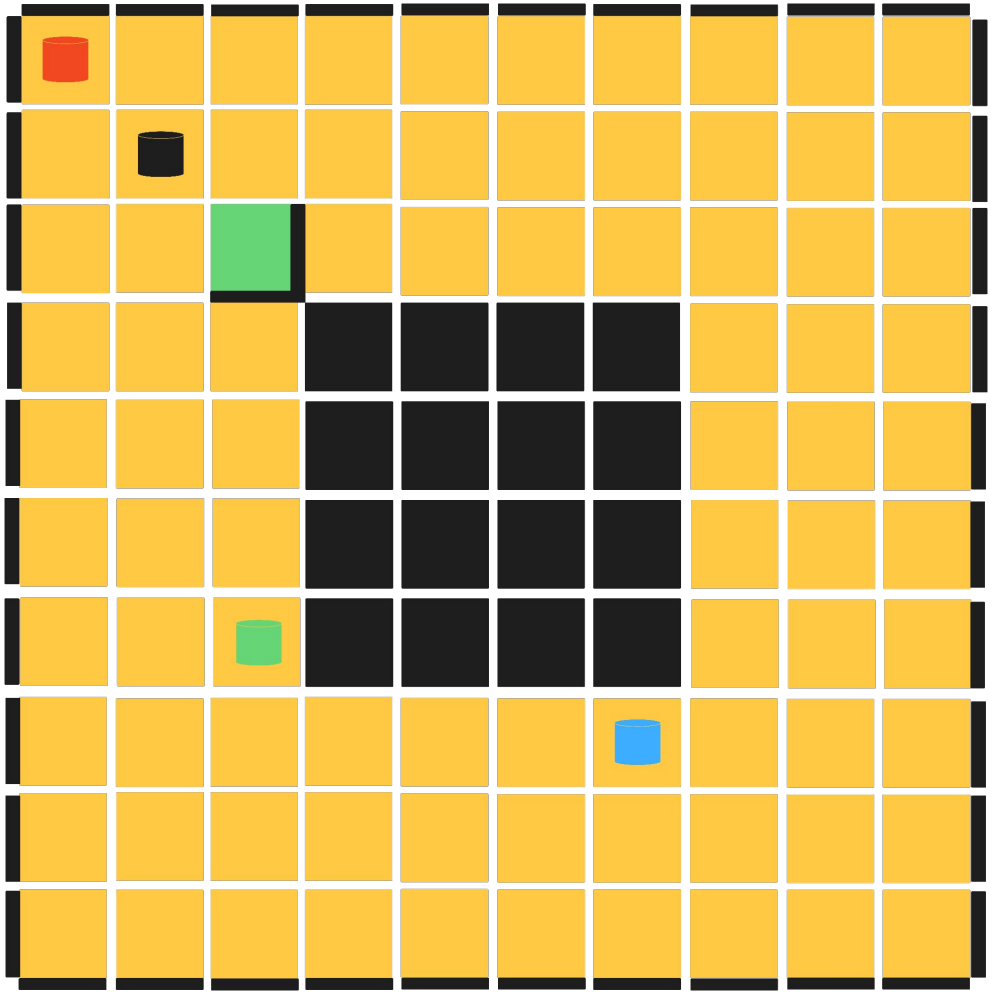
```
  :effect(and
    (not(robot_at ?r ?from))
    (not(occupied ?from))
    (robot_at ?r ?to)
    (occupied ?to)
    (increase (total-cost) 1)
  )
)
```

SINGLE GOAL

Initial state (:goal(robot_at red c33))

Legend

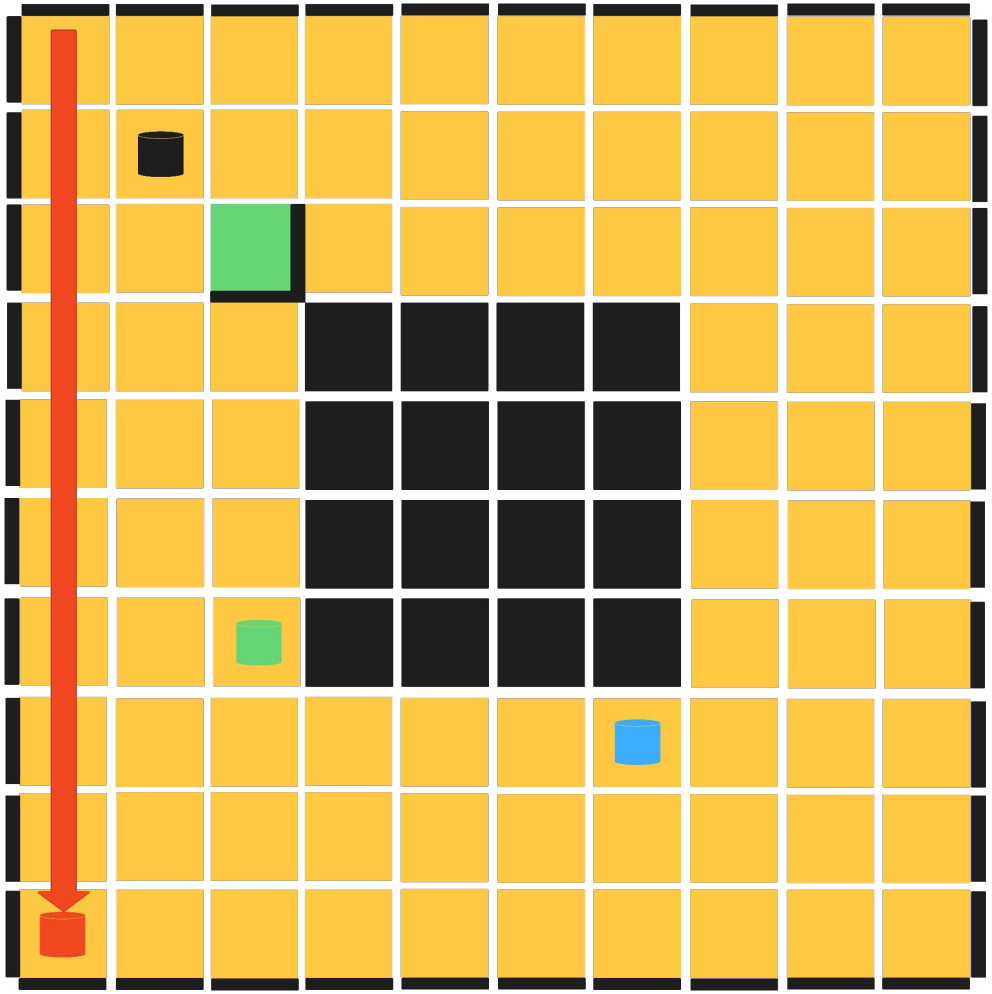
GOAL



1 - move_down red c11 c10_1

Legend

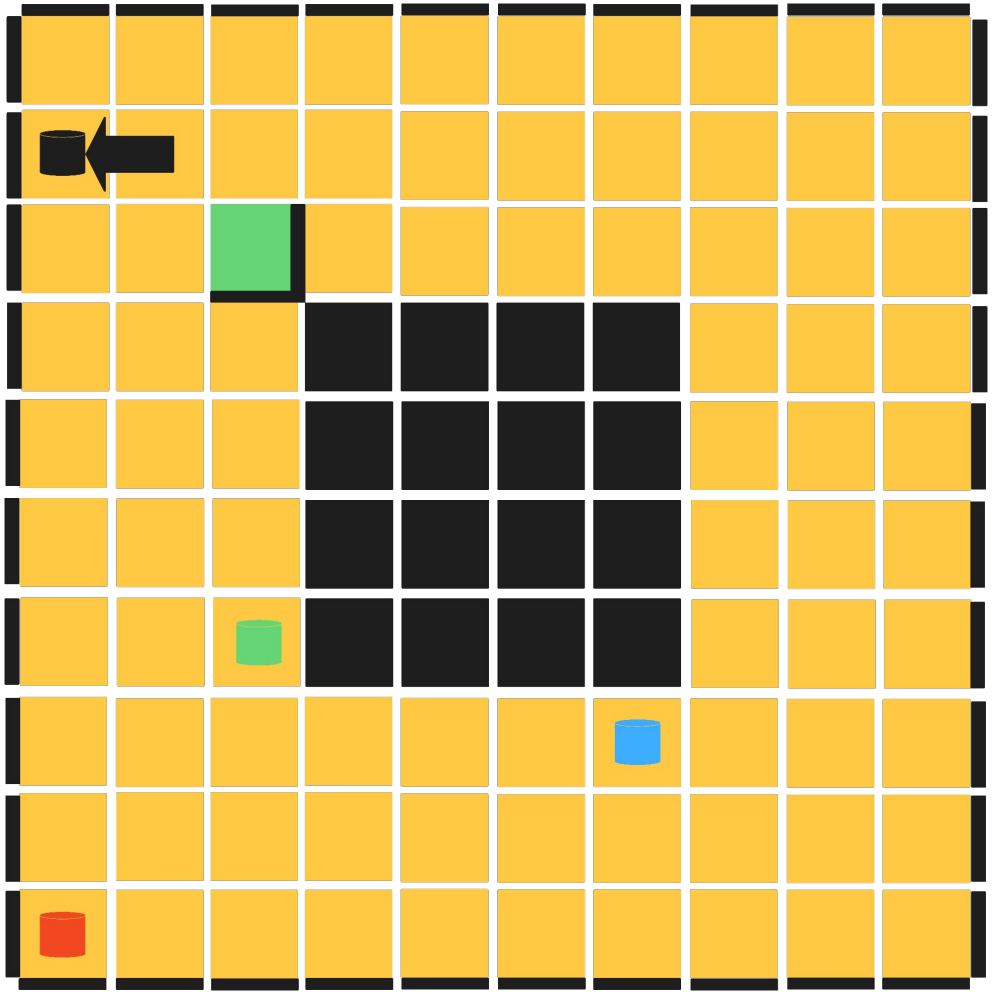
GOAL



2 - move_left black c22 c21

Legend

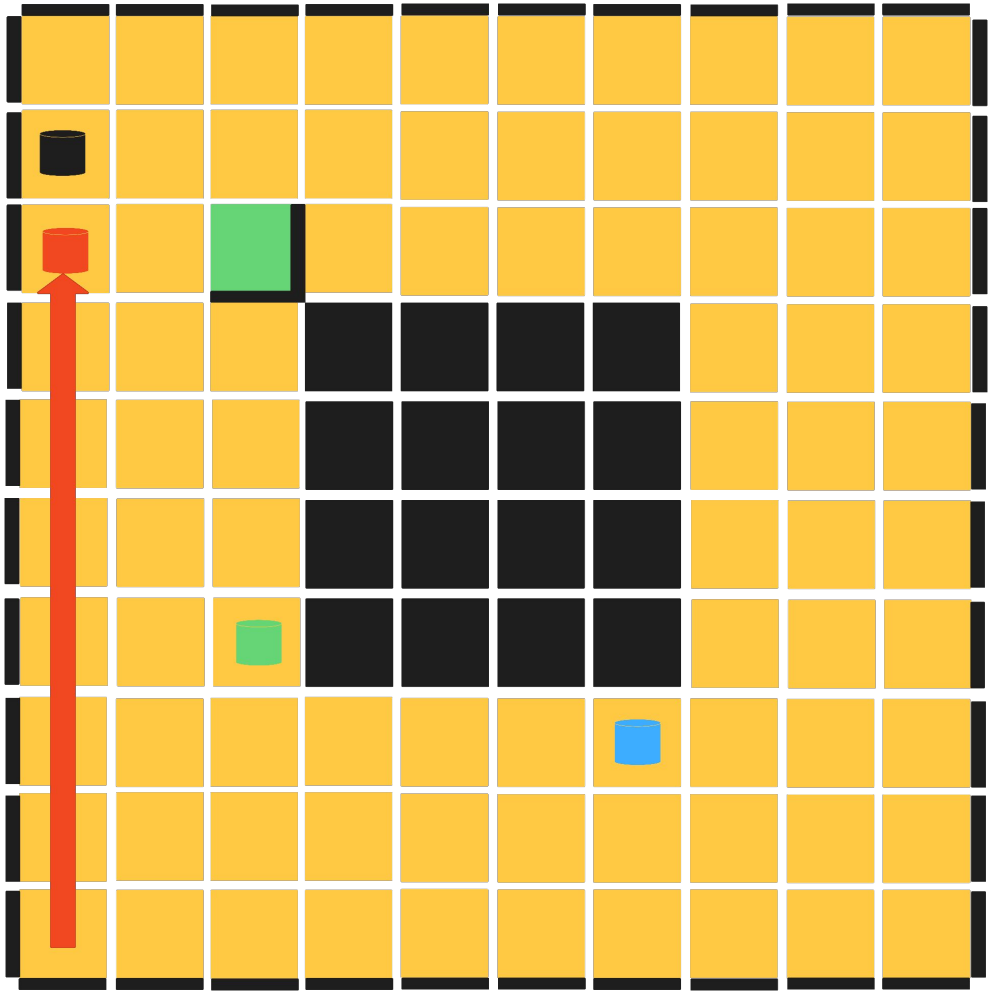
GOAL



3 - move_up red c10_1 c31

Legend

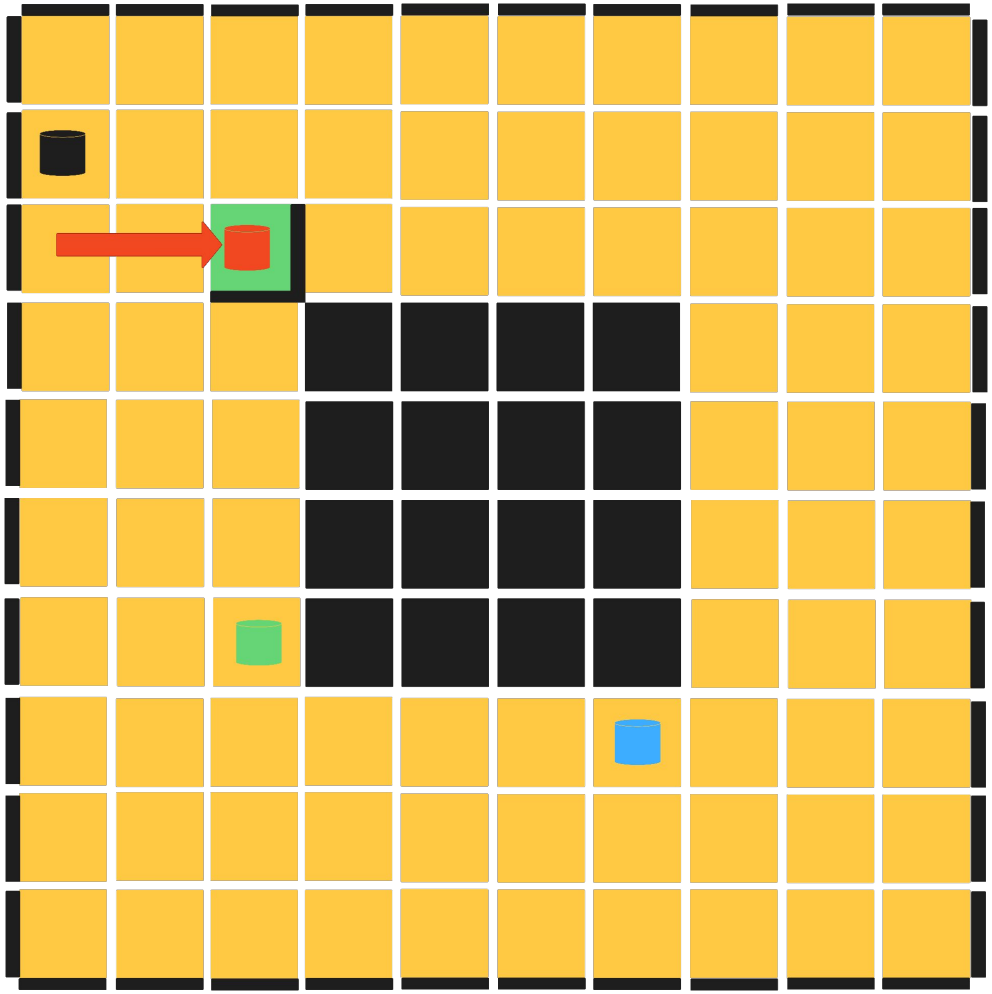
GOAL



4 - move_right red c31 c33

Legend

GOAL



(:goal(robot_at red c33))

	FD A* h^{max}	FD A* h^{add}	ENHSP A* h^{max}	ENHSP GBFS h^{add}
Plan Length	**4**	**4**	**4**	**4**
States evaluated	**44**	**44**	75	46
Expanded nodes	**5**	**5**	8	**5**
Search Time (s)	0.0088	**0.0082**	0.329	0.203
Total Time (s)	4.89	4.79	3.65	**3,47**

(:goal(robot_at red c58))

	FD A* h^{max}	FD A* h^{add}	ENHSP A* h^{max}	ENHSP GBFS h^{add}
Plan Length	**9**	**9**	**9**	19
States evaluated	**1690**	1866	4522	74955
Expanded nodes	416	**345**	746	10098
Search Time (s)	0.46	**0.41**	11.57	128
Total Time (s)	5.24	**5.18**	21,3	138

MULTIPLE GOALS WITH ORDER

Model extensions

Predicates:

(visited ?r - robot ?c - cell) ➡ in the :goal

(visit_before ?r - robot ?c1 - cell ?c2 - cell) ➡ in the :init

Added to all action preconditions (main conjunction):

```
(not(exists
      (?c - cell)
      (and
        (visit_before ?r ?c ?to)
        (not(visited ?r ?c))
)))
```

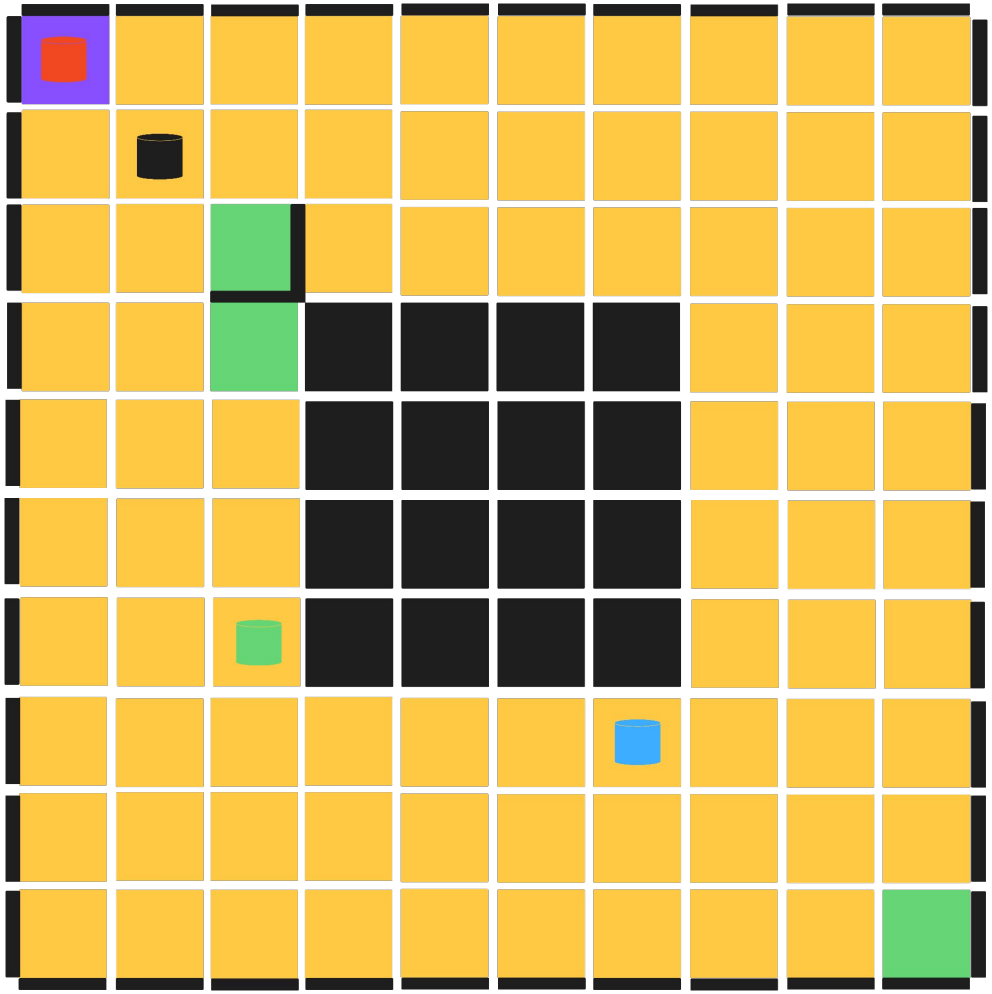
Added to all action effects (main conjunction):

```
(visited ?r ?to)
```

Initial state

Legend

GOAL



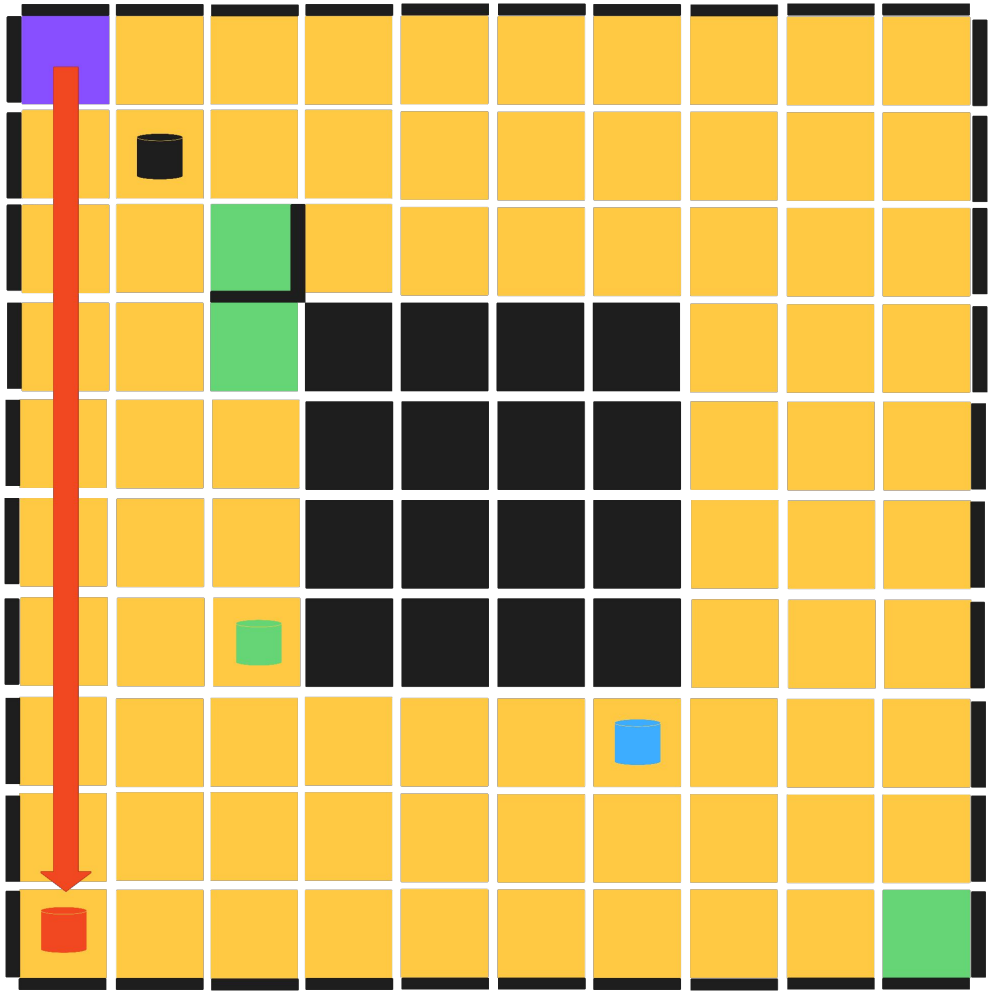
Order:

1. c33
2. c34
3. c1010

1 - move_down red c11 c10_1

Legend

GOAL



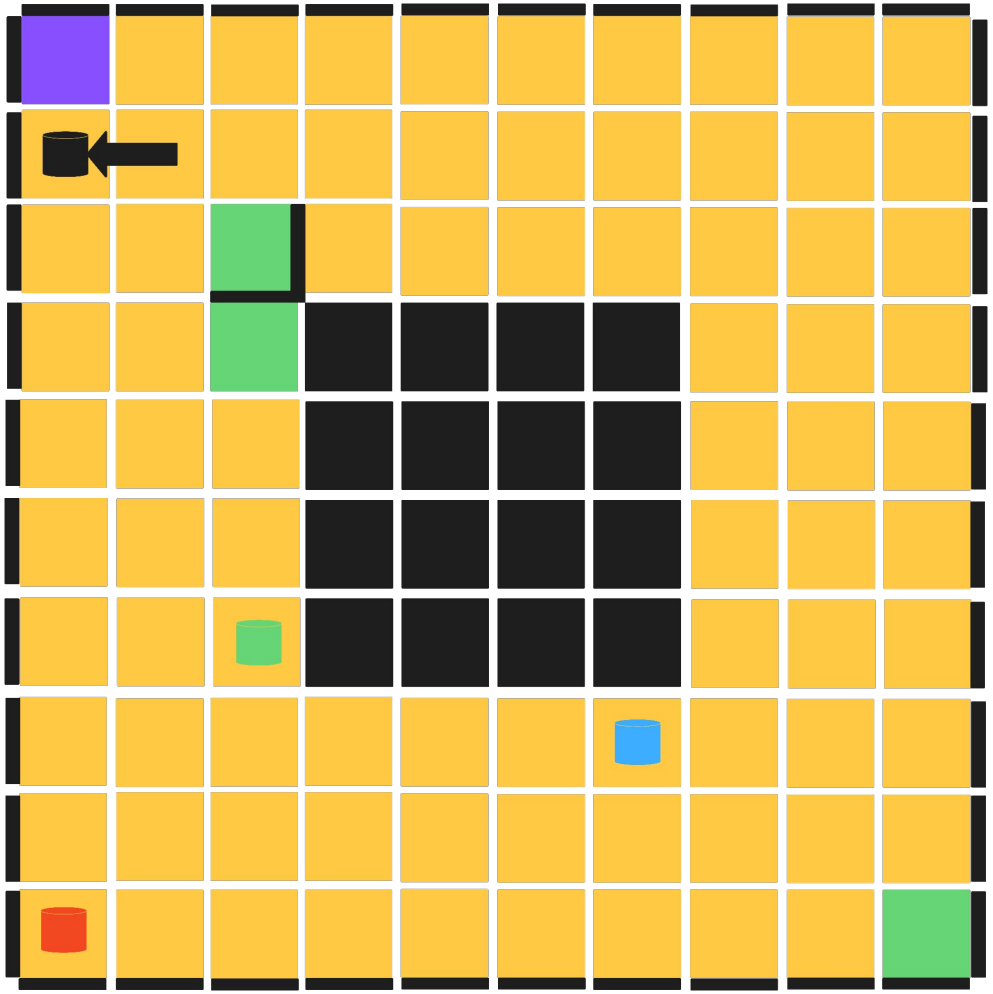
Order:

1. c33
2. c34
3. c1010

2 - move_left black c22 c21

Legend

GOAL



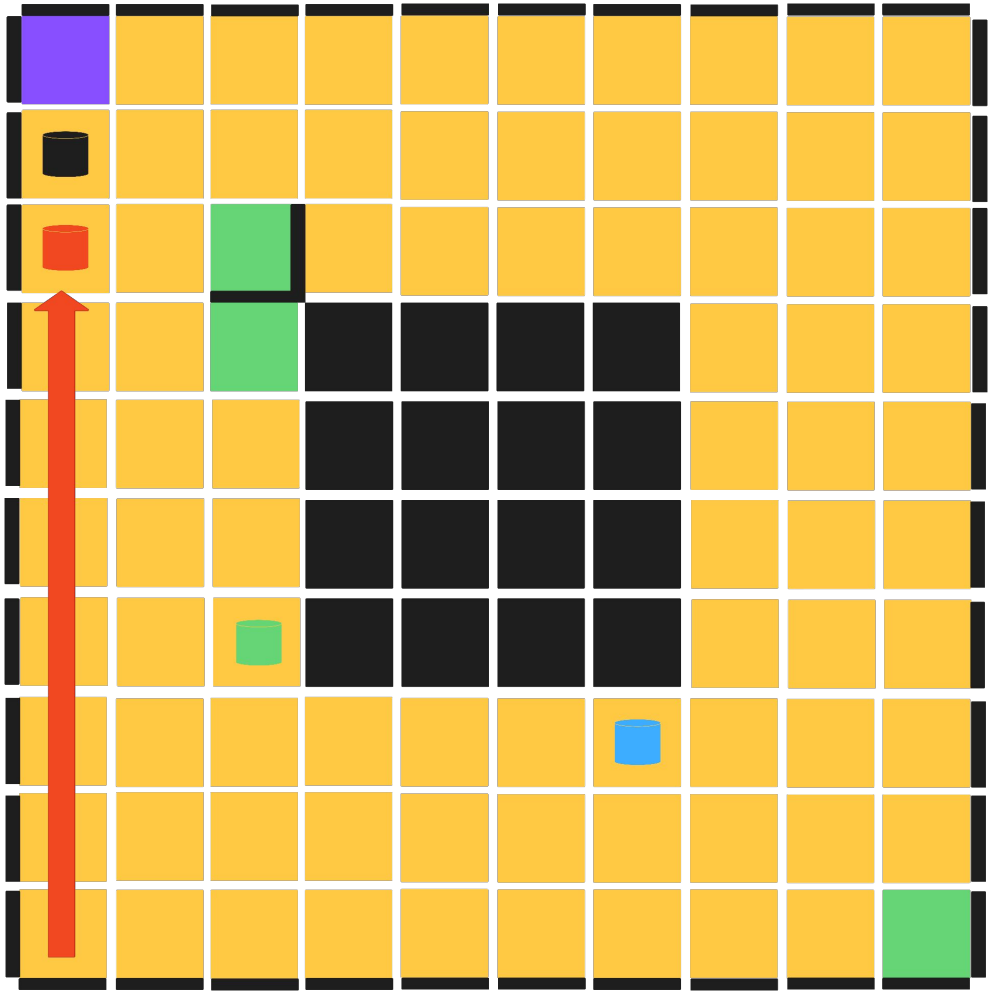
Order:

1. c33
2. c34
3. c1010

3 - move_up red c10_1 c31

Legend

GOAL



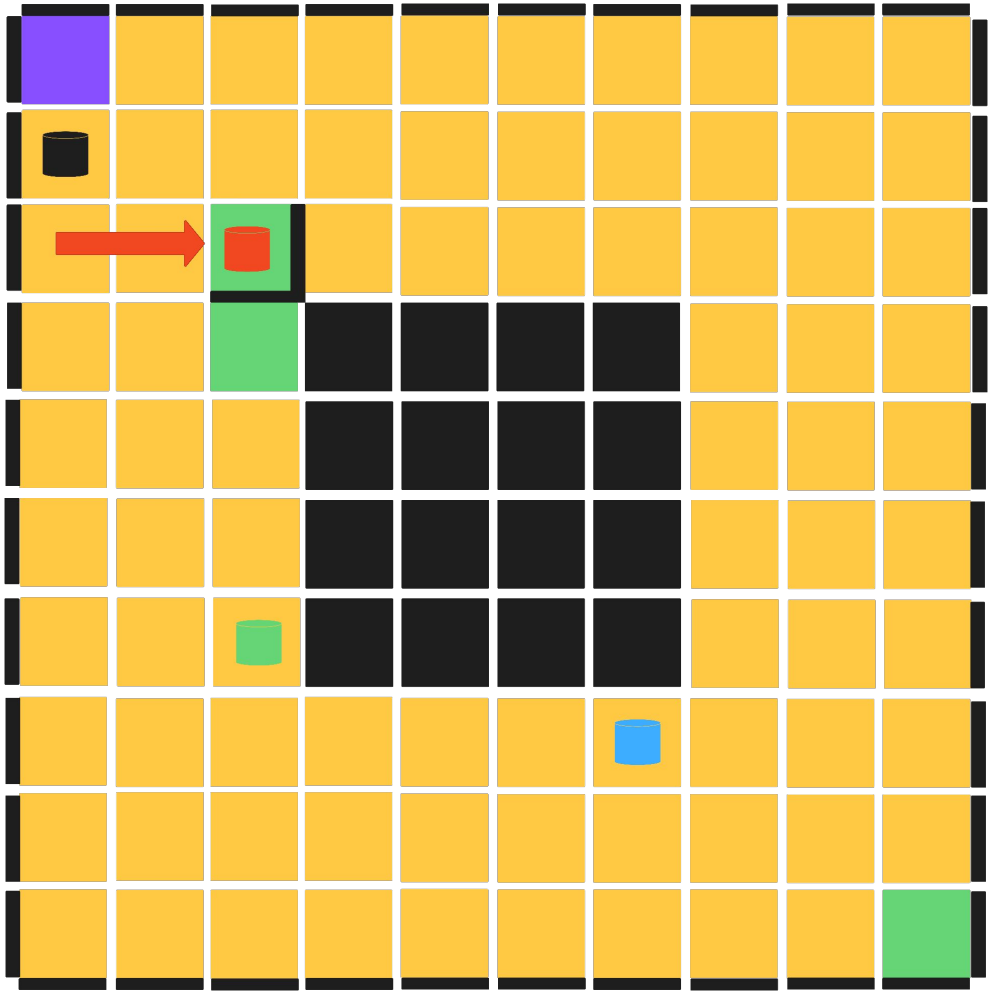
Order:

1. c33
2. c34
3. c1010

4 - move_right red c31 c33

Legend

GOAL



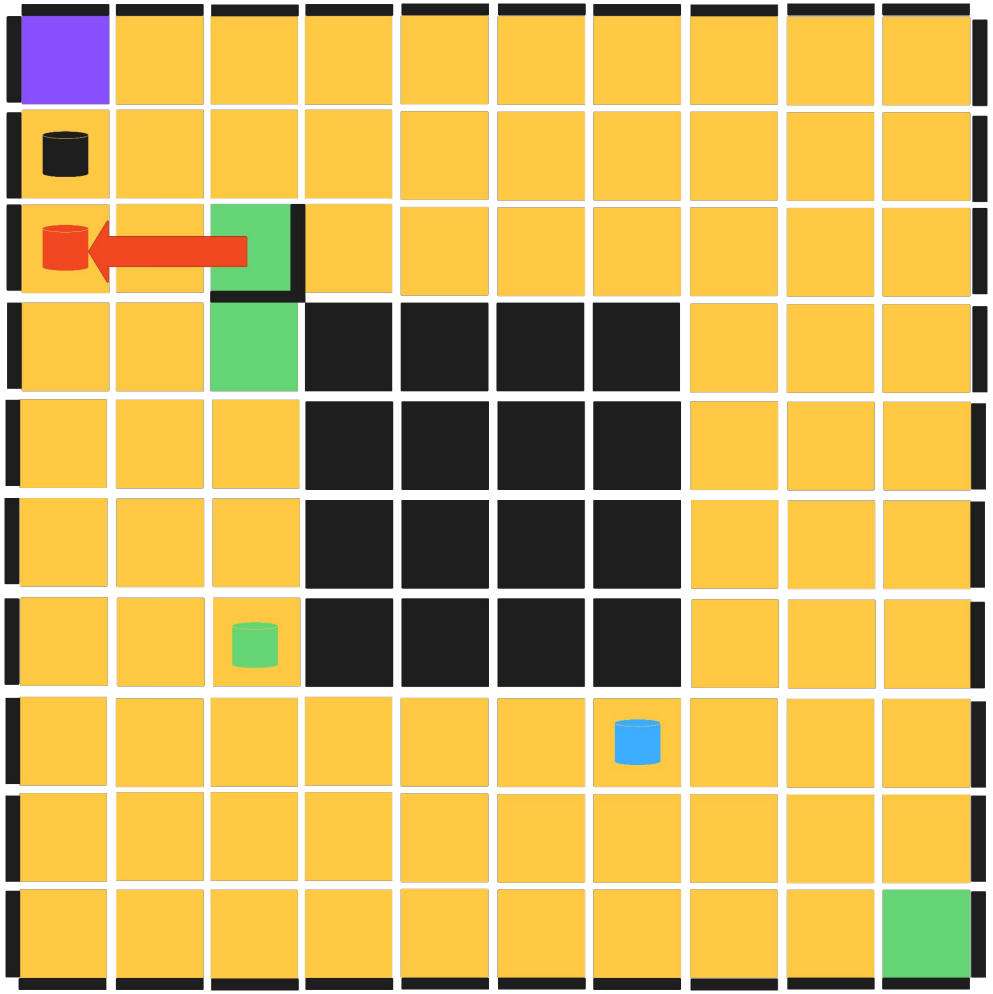
Order:

1. c33
2. c34
3. c1010

5 - move_left red c33 c31

Legend

GOAL



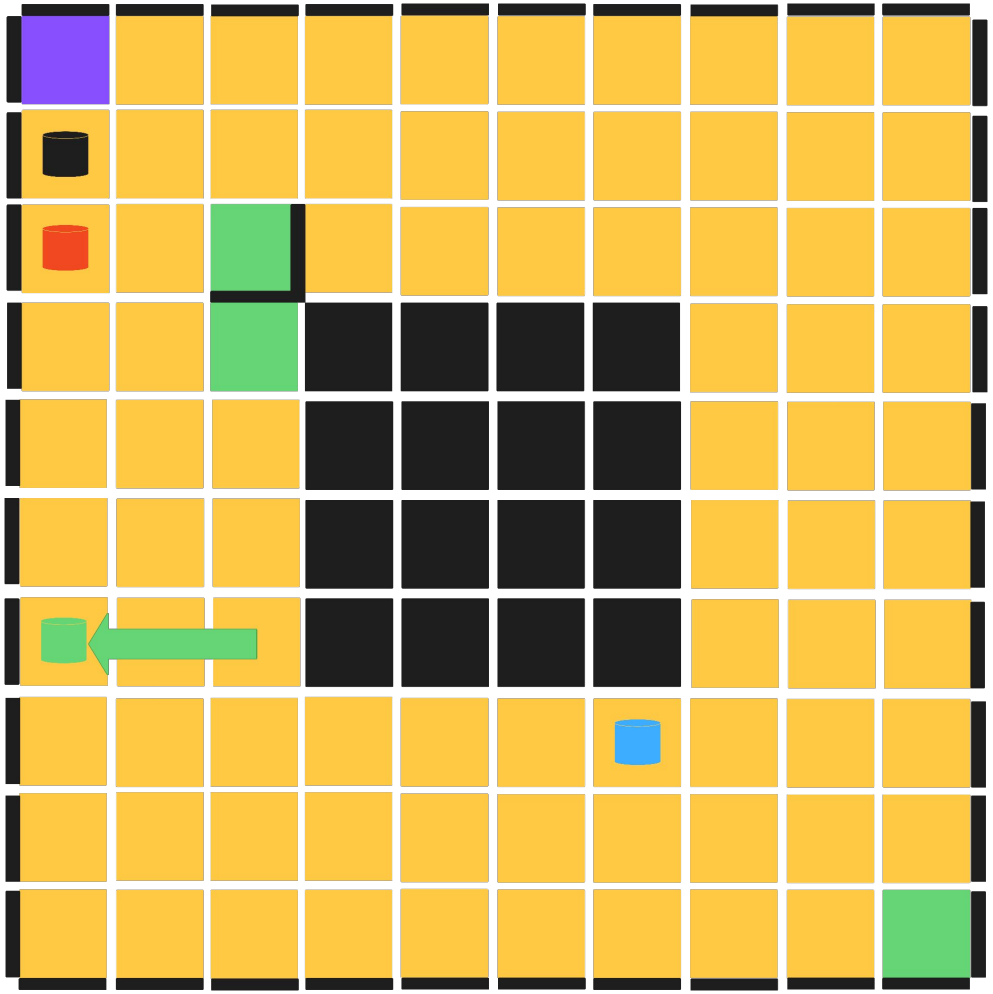
Order:

1. c33
2. c34
3. c1010

6 - move_left green c73 c71

Legend

GOAL



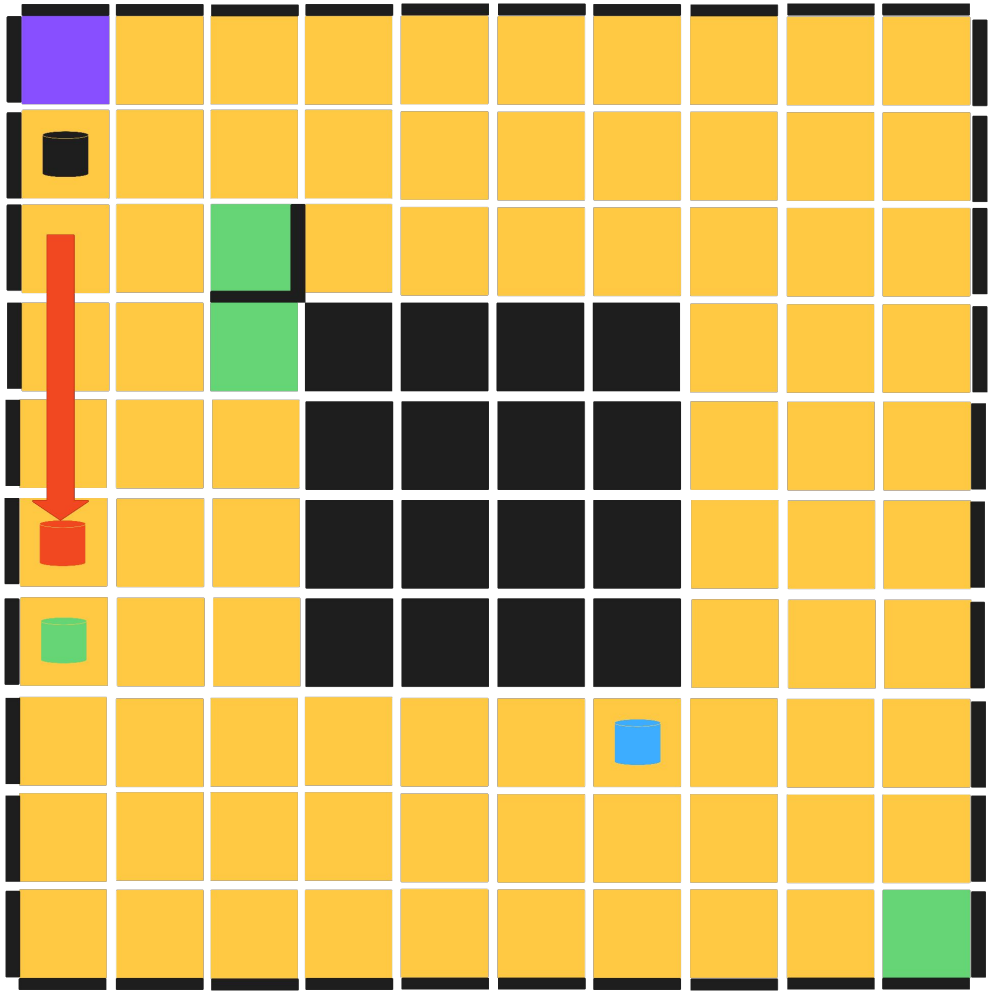
Order:

1. c33
2. c34
3. c1010

7 - move_down red c31 c61

Legend

GOAL



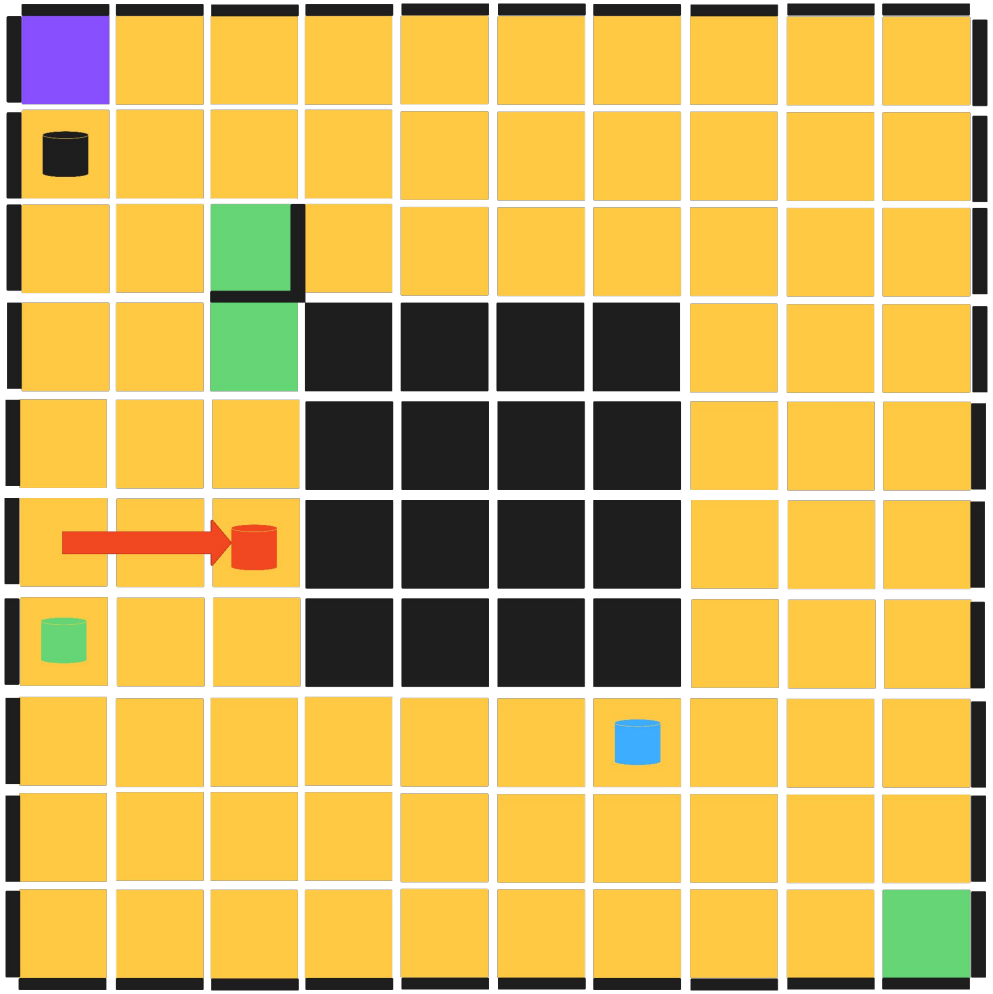
Order:

1. c33
2. c34
3. c1010

8 - move_right red c61 c63

Legend

GOAL



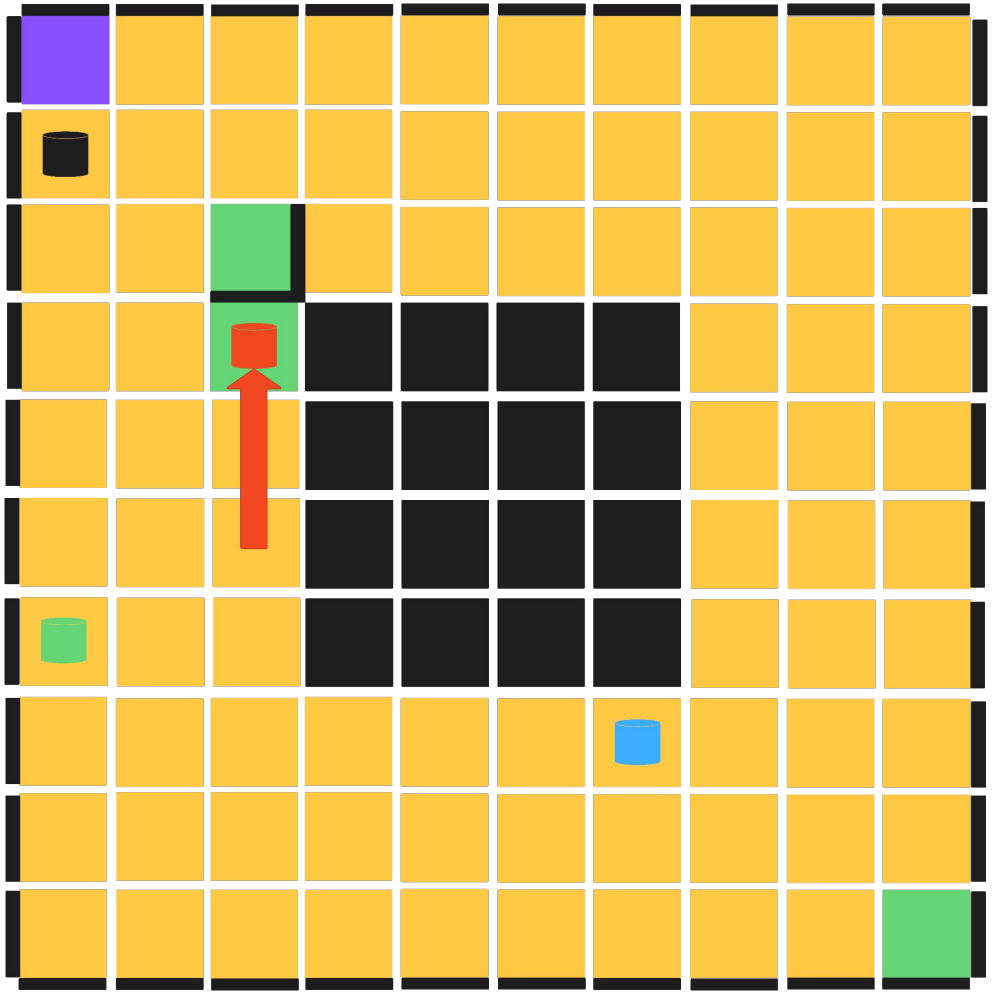
Order:

1. c33
2. c34
3. c1010

9 - move_up red c63 c43

Legend

GOAL



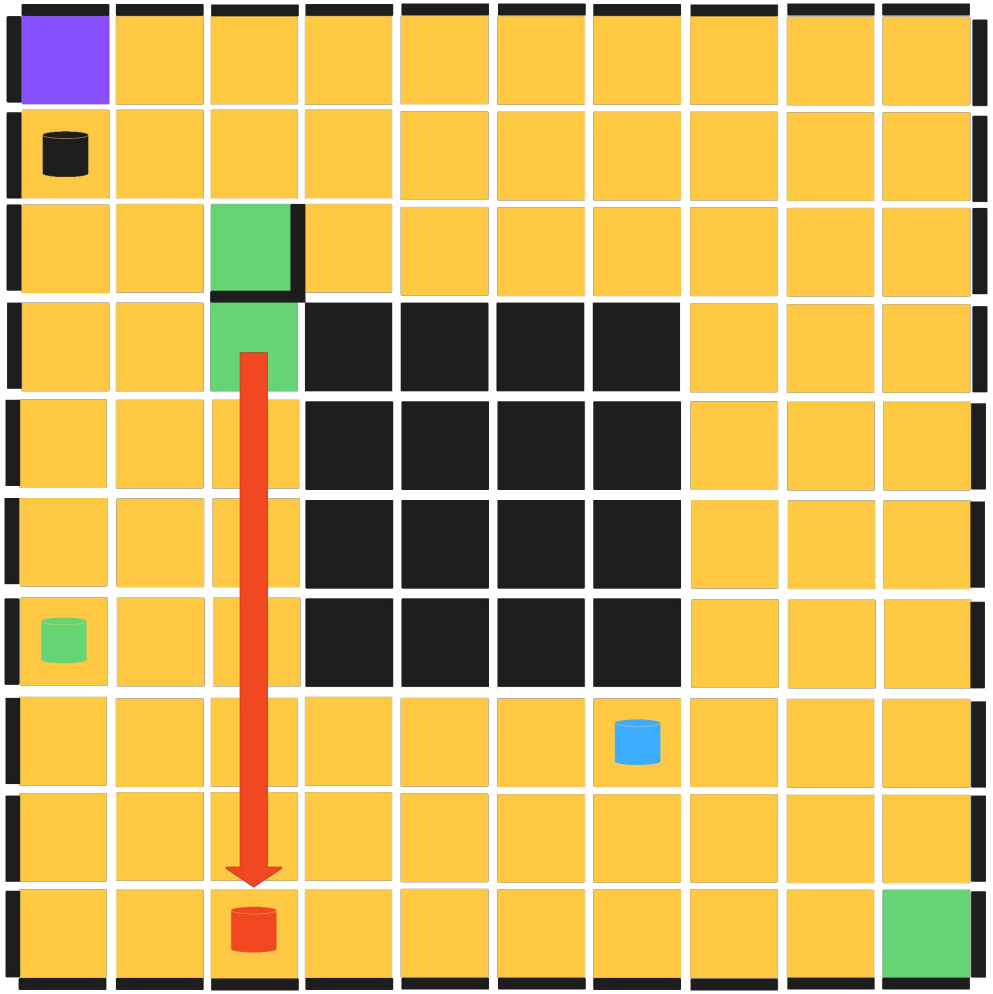
Order:

1. c33
2. c34
3. c1010

10 - move_down red c43 c10_3

Legend

GOAL



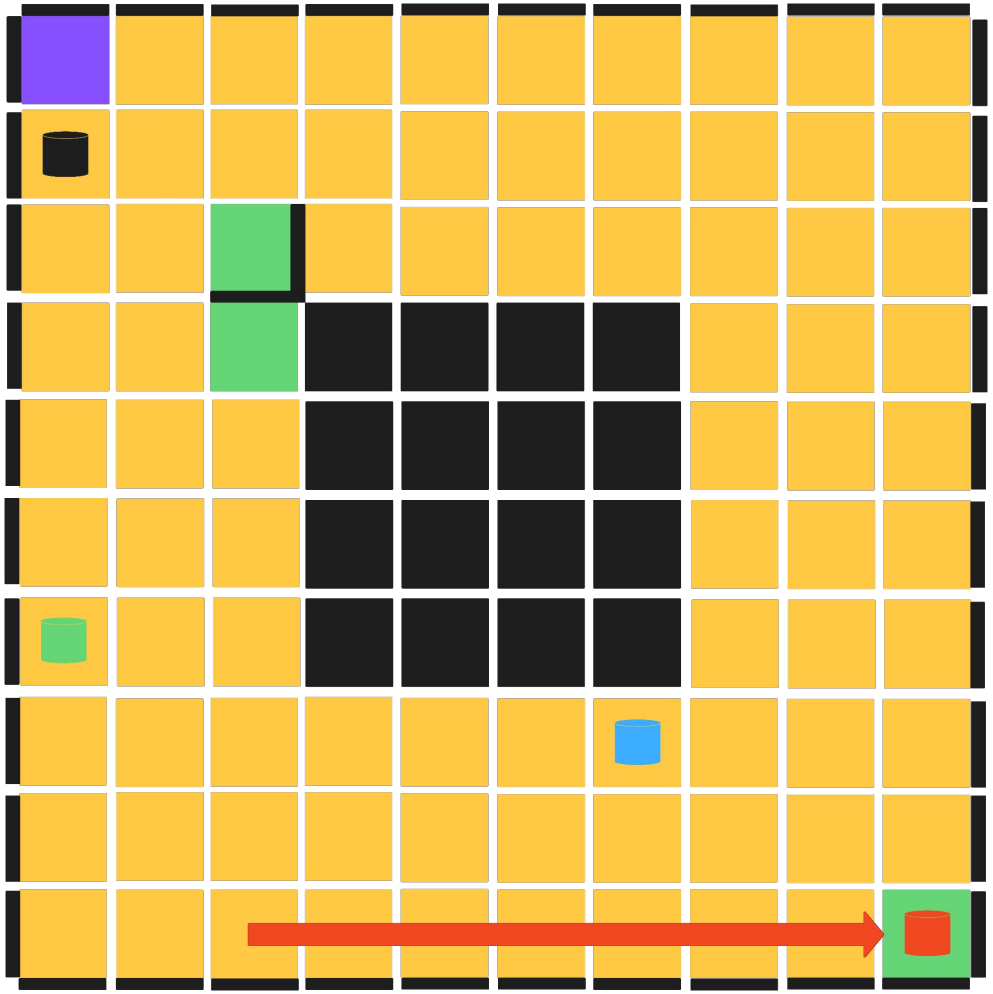
Order:

1. c33
2. c34
3. c1010

11 - move_right red c10_3 c1010

Legend

GOAL



Order:

1. c33
2. c34
3. c1010

```
(:goal(and(visited red c33)(visited red c43)(visited red c1010)))
```

With order	FD A* h^{max}	FD A* h^{add}	ENHSP A* h^{max}	ENHSP GBFS h^{add}
Plan Length	**11**	17	**11**	oom
States evaluated	11993	**3436**	68955	oom
Expanded nodes	3471	**521**	12190	oom
Search Time (s)	3.35	**0.78**	178	oom
Total Time (s)	8.11	**5.58**	188	oom

No order	FD A* h^{max}	FD A* h^{add}	ENHSP A* h^{max}	ENHSP GBFS h^{add}
Plan Length	**10**	13	**10**	oom
States evaluated	32323	**876**	249685	oom
Expanded nodes	8995	**125**	39502	oom
Search Time (s)	7.8	**0.18**	493	oom
Total Time (s)	12.61	**5.06**	503	oom

```
(:goal(and(visited red c33)(visited red c43)))
```

With order	FD A* h^{max}	FD A* h^{add}	ENHSP A* h^{max}	ENHSP GBFS h^{add}
Plan Length	**9**	**9**	**9**	**9**
States evaluated	5142	718	26567	**107**
Expanded nodes	1295	134	4087	**11**
Search Time (s)	1.26	**0.15**	57.39	0.460
Total Time (s)	6.08	**4.99**	66	8.11

No order	FD A* h^{max}	FD A* h^{add}	ENHSP A* h^{max}	ENHSP GBFS h^{add}
Plan Length	**8**	9	**8**	**8**
States evaluated	3651	751	18643	**136**
Expanded nodes	811	141	2640	**15**
Search Time (s)	0.84	**0.15**	40.7	0.46
Total Time (s)	5.66	**4.89**	50.1	8

Ranking the approaches



Fast Downward with A* and h^{max}:

An optimal solution is guaranteed with low execution time



Fast Downward with A* and h^{add}:

The fastest, but can return a sub-optimal solution when complex



ENHSP with A* and h^{max}:

An optimal solution is guaranteed but really slow



ENHSP with GBFS and h^{add}:

Works only in very simple scenarios. Get easily stuck in local min

MULTIPLE GOALS WITH ORDER AND LIMITED ENERGY

Model extensions

Functions:

(battery_level ?r - robot)

(max_battery_level)

(necessary_battery_level)

Predicate:

(charging_station_cell ?c - cell)

Added to all action preconditions (main conjunction):

```
(>= (battery_level ?r)(necessary_battery_level))
```

Added to all action effects (main conjunction):

```
(decrease (battery_level ?r)(necessary_battery_level))
```

Adding recharge action

```
(:action recharge
  :parameters(?r - robot ?c - cell)
  :precondition(and
    (robot_at ?r ?c)
    (charging_station_cell ?c)
  )
  :effect(and
    (assign (battery_level ?r)(max_battery_level))
  )
)
```

Numeric Fluents

FastDownward does not support :numeric-fluents or numeric operators like >=, assign, decrease.

Then, to model the battery as a counter and preserve the original idea, a numeric planner (**ENHSP**) is required.

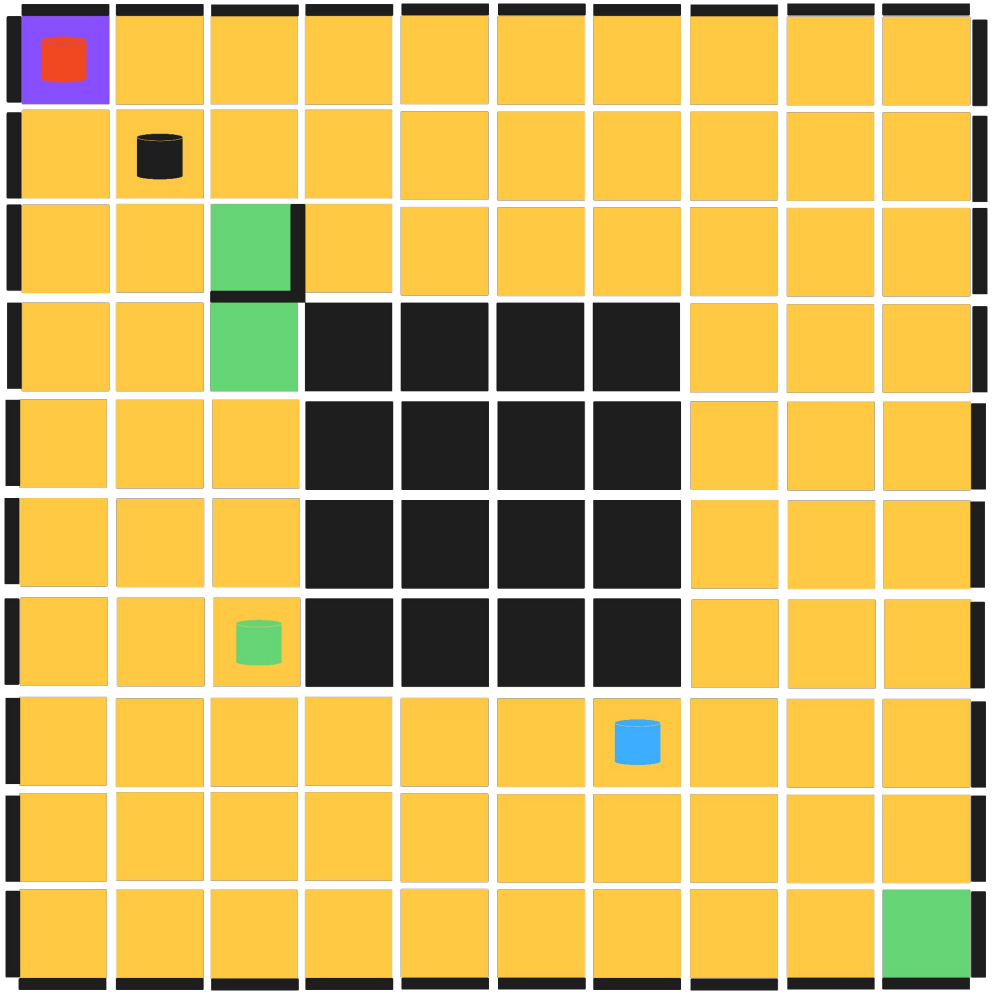
In FastDownward, the alternative would be to use an unary predicate (?r) for each battery level, managing the transition logic between them with conditional effects.

Initial state

Legend

CHARGING STATION

GOAL



Order:

1. c33
2. c34
3. c1010

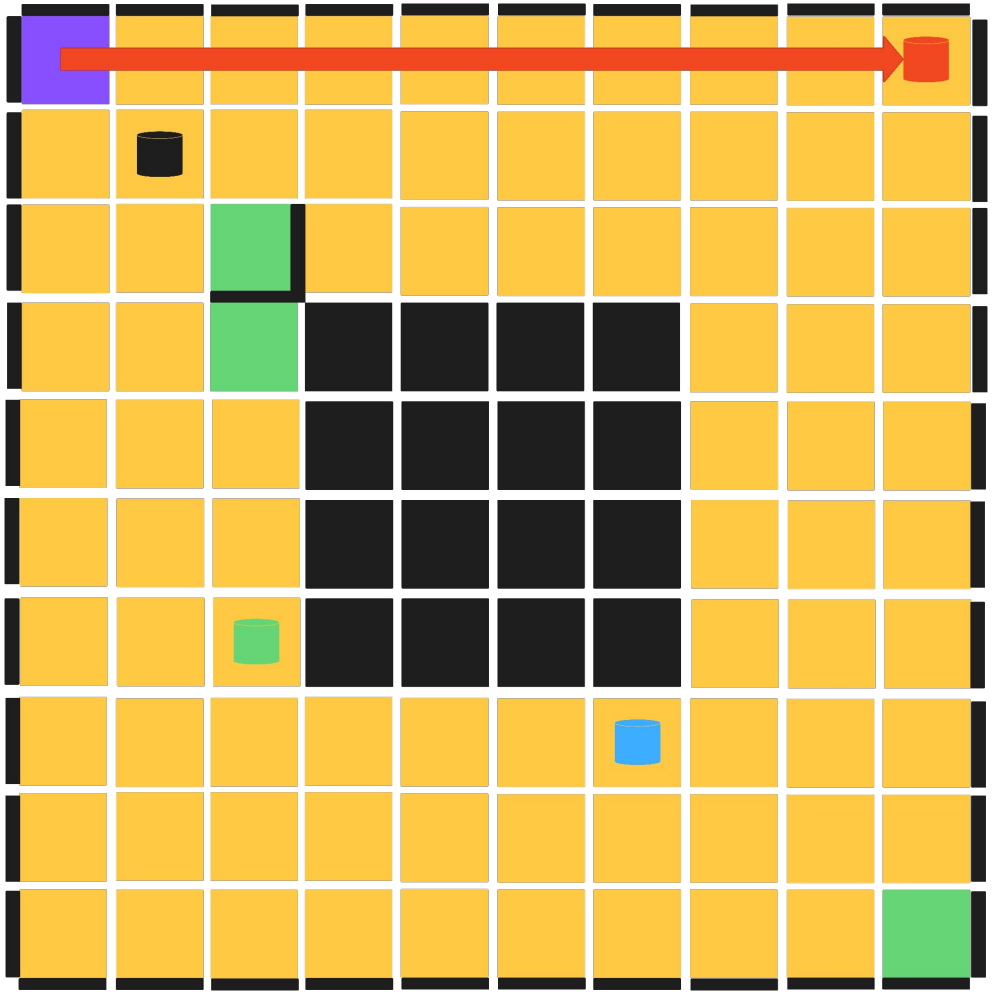
Red Battery:6
Black Battery:4
Green Battery:4
Blue Battery:4

1 - move_right red c11 c1_10

Legend

CHARGING STATION

GOAL



Order:

1. c33
2. c34
3. c1010

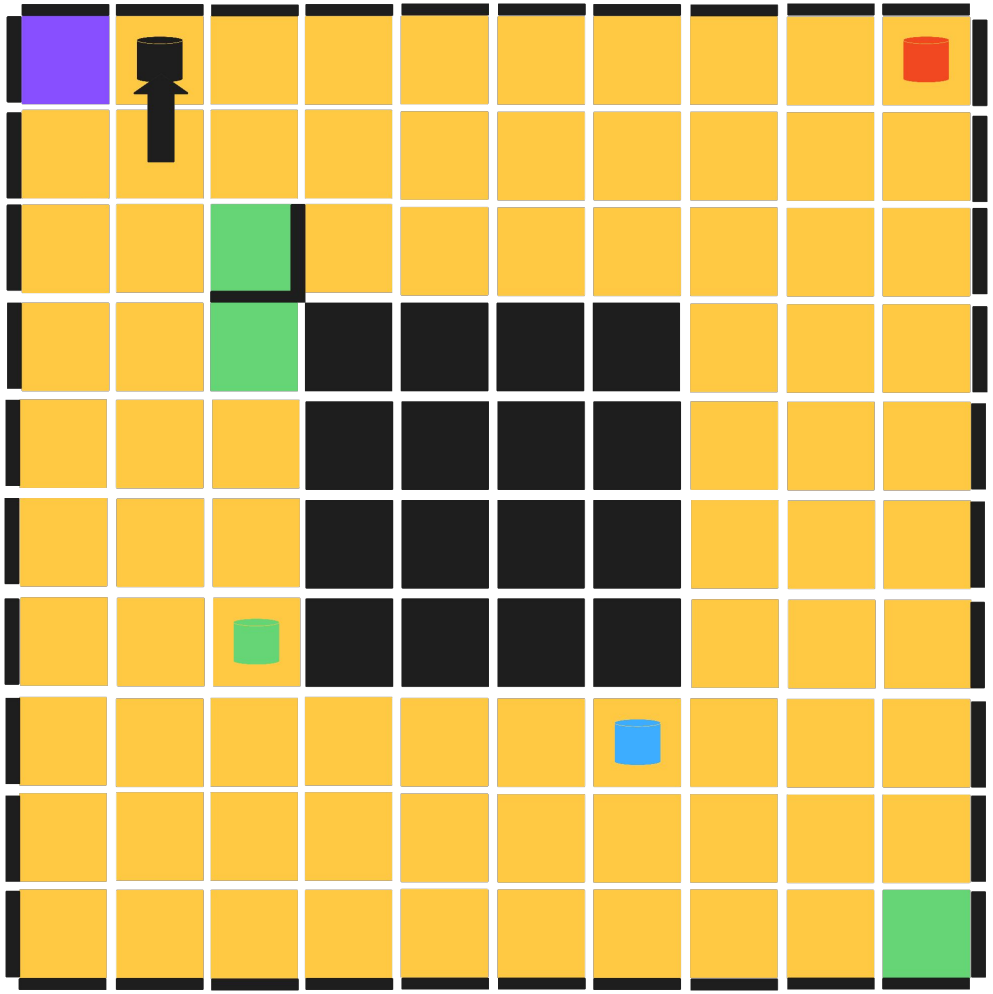
Red Battery:5
Black Battery:4
Green Battery:4
Blue Battery:4

2 - move_up black c22 c12

Legend

CHARGING STATION

GOAL



Order:

1. c33
2. c34
3. c1010

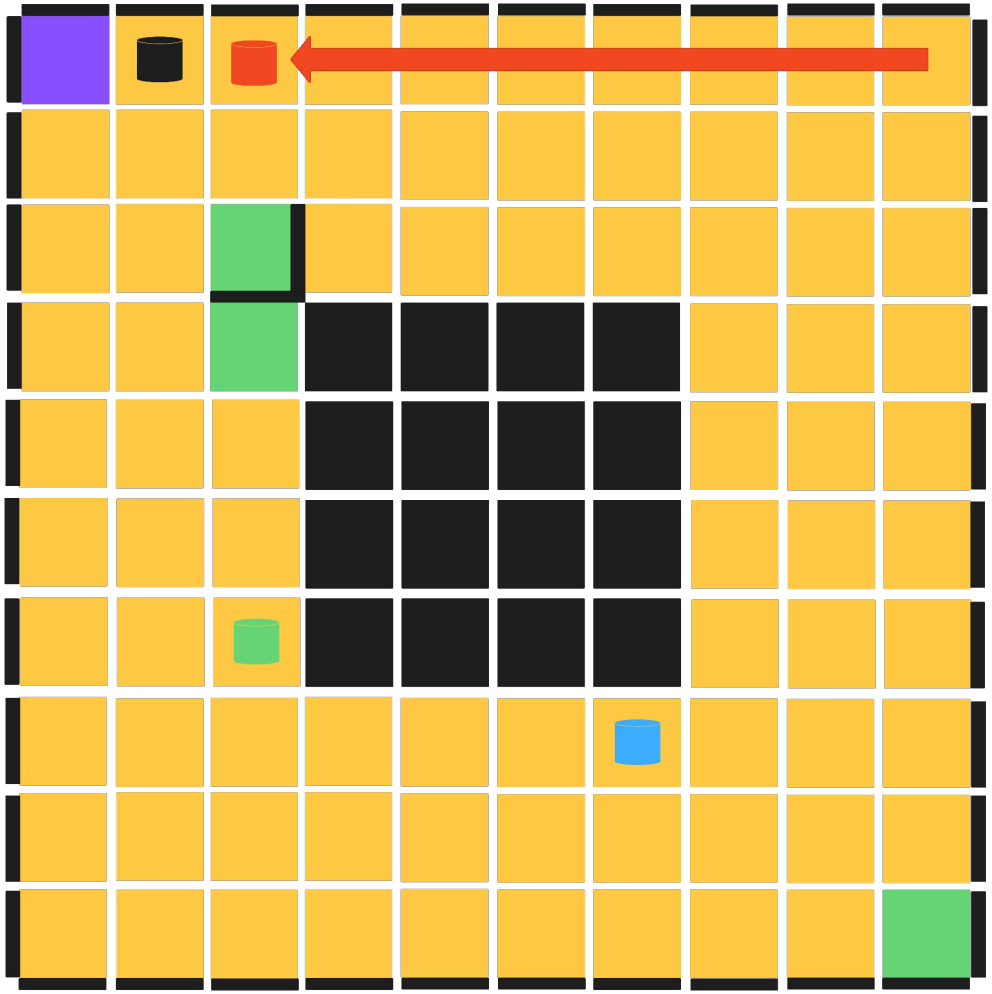
Red Battery:5
Black Battery:3
Green Battery:4
Blue Battery:4

3 - move_left red c10_1 c13

Legend

CHARGING STATION

GOAL



Order:

1. c33
2. c34
3. c1010

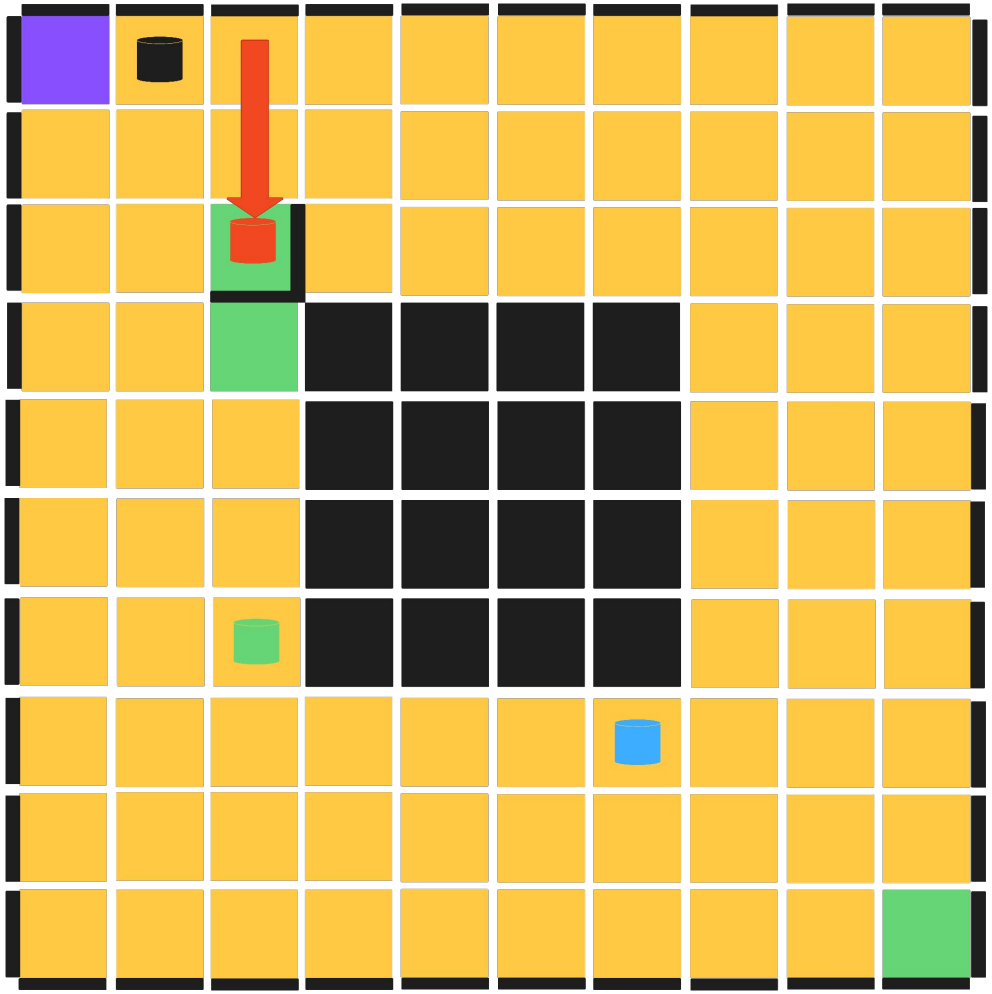
Red Battery:4
Black Battery:3
Green Battery:4
Blue Battery:4

4 - move_down red c13 c33

Legend

CHARGING STATION

GOAL



Order:

1. c33
2. c34
3. c1010

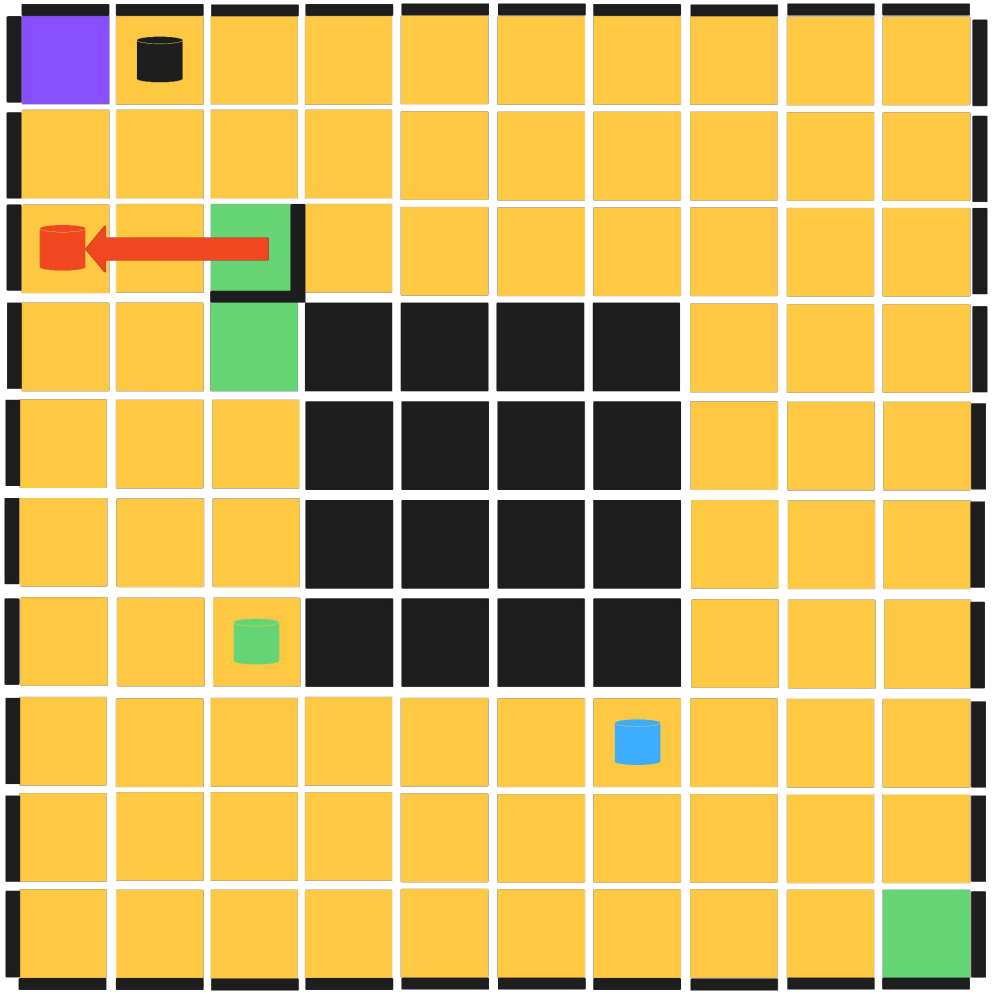
Red Battery:3
Black Battery:3
Green Battery:4
Blue Battery:4

5 - move_left red c33 c31

Legend

CHARGING STATION

GOAL



Order:

1. c33
2. c34
3. c1010

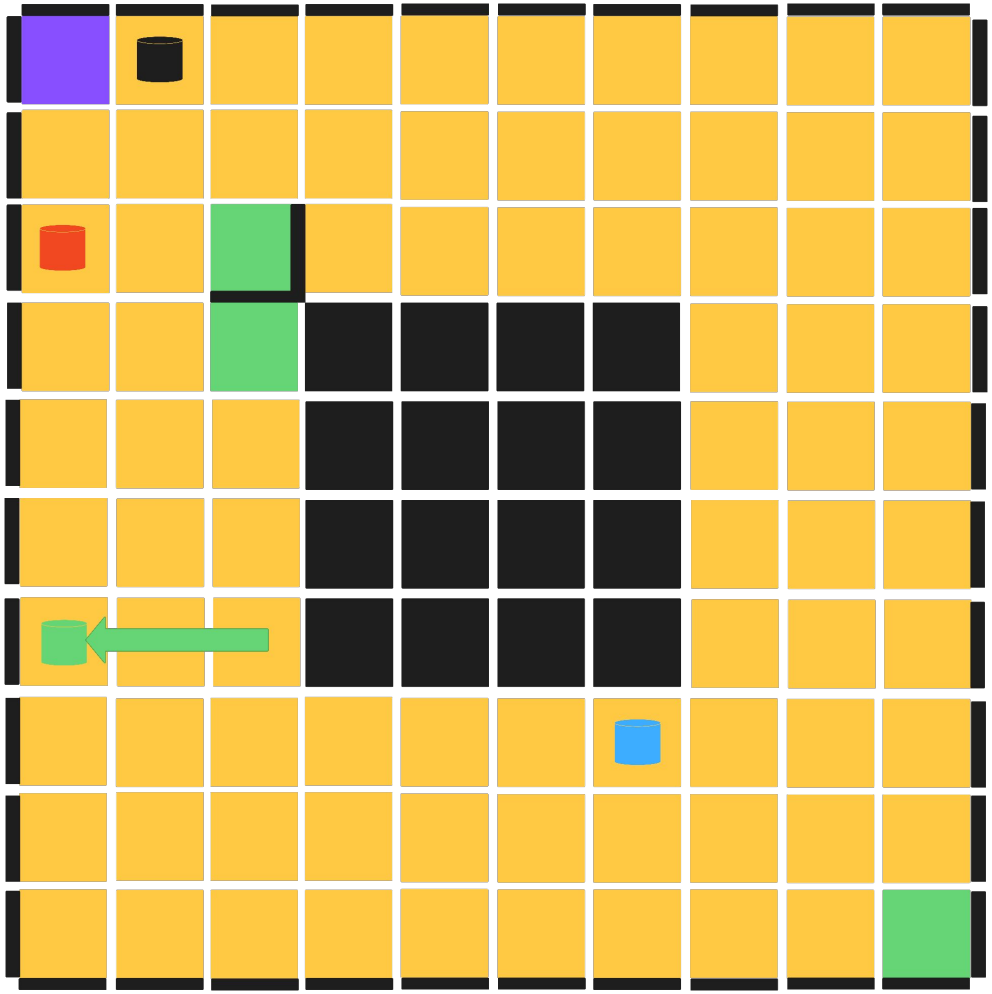
Red Battery:2
Black Battery:3
Green Battery:4
Blue Battery:4

6 - move_left green c73 c71

Legend

CHARGING STATION

GOAL



Order:

1. c33
2. c34
3. c1010

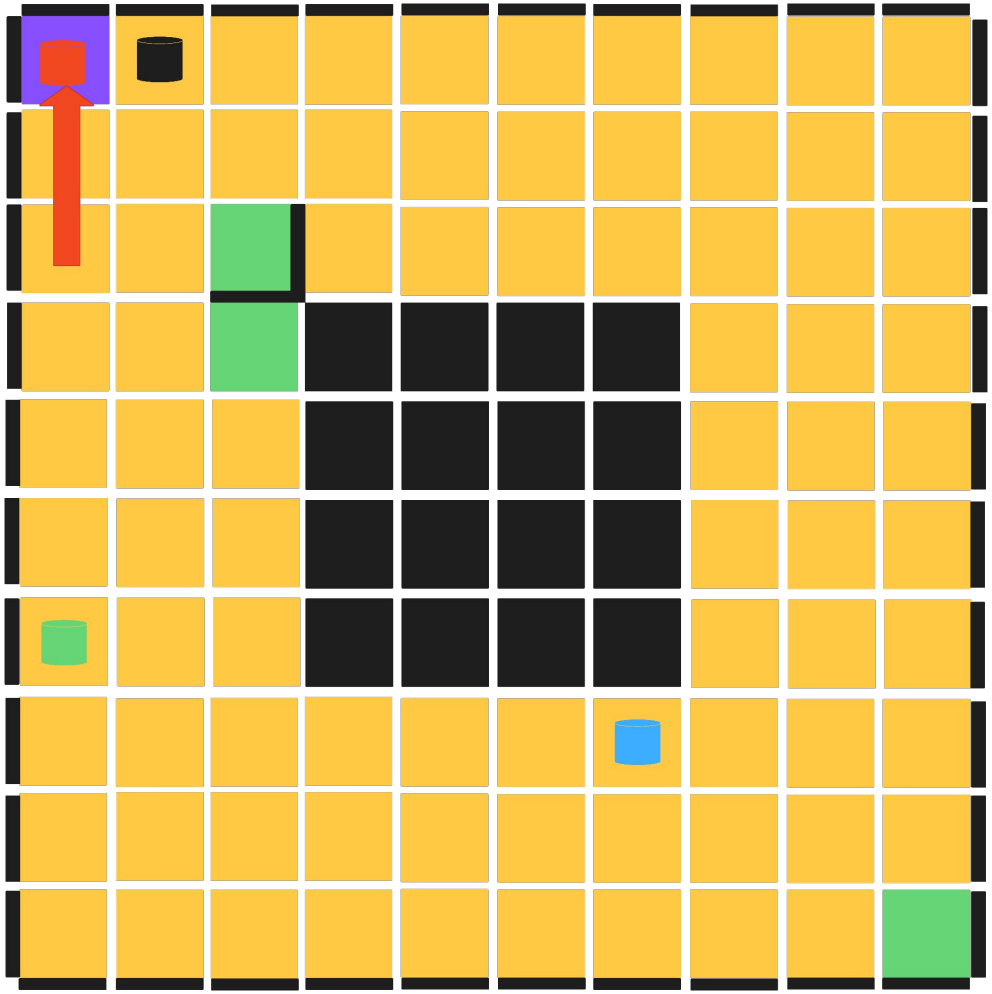
Red Battery:2
Black Battery:3
Green Battery:3
Blue Battery:4

7 - move_up red c31 c11

Legend

CHARGING STATION

GOAL



Order:

1. c33
2. c34
3. c1010

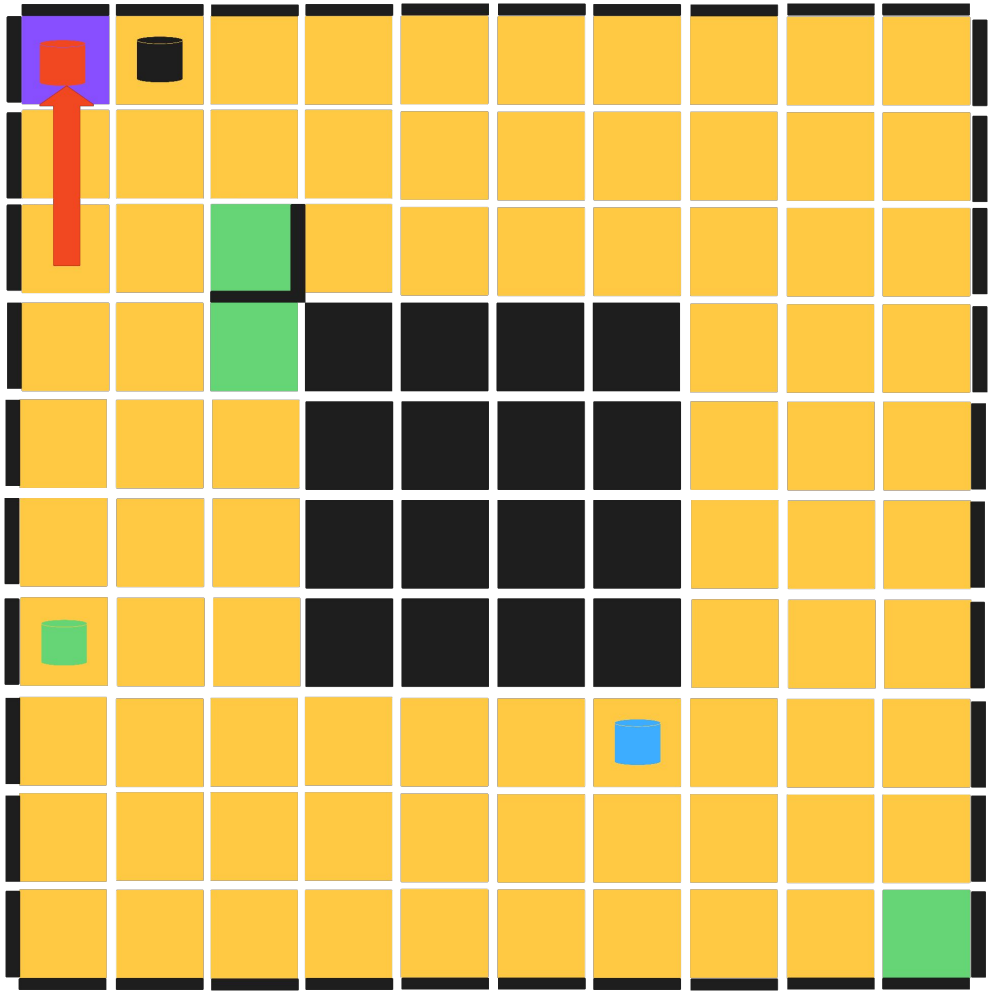
Red Battery:1
Black Battery:3
Green Battery:3
Blue Battery:4

8 - recharge red c11

Legend

CHARGING STATION

GOAL



Order:

1. c33
2. c34
3. c1010

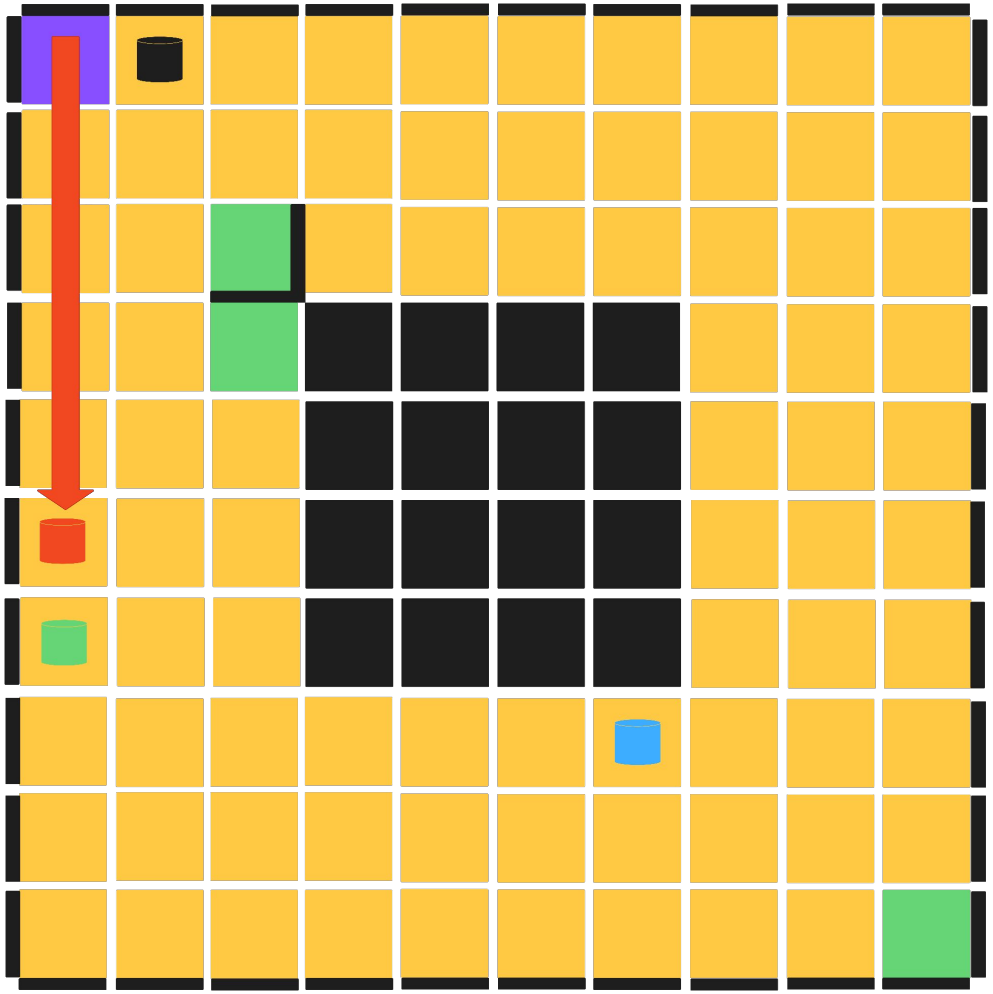
Red Battery:6
Black Battery:3
Green Battery:3
Blue Battery:4

9 - move_down red c11 c61

Legend

CHARGING STATION

GOAL



Order:

1. c33
2. c34
3. c1010

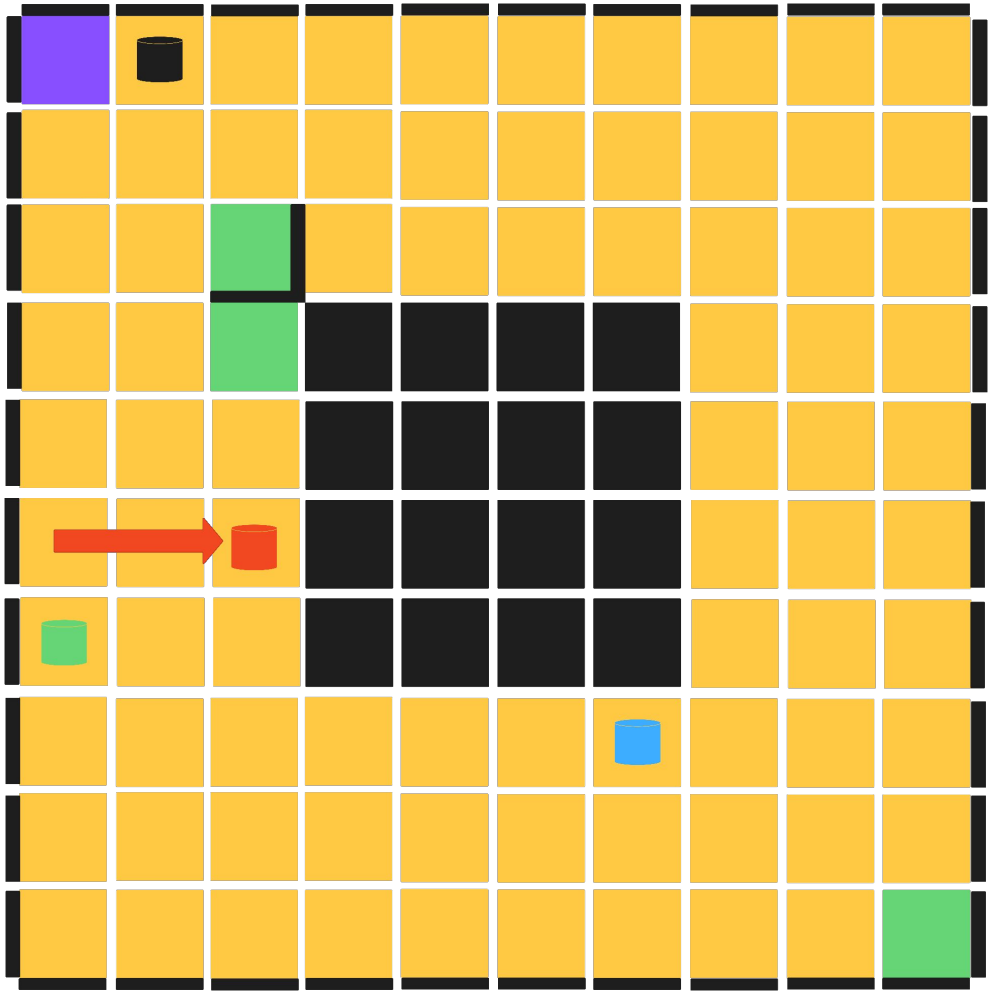
Red Battery:5
Black Battery:3
Green Battery:3
Blue Battery:4

10 - move_right red c61 c63

Legend

CHARGING STATION

GOAL



Order:

1. c33
2. c34
3. c1010

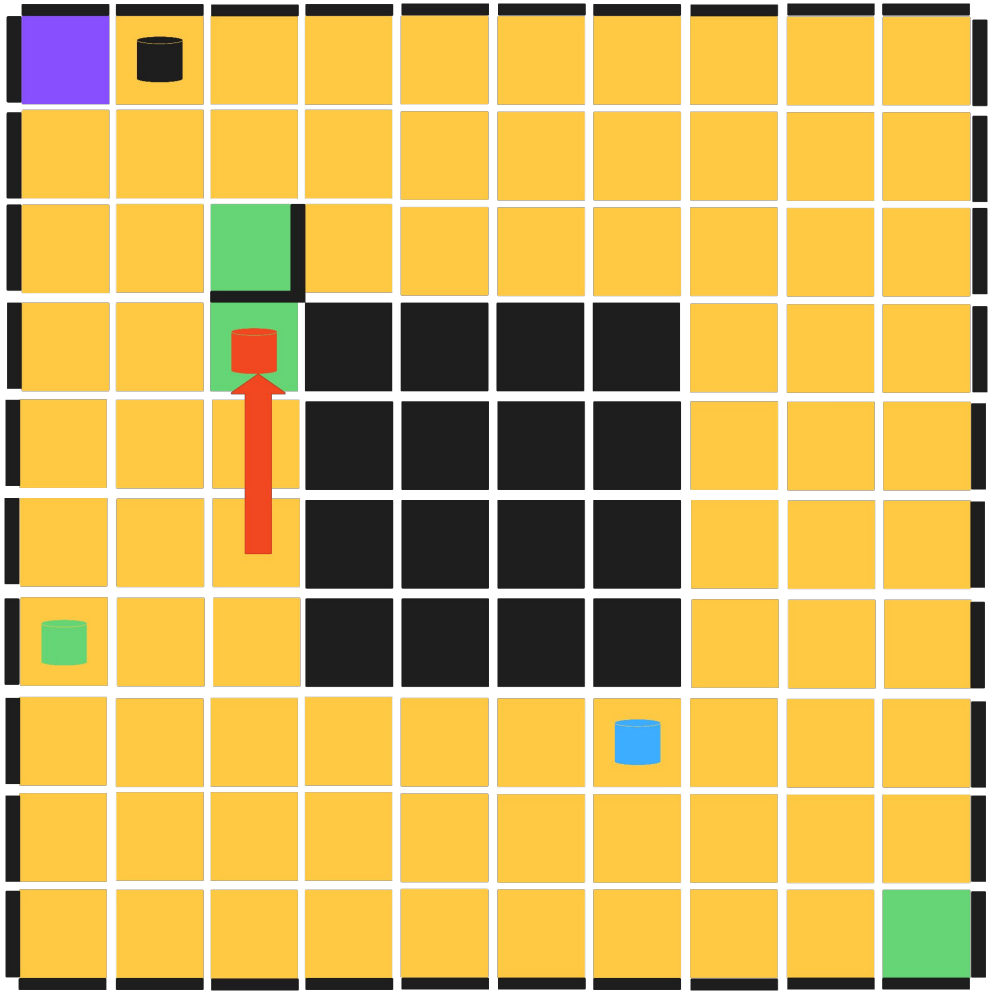
Red Battery:4
Black Battery:3
Green Battery:3
Blue Battery:4

11 - move_up red c63 c43

Legend

CHARGING STATION

GOAL



Order:

1. c33
2. c34
3. c1010

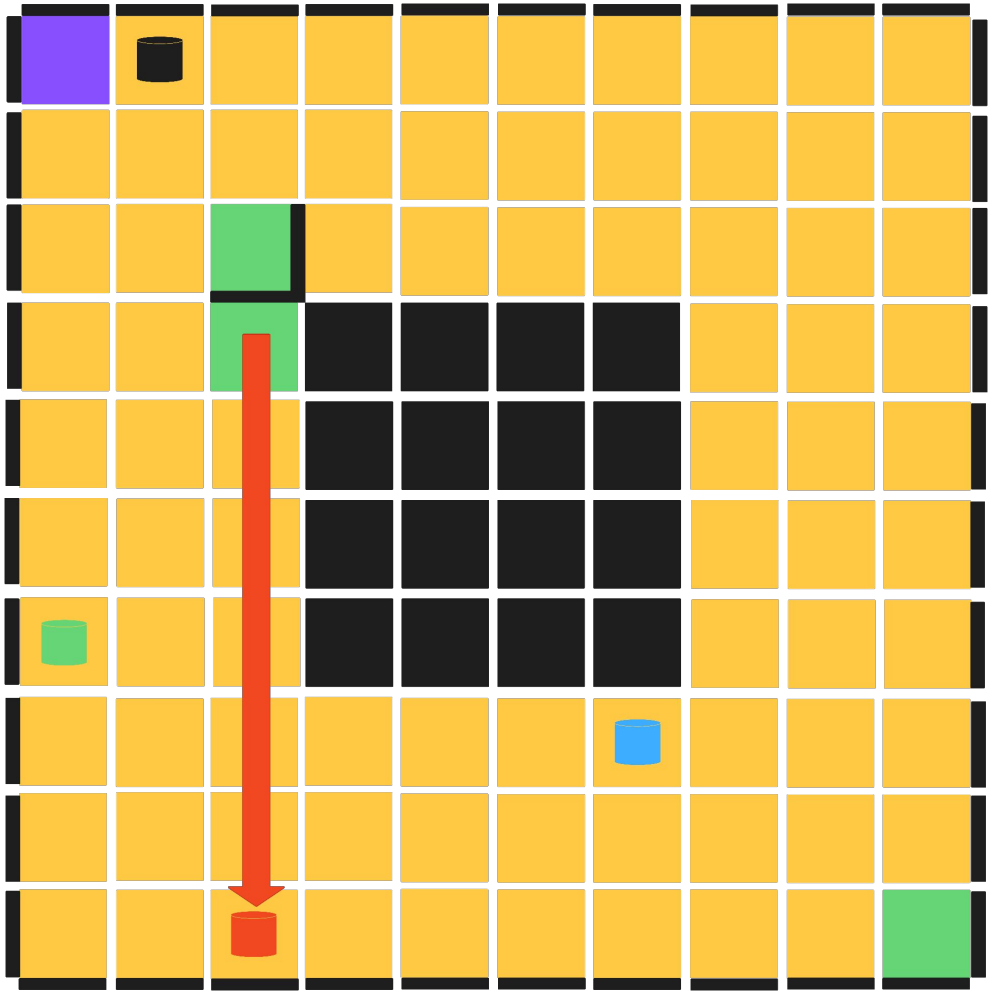
Red Battery:3
Black Battery:3
Green Battery:3
Blue Battery:4

12 - move_down red c43 c10_3

Legend

CHARGING STATION

GOAL



Order:

1. c33
2. c34
3. c1010

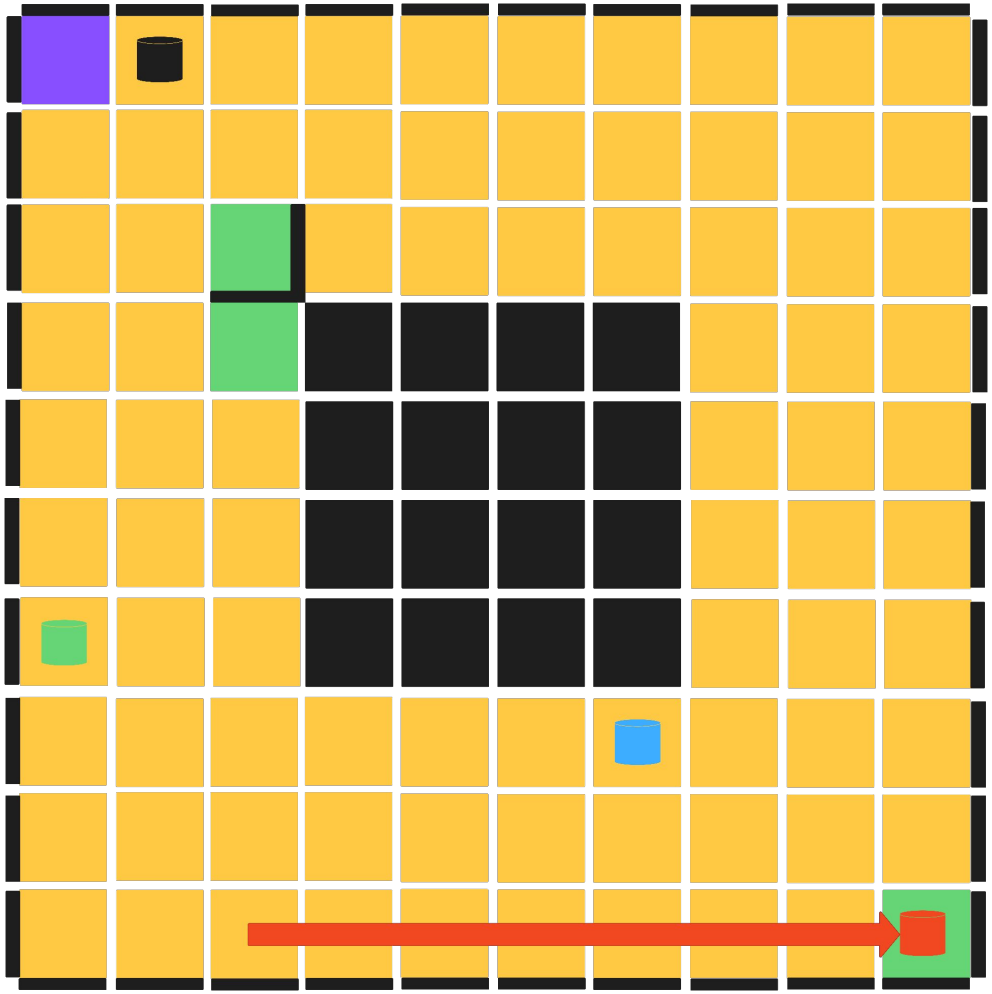
Red Battery:2
Black Battery:3
Green Battery:3
Blue Battery:4

13 - move_right red c10_3 c1010

Legend

CHARGING STATION

GOAL



Order:

1. c33
2. c34
3. c1010

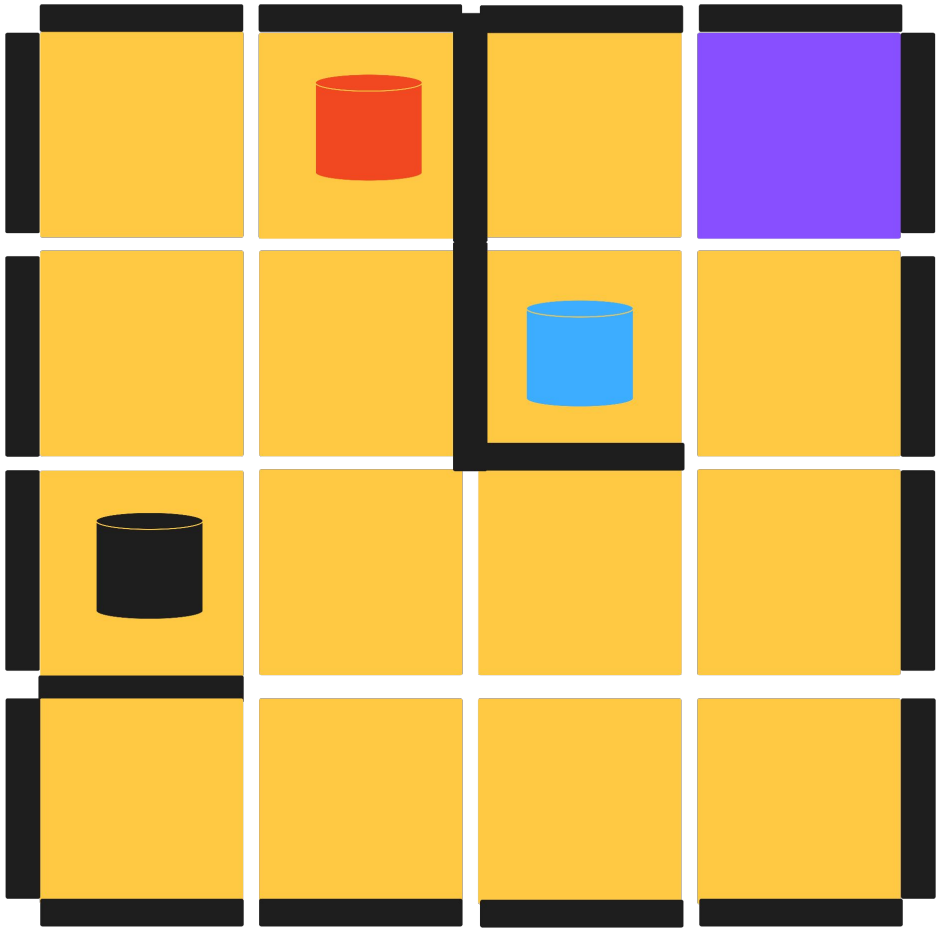
Red Battery:1
Black Battery:3
Green Battery:3
Blue Battery:4

INDIGOLOG

Instance semplification

To implement the problem in IndiGolog, we needed to simplify our domain.

We reduced the grid dimension and the number of objects.



Fluents

- **pos(R)**: states the current position ID (1-16) of robot R.
  fun_fluent(pos(R)) :- robot(R).

- **battery_red**: tracks the remaining battery level of the red robot.
  fun_fluent(battery_red).

- **move_count**: counters the total number of moves performed.
  fun_fluent(move_count).

- **active_goals**: list of currently active target IDs (used by the reactive controller).
  fun_fluent(active_goals).

- **goal_active(N)**: relational fluent stating that location N is an active target.
  rel_fluent(goal_active(N)).

- **Auxiliary flags**: boolean fluents used to handle exogenous events and interrupts.
  rel_fluent(new_goal_event).
  rel_fluent(stop_requested).
  rel_fluent(robot_config_changed).

Actions

- **move(R, D)**: robot R attempts to move in direction D (n, s, e, w).
  prim_action(move(R,D)) :- robot(R), dir(D).
  poss(move(red,_), and(pos(red)>0, neg(battery_red=0))).
  poss(move(R,_), pos(R)>0) :- robot(R), R \= red.

- **recharge**: the red robot recharges its battery to the maximum level (10).
  prim_action(recharge).
  poss(recharge, and(pos(red)=ID, and(station_id=ID, neg(battery_red=10)))).

Exogenous actions & auxiliary action

Exogenous Actions:

- **add_goal(N)**: an external event adds a new target at position N.
  exog_action(add_goal(N)) :- target_id(N).

- **spawn_robot(R, N) / despawn_robot(R)**: external events that modify the board configuration by adding or removing robots.
  exog_action(spawn_robot(R,N)) :- robot(R), target_id(N).
  exog_action(despawn_robot(R)) :- robot(R).

- **stop_game**: an external signal requests the agent to stop execution.
  exog_action(stop_game).

Auxiliary Actions (Controller Internal):

- **dismiss_goal(N)**: the agent internally marks goal N as completed.
  prim_action(dismiss_goal(N)) :- target_id(N).

- **unset_new_goal_event**: resets the event flag after handling a new goal.
  prim_action(unset_new_goal_event).

- **wait_exog_action**: the agent waits idly for an external event to occur.
  prim_action(wait_exog_action).

Move Logic

Unlike standard grid worlds, a move action continues until the robot hits a wall or another robot. We implemented specific predicates to calculate the final position efficiently.

- **ray_ids_until(Start, Dir, StaticStop, Ray)**:
  Generates the list of cell IDs (Ray) from the start position to the nearest wall (StaticStop) in direction Dir.
  ray_ids_until(StartID, D, StaticStopID, Ray) :- ...

- **occ_ids_without(R, ...)**:
  Identifies the positions of all other robots (dynamic obstacles) that are currently on the board.
  occ_ids_without(red, _PR, PB, PBu, [PB, PBu]).

- **dynamic_stop_id(Start, Dir, Occ, Final)**:
  Determines the actual stopping cell by checking the "Ray" against the list of occupied cells ("Occ"). If an obstacle is hit, the robot stops at the previous cell; otherwise, it stops at the wall.

```
dynamic_stop_id(StartID, D, OccIDs, StopID) :-
    static_stop_id(StartID, D, StaticStopID),
    ray_ids_until(StartID, D, StaticStopID, Ray),
    first_hit(Ray, OccIDs, Hit), …
```

Legality Task

The legality task verifies if a given sequence of actions is executable from the initial state. Uniquely to our domain, it checks if the Red robot has enough battery to perform the moves and if the moves actually change the robot's position (i.e., not moving directly into a wall).

```
legality_task :-
    format("Write the sequence of actions '[a1(), ..., an()].' :~n", []),
    read(SEQ), nl,
    (   rr_simulate(SEQ, SF)
    ->  format("Sequence ~w IS EXECUTABLE~n", [SEQ]),
        rr_print_state(SF)
    ;   format("Sequence ~w is NOT executable~n", [SEQ])
    ).
```

Legal Example:

Input: `[move(red, e), move(black, n), move(red, s)]`

Output: Sequence IS EXECUTABLE. Final state: PR=... Bred=8 ...

Illegal Example (Battery Constraint):

Input: `[set_battery(0), move(red, n)]`

Output: Sequence is NOT executable. *(Fails because `rr_executable` checks `B =\= 0` for red moves)*

Projection Task (Battery)

The first projection task verifies whether the Battery Constraint holds after executing a user-provided sequence of actions. It checks if the Red robot still has charge (battery > 0) at the end of the simulation.

```
projection_task_charge :-
    format("Write the sequence of actions '[a1(), ..., an()].' :~n", []),
    read(SEQ), nl,
    (   rr_simulate(SEQ, st(_PR,_PB,_PBu,B,_MC,_SID))
    ->  ( B =\= 0
        -> format("After executing ~w, red STILL HAS CHARGE (battery=~w).~n", [SEQ, B])
        ;  format("After executing ~w, red has NO CHARGE (battery=0).~n", [SEQ])
        )
    ;   format("Sequence ~w is NOT executable.~n", [SEQ])
    ).
```

Example (Battery OK):

Input SEQ: `[move(red, n), move(red, w)]`

Output: After executing ..., red STILL HAS CHARGE (battery=8).

Example (Battery Drained):

Input SEQ: `[set_battery(1), move(red, n)]`

Output: After executing ..., red has NO CHARGE (battery=0).

Projection Task (Goal Reachability)

The second projection task allows the user to specify a Goal Position and a sequence of actions. It projects the state forward to verify if the Red robot ends up exactly at the desired location.

```
projection_task_not_reached :-
   format("Write the goal-position (ID 1..16). Example: 4.~n", []),
   read(GOAL_IN), goal_in_to_id(GOAL_IN, GID),
   format("Write the sequence of actions...~n", []), read(SEQ),
   (   rr_simulate(SEQ, st(PR,_,_,_,_,_))
   ->  ( PR =\= GID
       -> format("Red HAS NOT reached goal ~w. Final PR=~w.~n", [GID, PR])
       ;  format("Red HAS reached goal-position ~w.~n", [GID])
       )
   ;   format("Sequence ~w is NOT executable.~n", [SEQ])
   ).
```

Example (Goal Not Reached):

Input Goal: 4 (Station) **Input SEQ:** `[move(red, s)]` (Assuming Red starts at 2 and hits a wall)

Output: After executing ..., red HAS NOT reached goal 4. Final PR=6.

Example (Goal Reached):

Input Goal: 4 **Input SEQ:** `[...correct sequence...]`

Output: After executing ..., red HAS reached goal-position 4.

Base Controller

The **Base Controller** operates sequentially: it selects a goal, calculates a complete plan, and executes it blindly. It assumes the environment remains static during execution.

Code Limitation: The execution block `serve_goal_plan(n)` is atomic. There are no guards monitoring the environment state during the sequence of moves.

```
% 1. BASE: Finishes the current goal BEFORE listening to new events
proc(control(base_ricochet), [
    prioritized_interrupts([
        interrupt(stop_requested, [ show_report, ?(fail) ]),
        interrupt(some_pending,
            pi(n, [
                ?(and(target_id(n), goal_active(n))),
                unset_new_goal_event,
                % CRITICAL: No guard here. Even if a robot spawns, this continues!
                serve_goal_plan(n)
            ])
        ),
        interrupt(true, ?(wait_exog_action))
    ])
]).
```

Failure Case: If the plan relies on ricocheting off a specific robot (e.g., Blue), and an exogenous **`despawn_robot(blue)`** occurs during execution, the Base Controller will fail. It cannot adapt the plan and will likely end up in an invalid state.

Reactive Controller

The **Reactive Controller** improves upon the base strategy by monitoring environment changes *during* plan execution using `gexec(Guard, Program)`. It reacts to two main types of events:

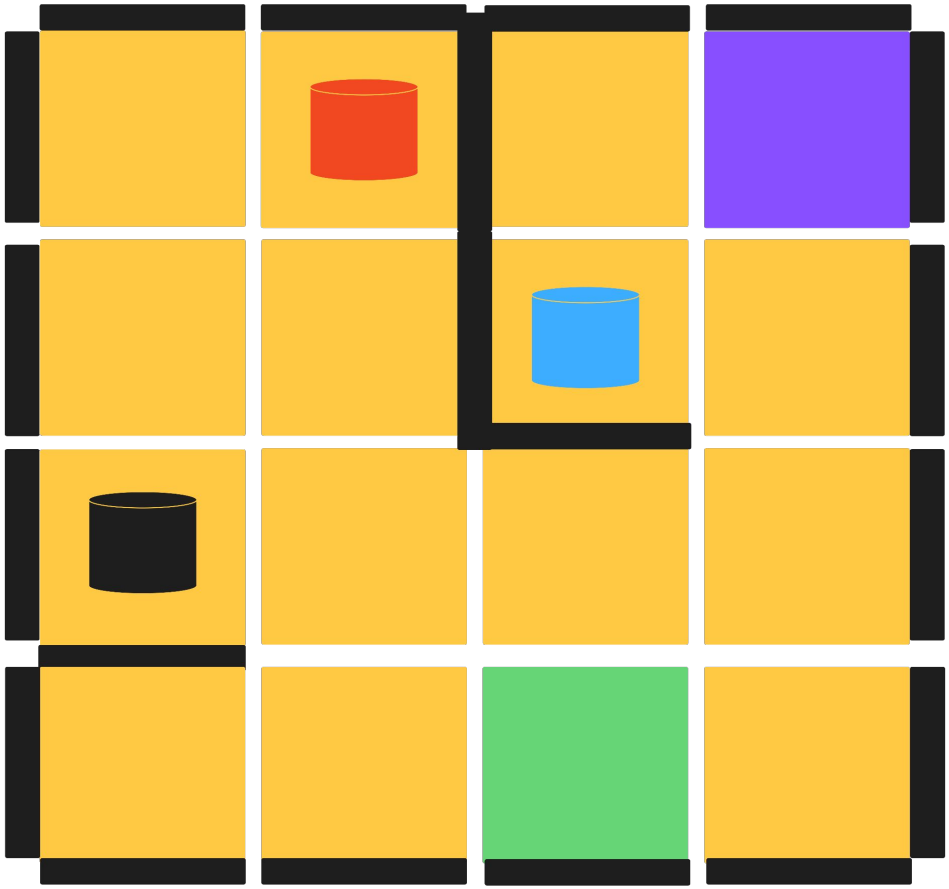
1. **New Goals:** A new target is added (`new_goal_event`).
2. **Map Changes:** A robot/obstacle appears or disappears (`spawn`/`despawn` triggering `robot_config_changed`).

```
% 2. REACTIVE: Interrupts if a new goal arrives or map changes
proc(control(reactive_ricochet), [
    prioritized_interrupts([
        interrupt(stop_requested, [ show_report, ?(fail) ]),
        interrupt(some_pending,
            pi(n, [
                ?(and(target_id(n), goal_active(n))),
                unset_new_goal_event,
                % Guarded Execution: aborts if 'new_goal' or 'config_changed' occurs
                gexec(and(neg(new_goal_event), neg(robot_config_changed)),
                      serve_goal_plan(n))
            ])
        ),
        interrupt(true, ?(wait_exog_action))
    ])
]).
```

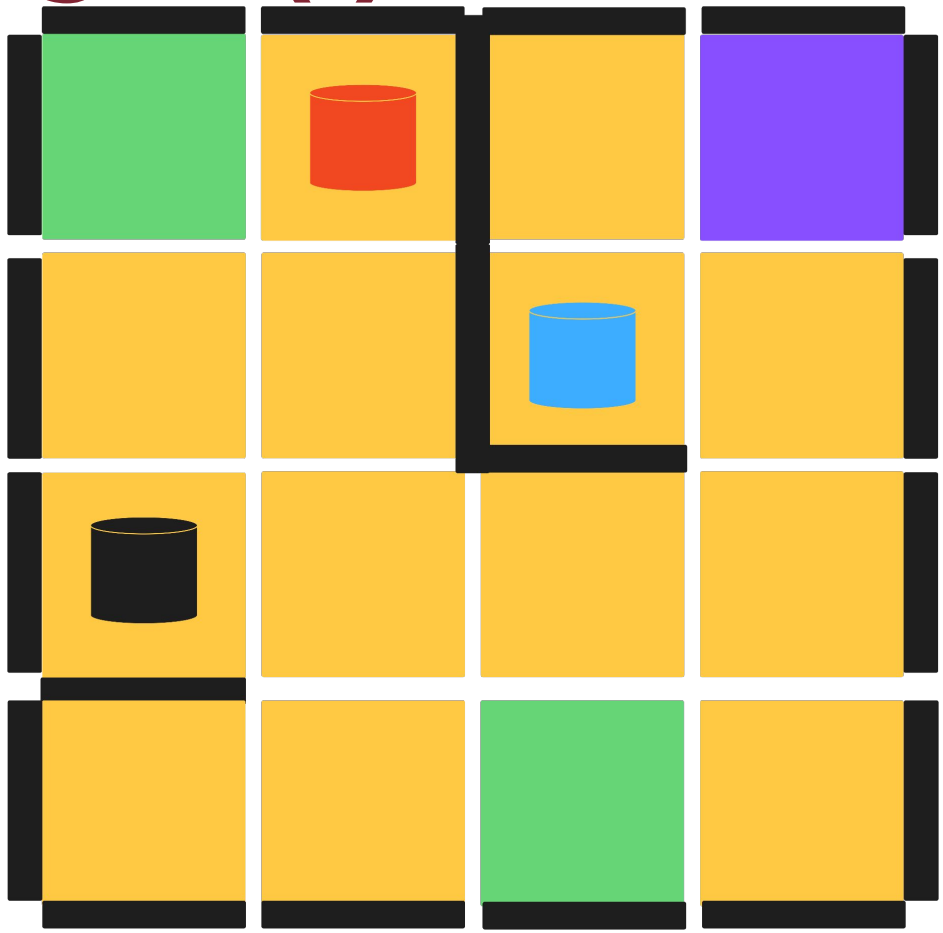
Behavior: If an exogenous action like `spawn_robot(blue) or add_goal(1)` occurs while the Red robot is moving, `robot_config_changed` becomes true. The `gexec` guard fails, instantly **aborting the movement** to avoid collisions or recalculate the path with the new obstacle.

Base Controller add_goal exogenus

add_goal(15)



add_goal(1)



Legend

CHARGING STATION

GOAL

Order:

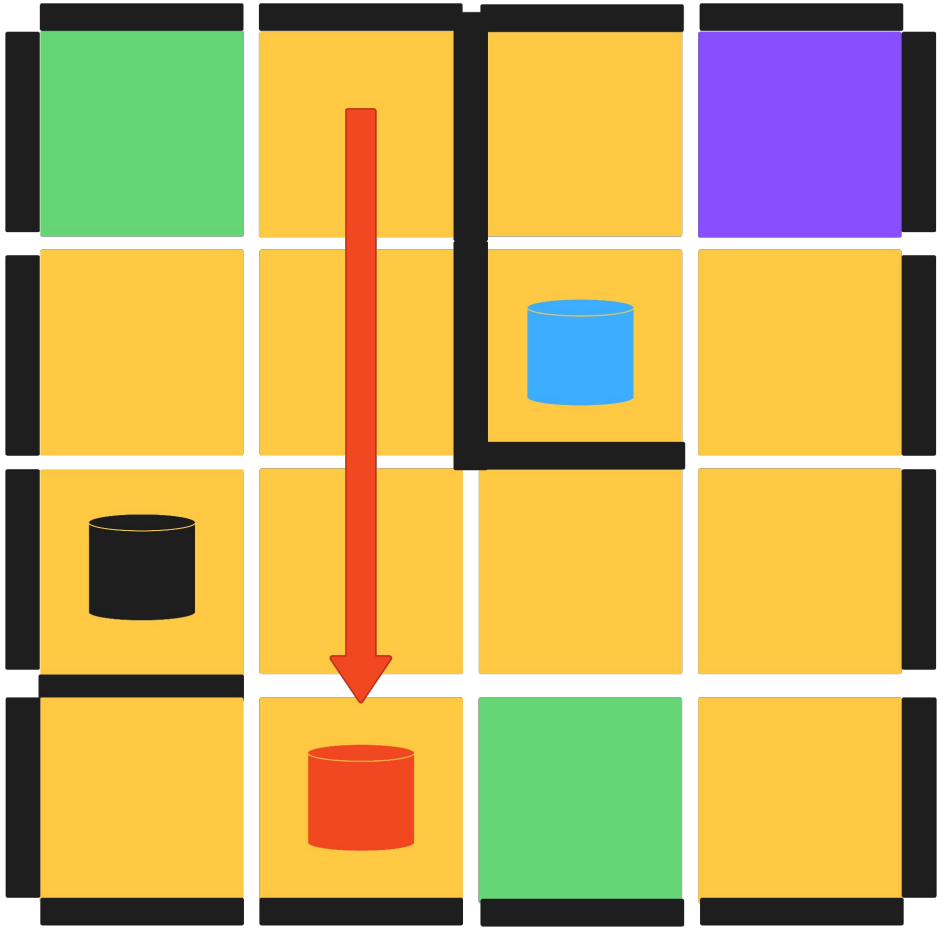
1. c43
2. c11

move_down red c12 c42

Legend

CHARGING STATION

GOAL



Order:

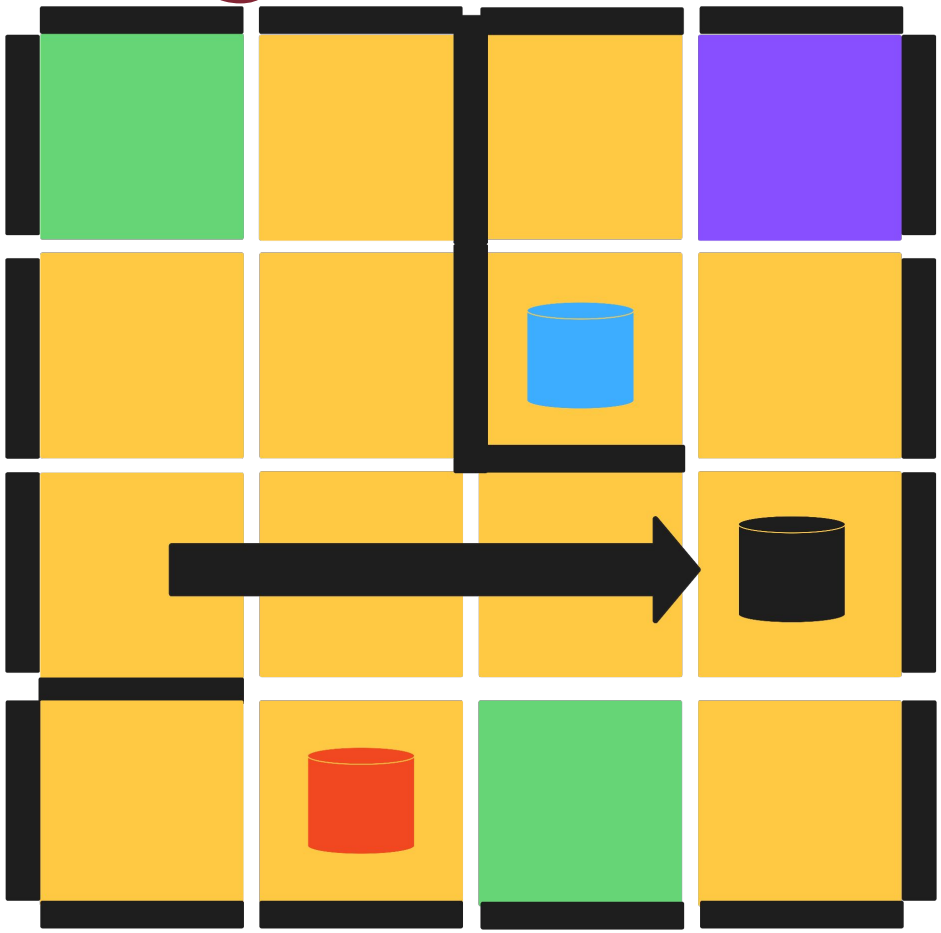
1. c43
2. c11

move_right black c31 c34

Legend

CHARGING STATION

GOAL



Order:

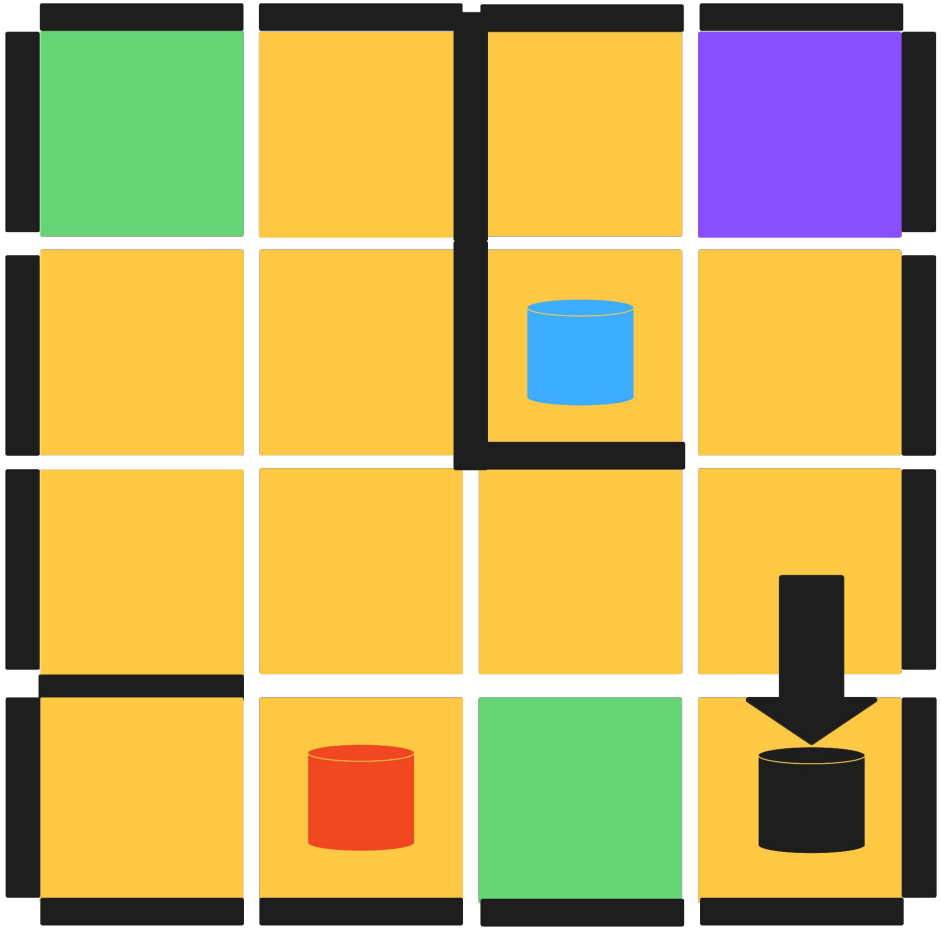
1. c43
2. c11

move_down black c34 c44

Legend

CHARGING STATION

GOAL



Order:

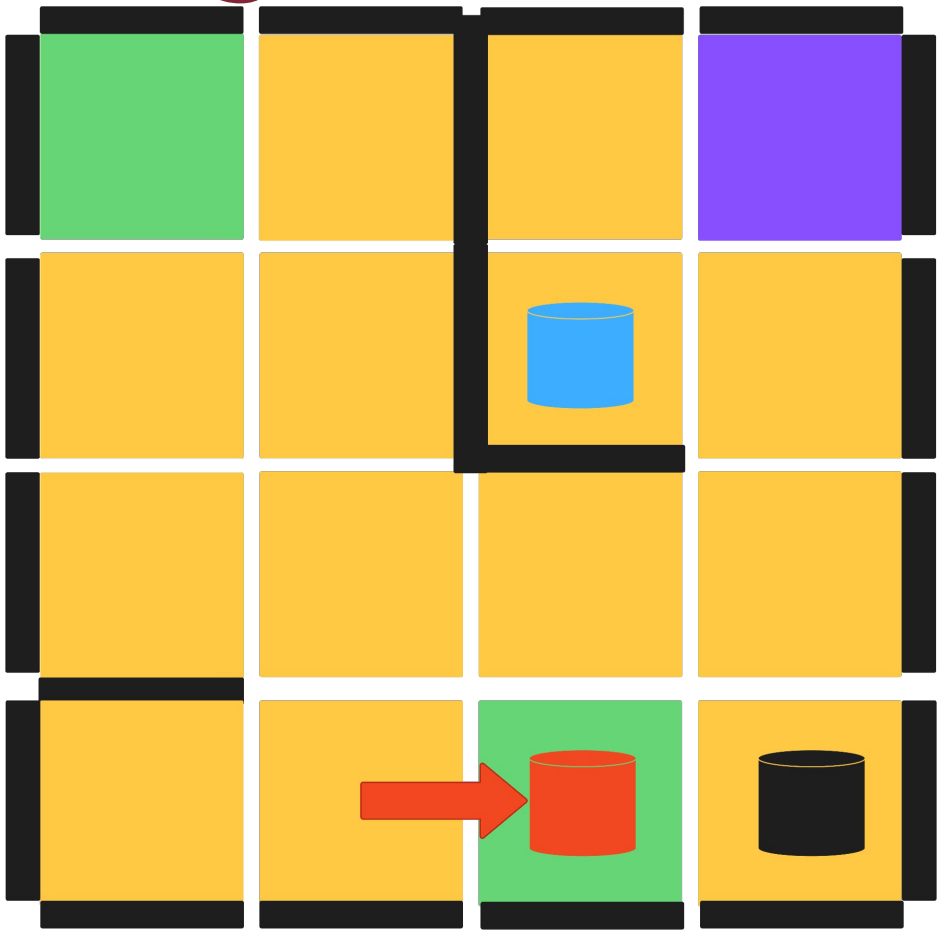
1. c43
2. c11

move_right red c42 c43

Legend

CHARGING STATION

GOAL



Order:

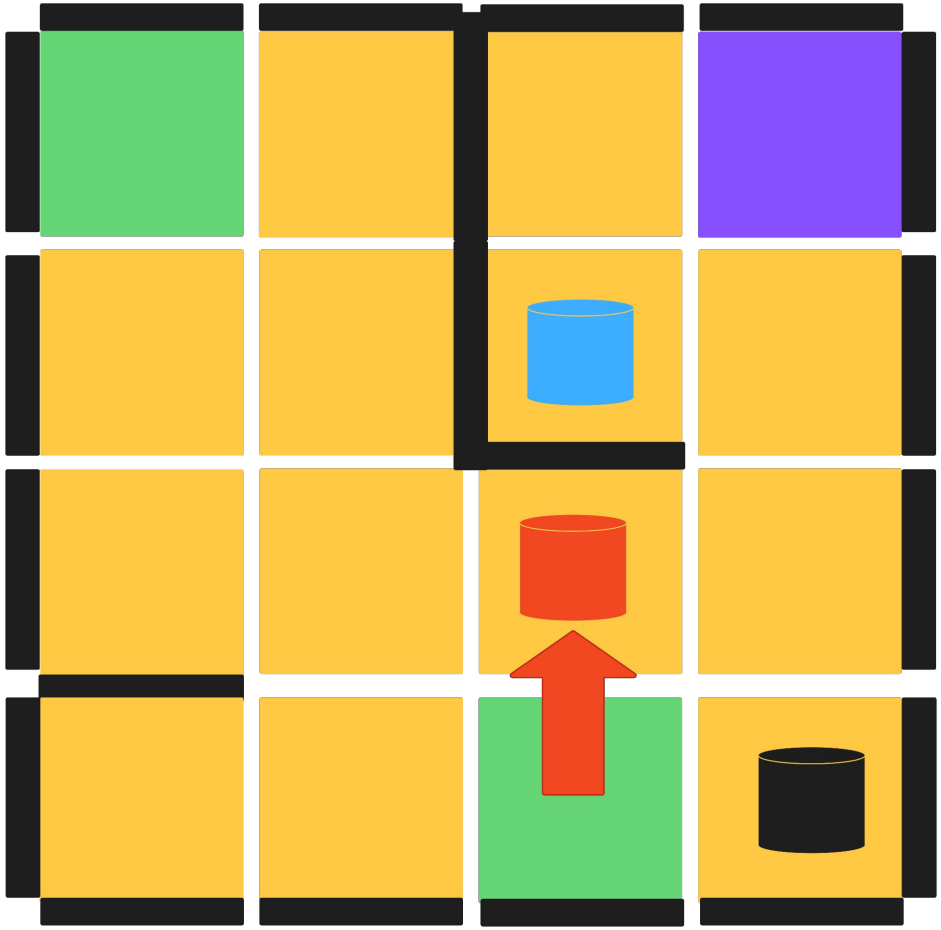
1. c43
2. c11

move_up red c43 c33

Legend

CHARGING STATION

GOAL



Order:

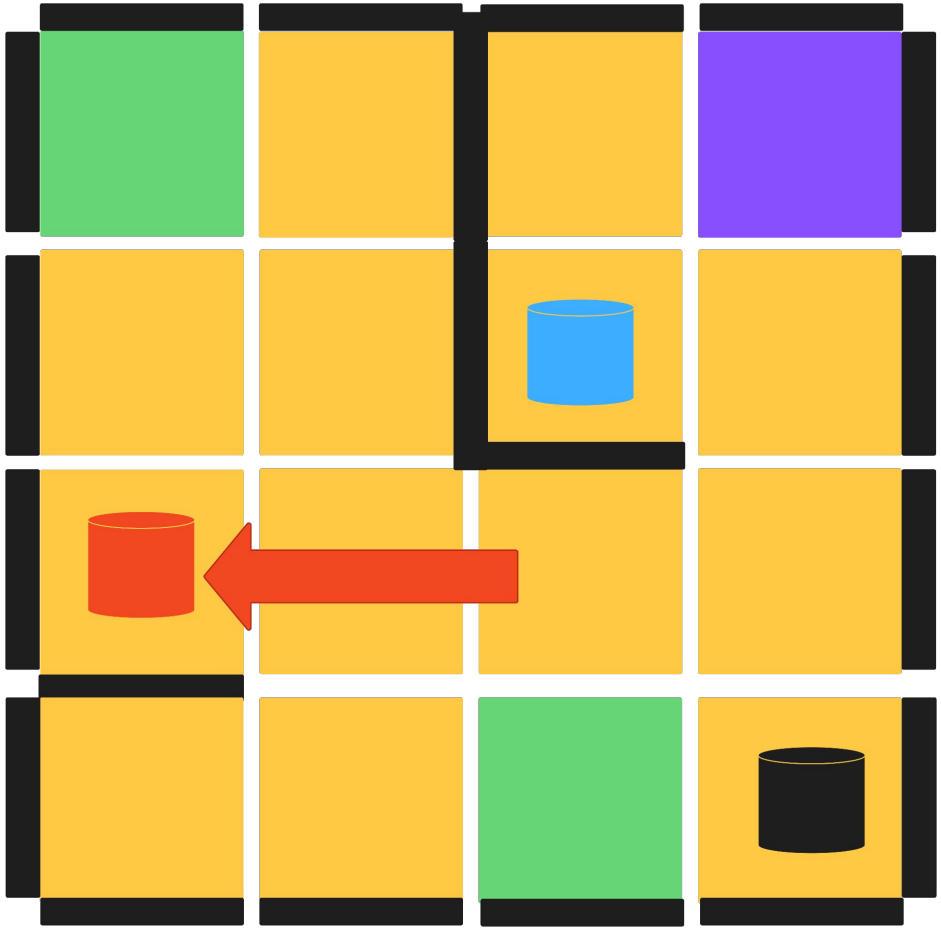
1. c43
2. c11

move_left red c33 c31

Legend

CHARGING STATION

GOAL



Order:

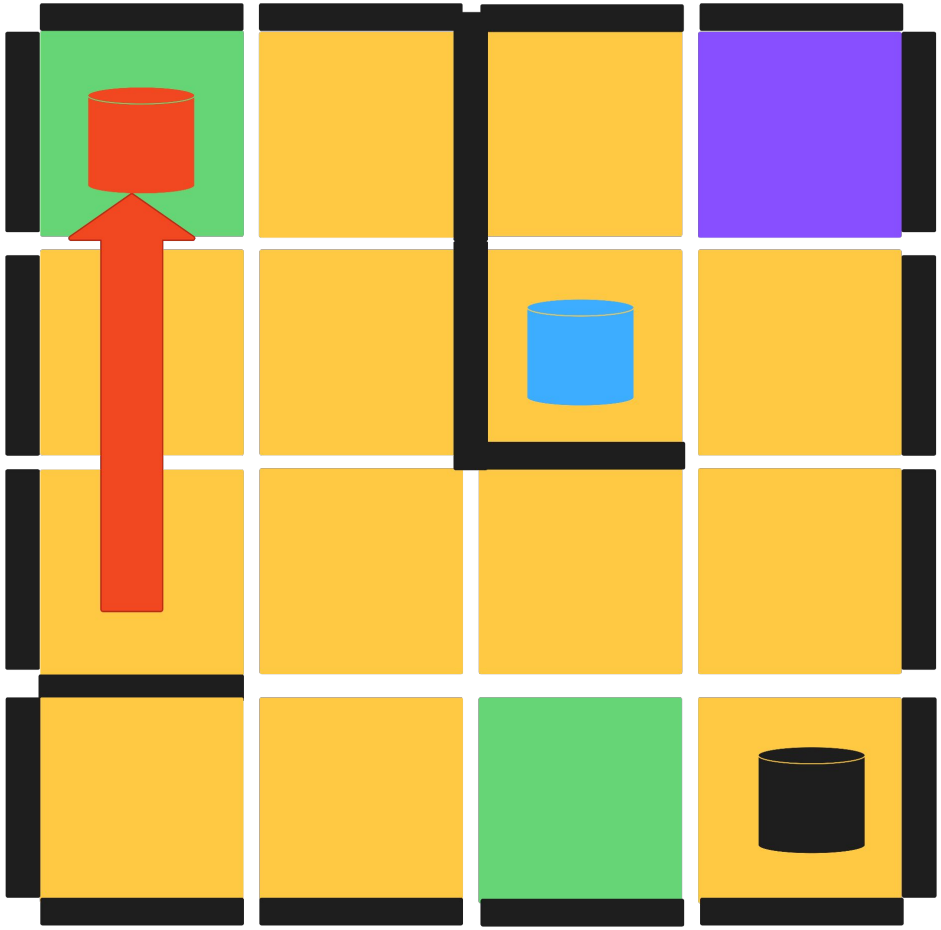
1. c43
2. c11

move_up red c31 c11

Legend

CHARGING STATION

GOAL



Order:

1. c43
2. c11

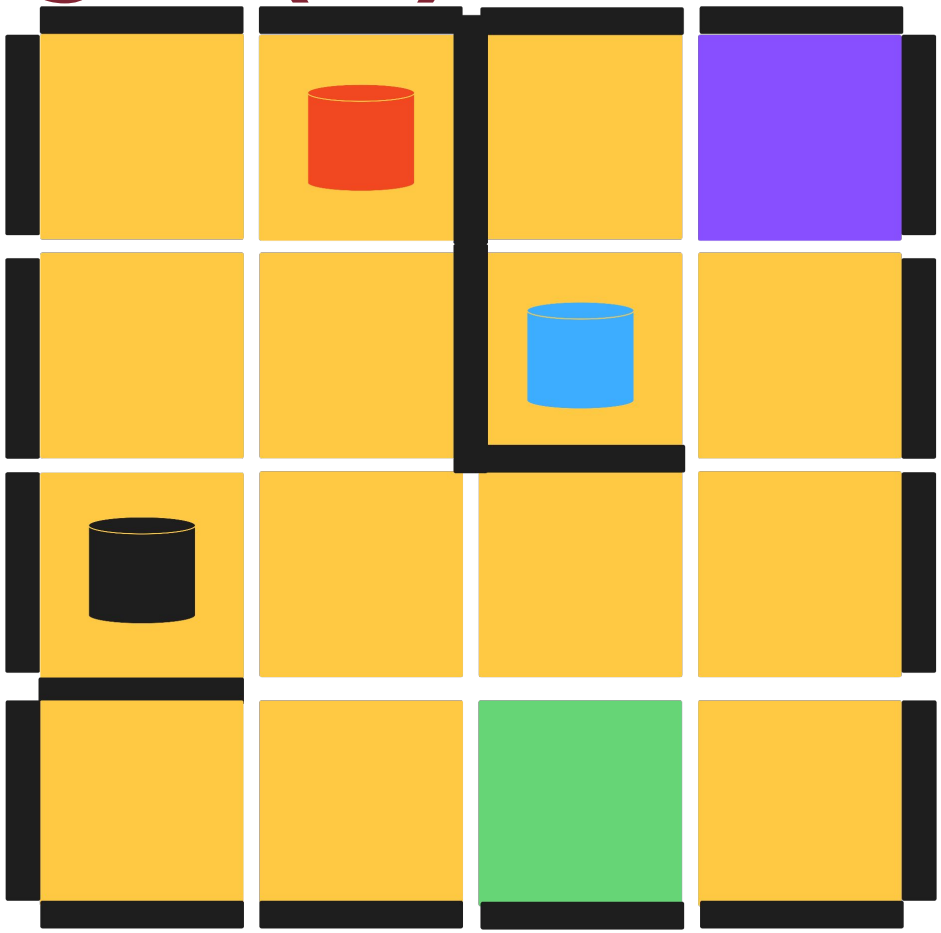
Reactive Controller add_gol exogenus

add_goal(15).

Legend

CHARGING STATION

GOAL



Order:

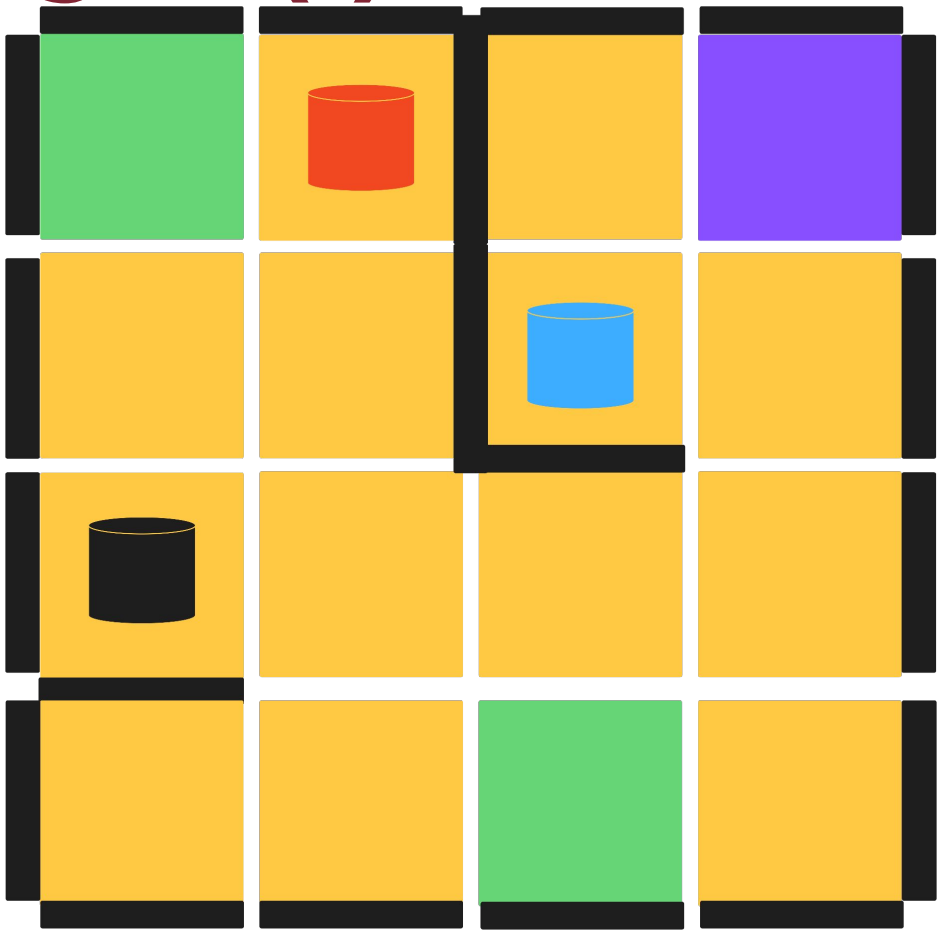
1. c43
2. c11

add_goal(1).

Legend

CHARGING STATION

GOAL



Order:

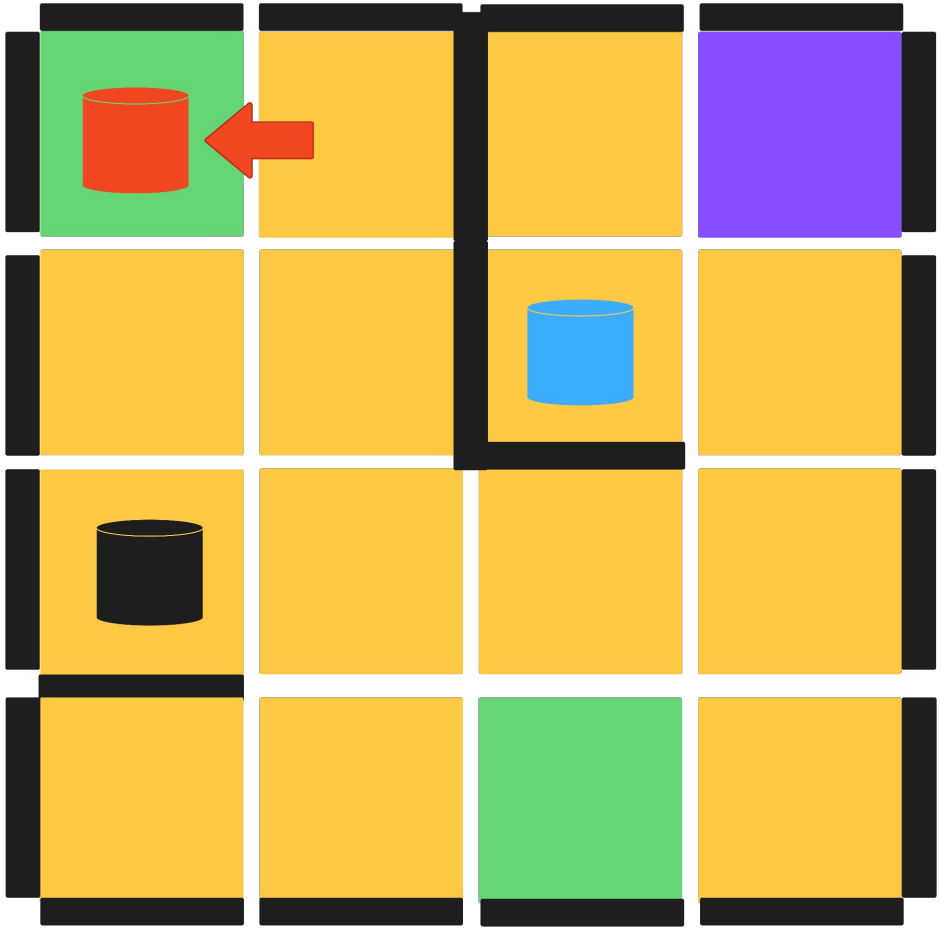
1. c43
2. c11

move_left red c12 c11

Legend

CHARGING STATION

GOAL



Order:

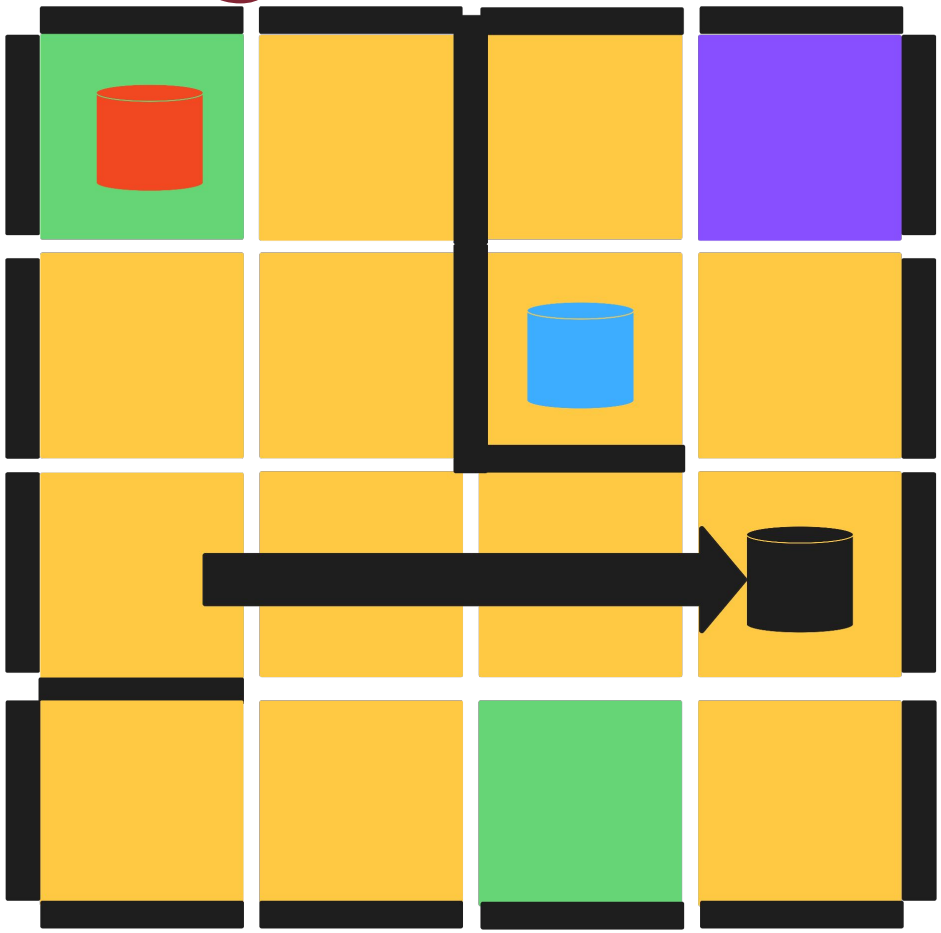
1. c43
2. c11

move_right black c31 c34

Legend

CHARGING STATION

GOAL



Order:

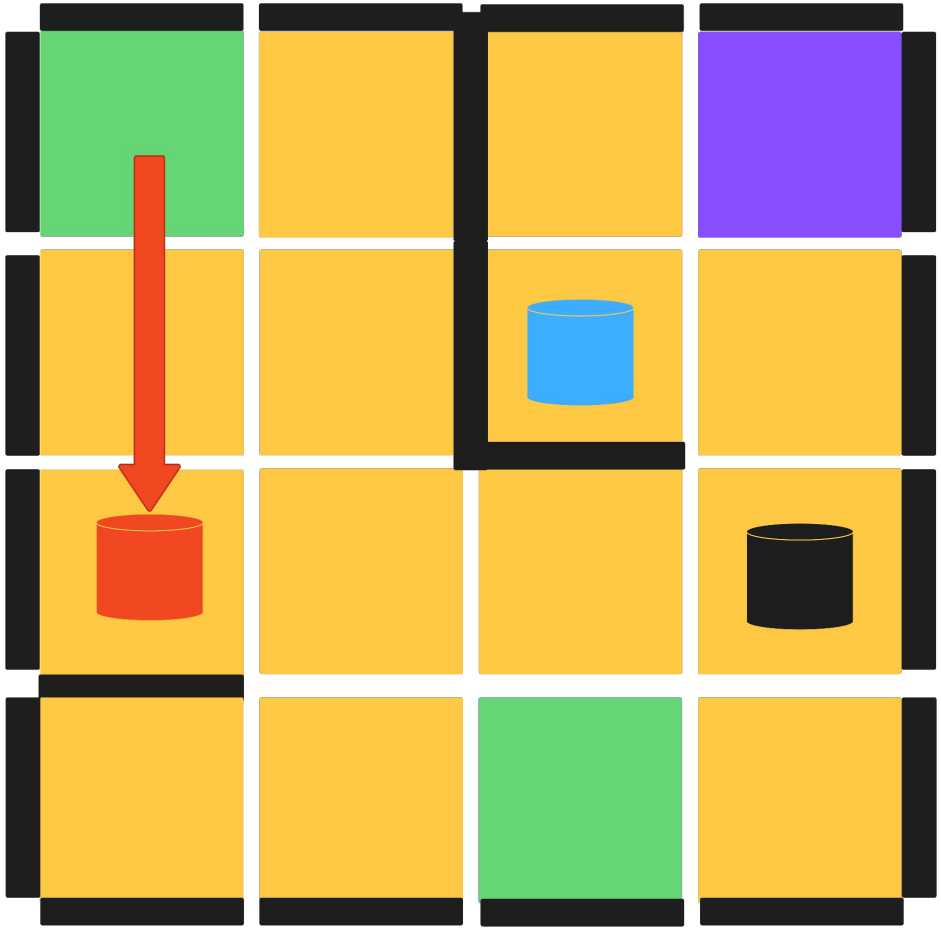
1. c43
2. c11

move_down red c11 c31

Legend

CHARGING STATION

GOAL



Order:

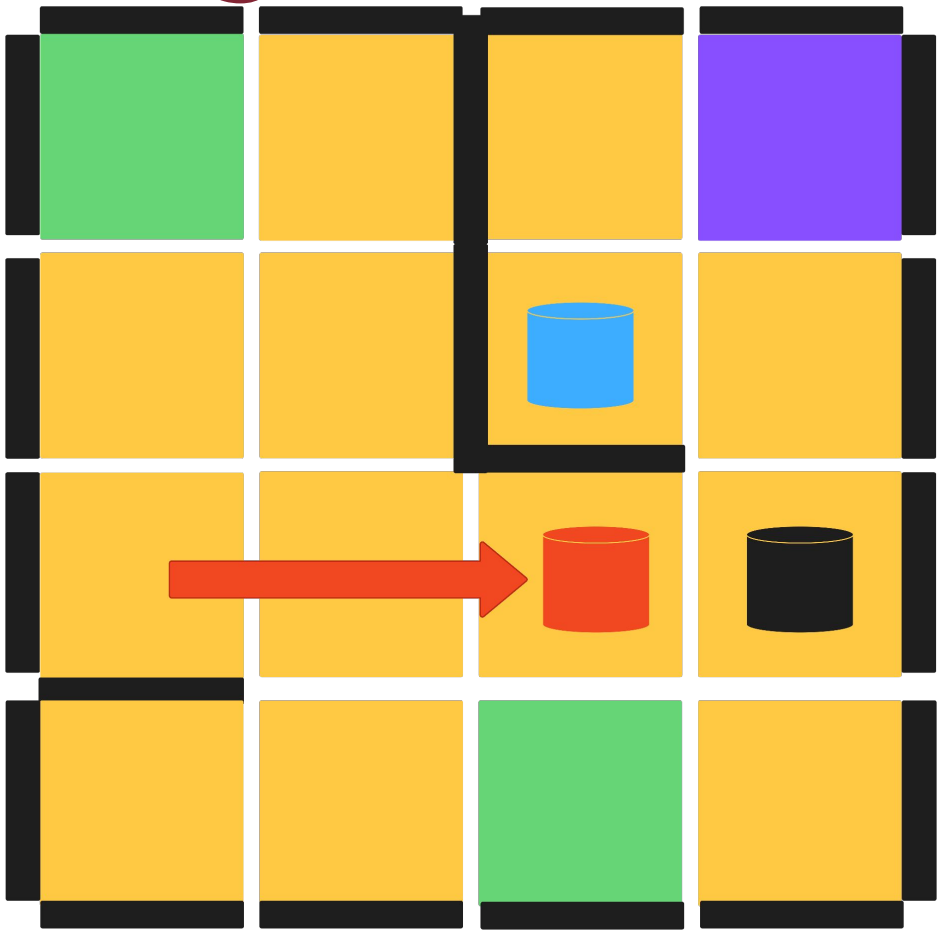
1. c43
2. c11

move_right red c31 c33

Legend

CHARGING STATION

GOAL



Order:

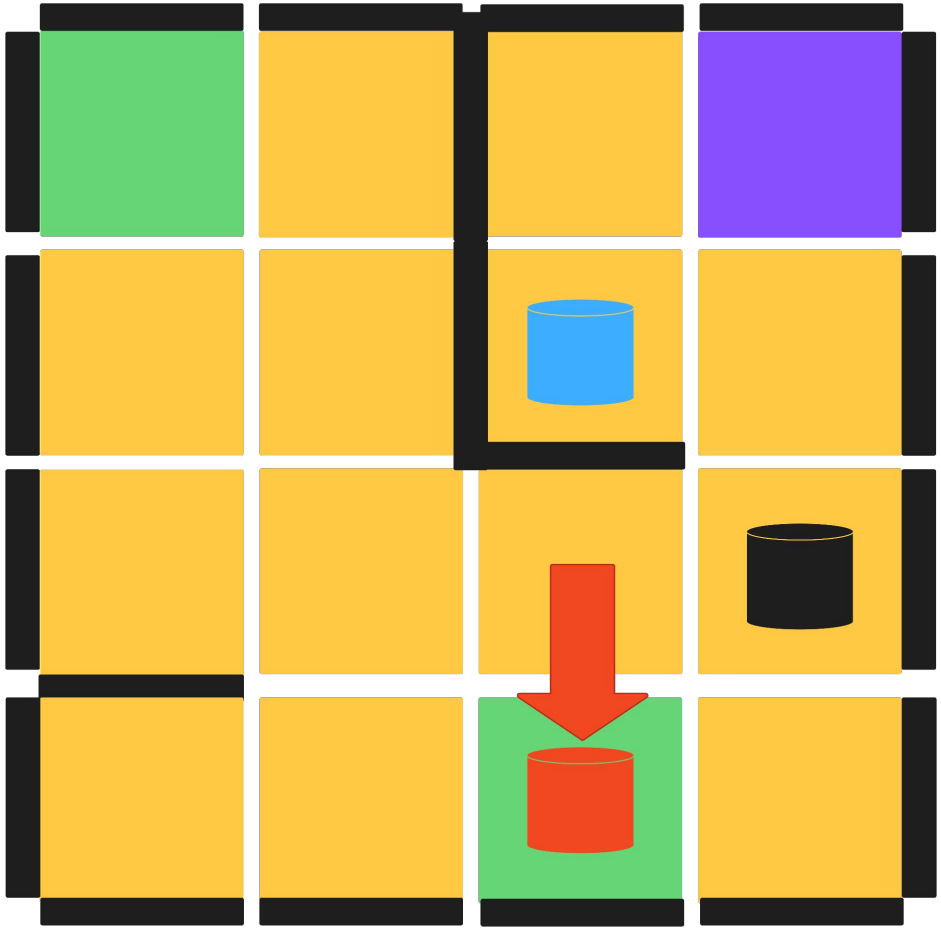
1. c43
2. c11

move_down red c33 c43

Legend

CHARGING STATION

GOAL



Order:

1. c43
2. c11

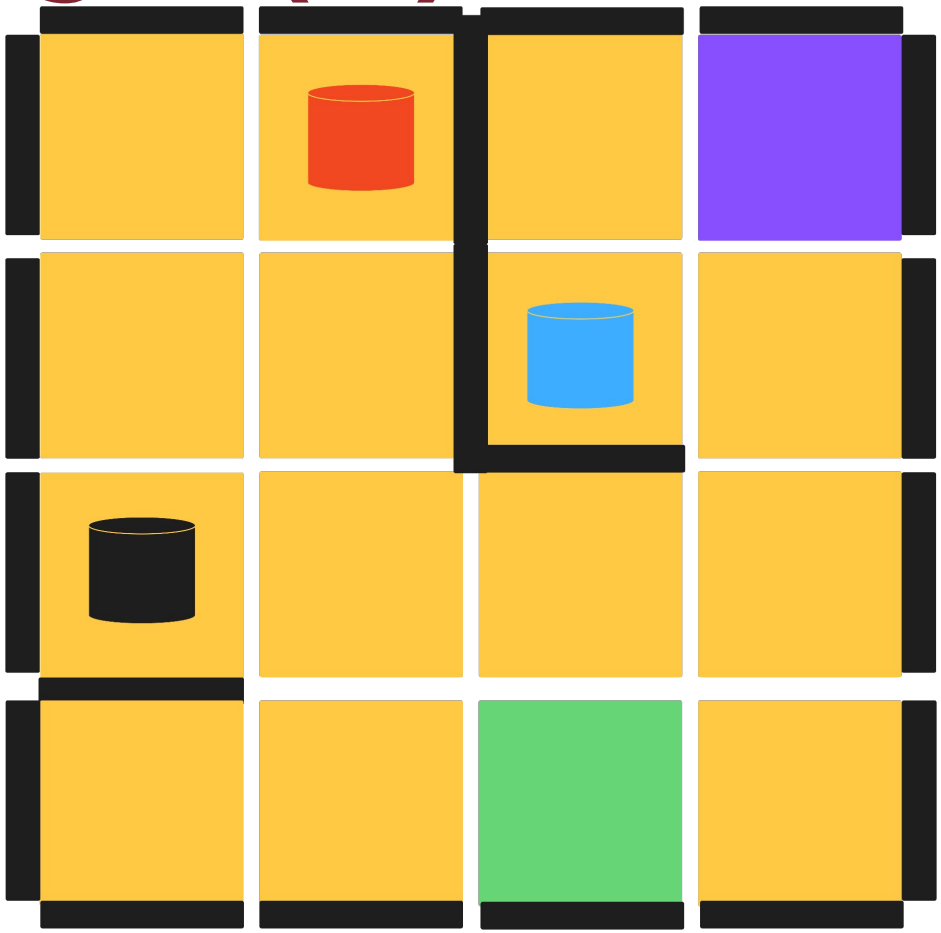
Base Controller despawn exogenus

add_goal(15)

Legend

CHARGING STATION

GOAL

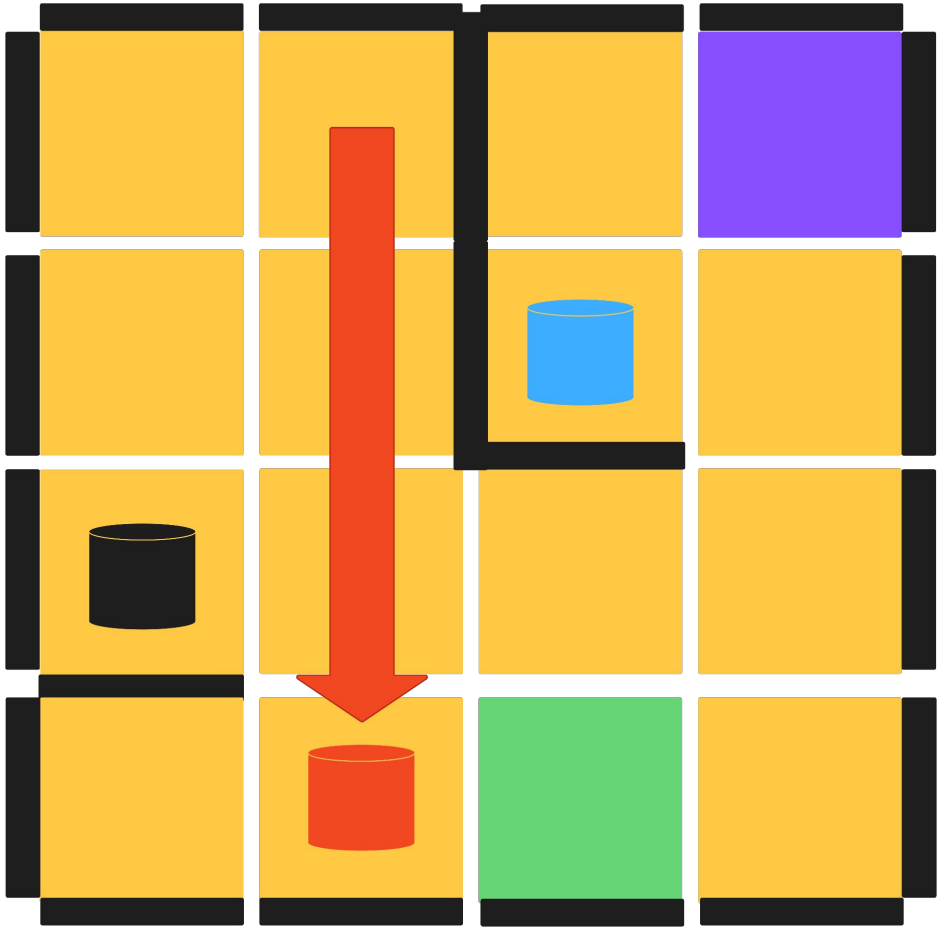


move_down red c12 c42

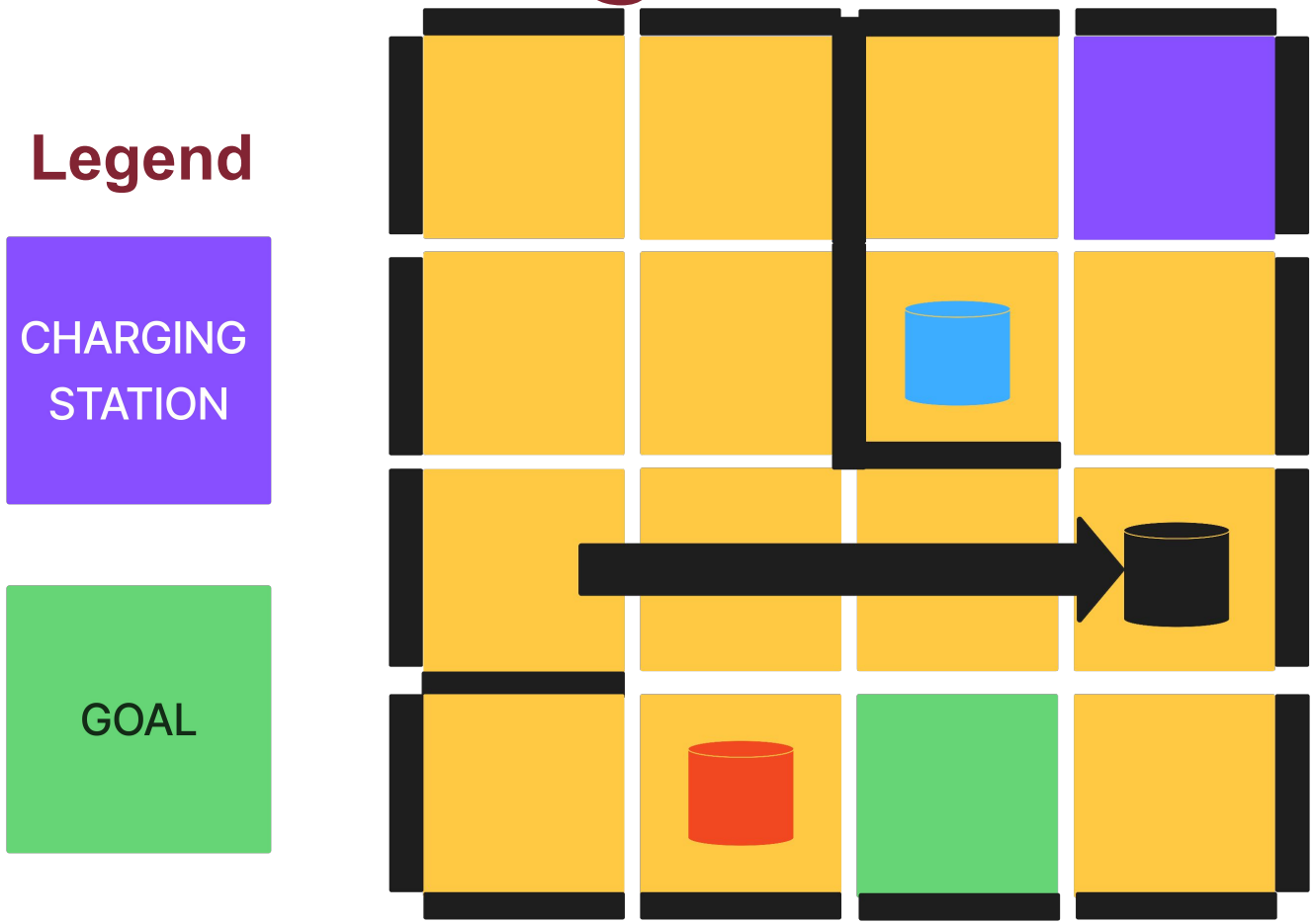
Legend

CHARGING STATION

GOAL



move_right black c31 c34

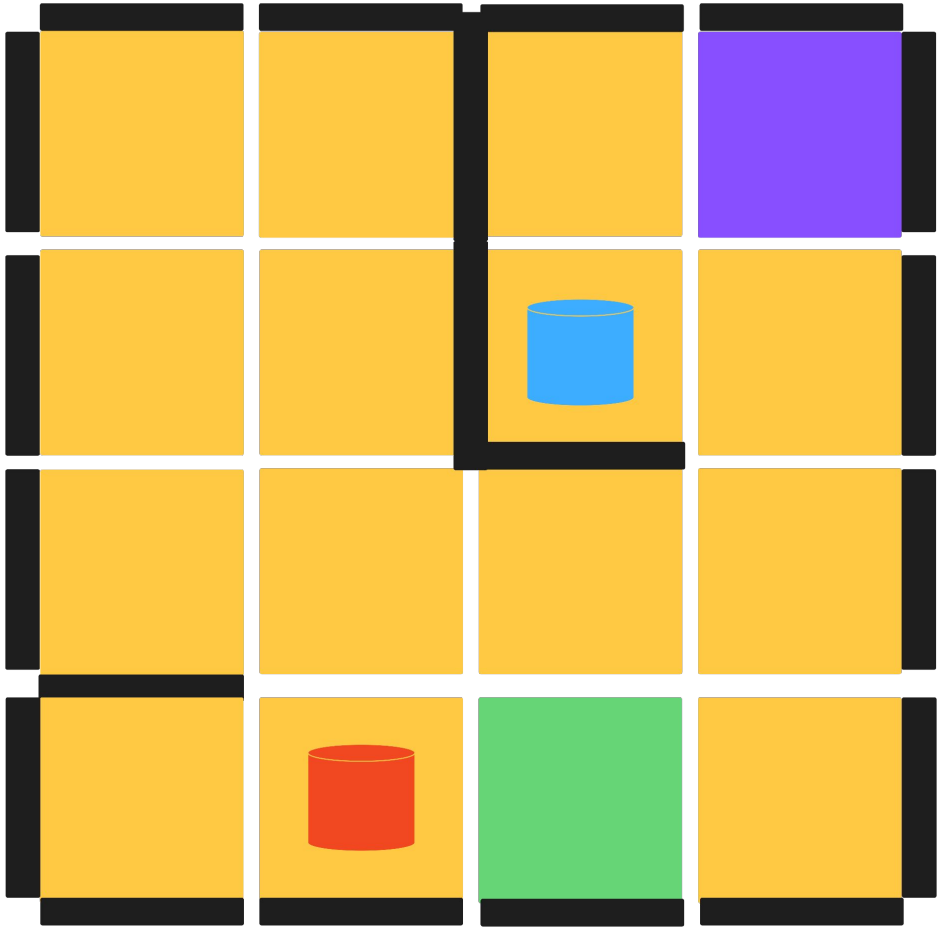


despawn(black)

Legend

CHARGING STATION

GOAL



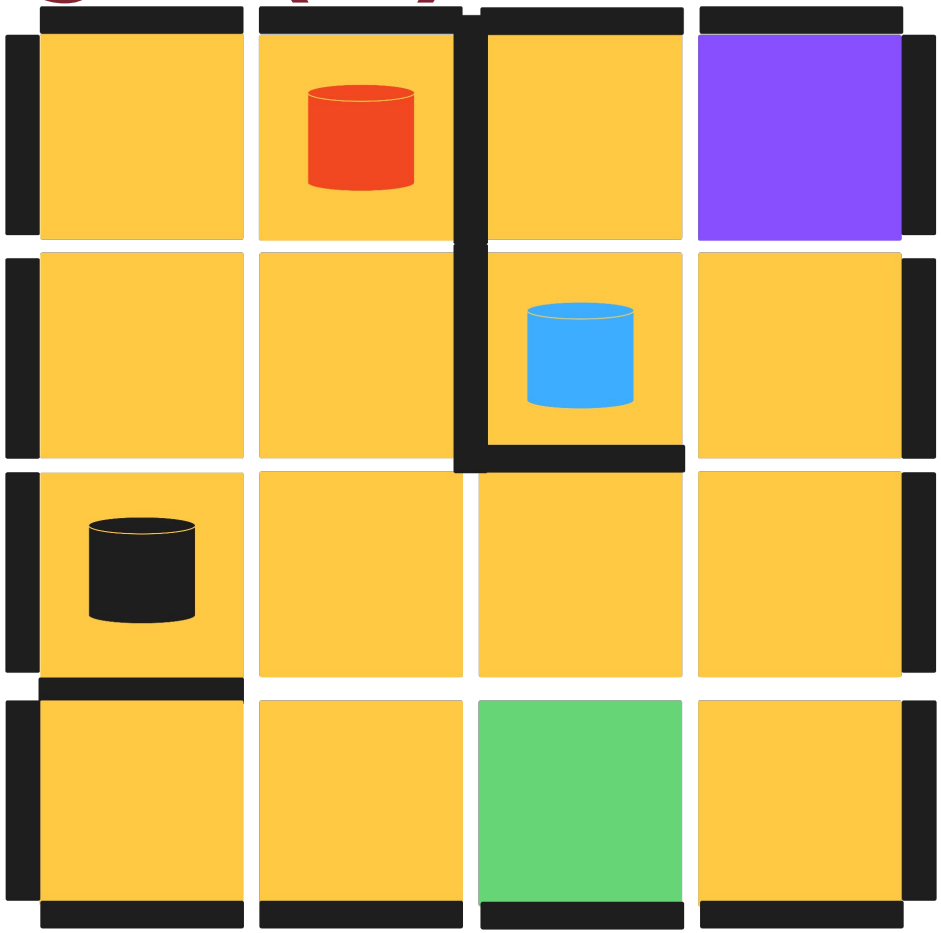
Reactive Controller despawn exogenus

add_goal(15)

Legend

CHARGING STATION

GOAL

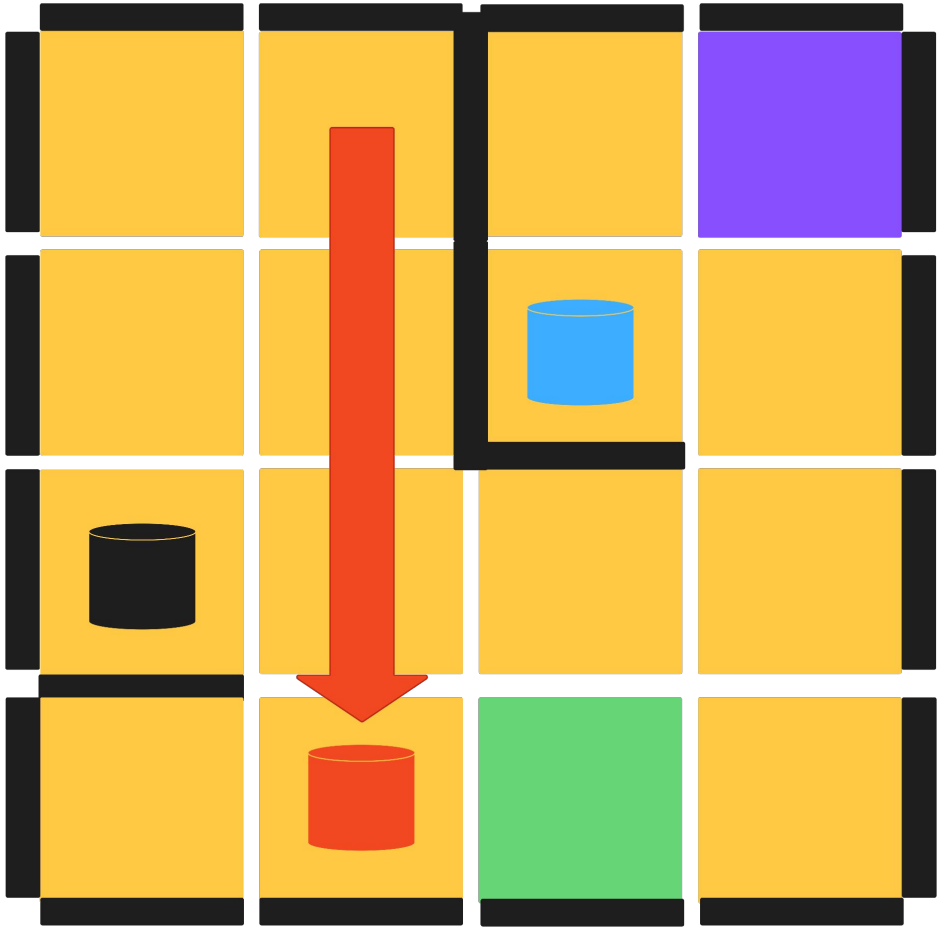


move_down red c12 c42

Legend

CHARGING STATION

GOAL

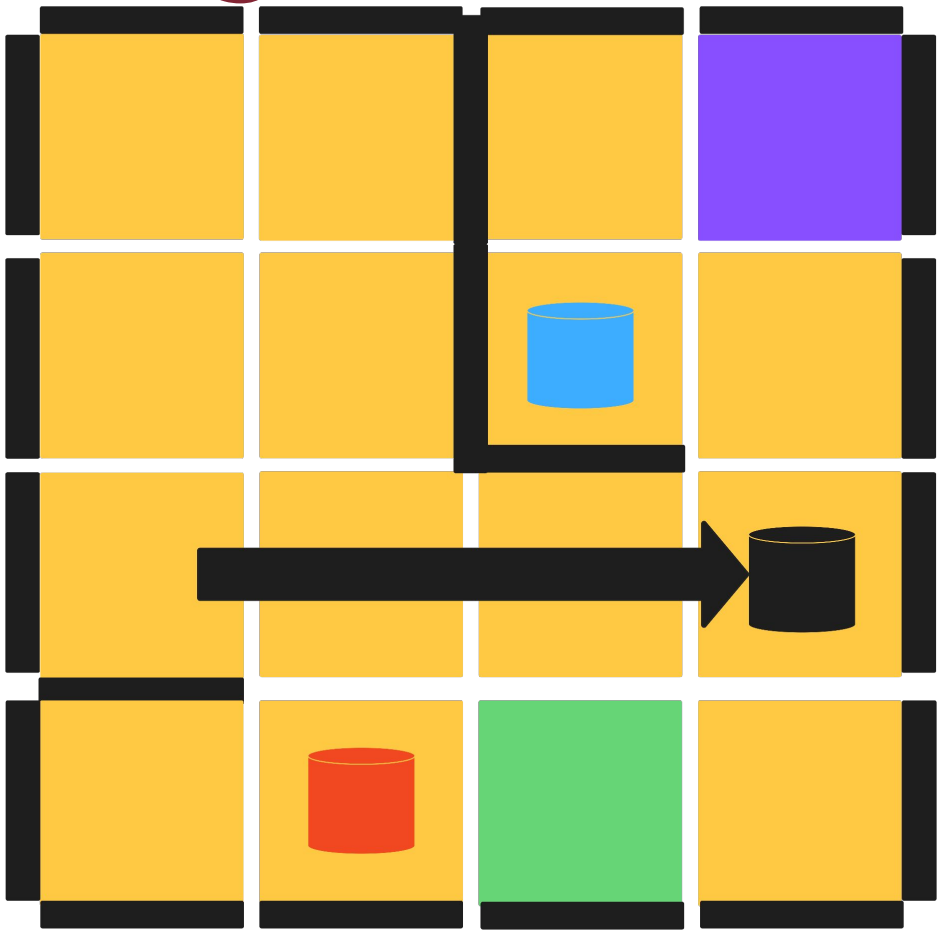


move_right black c31 c34

Legend

CHARGING STATION

GOAL

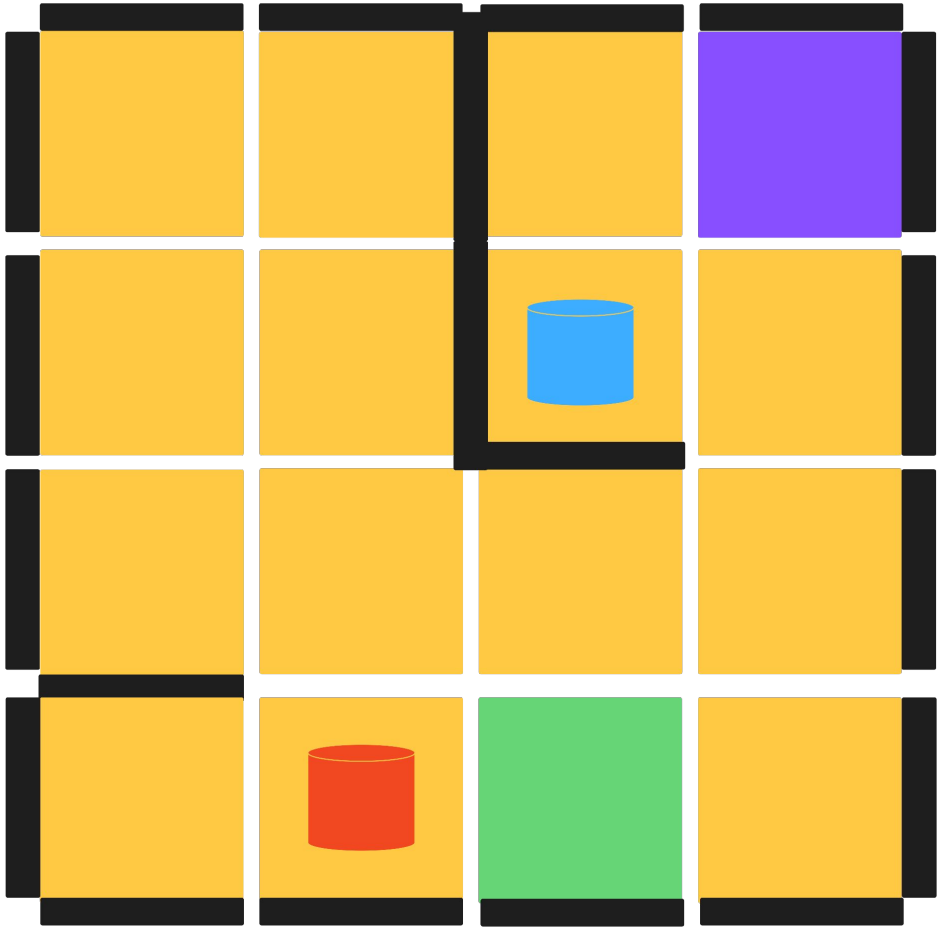


despawn(black)

Legend

CHARGING STATION

GOAL

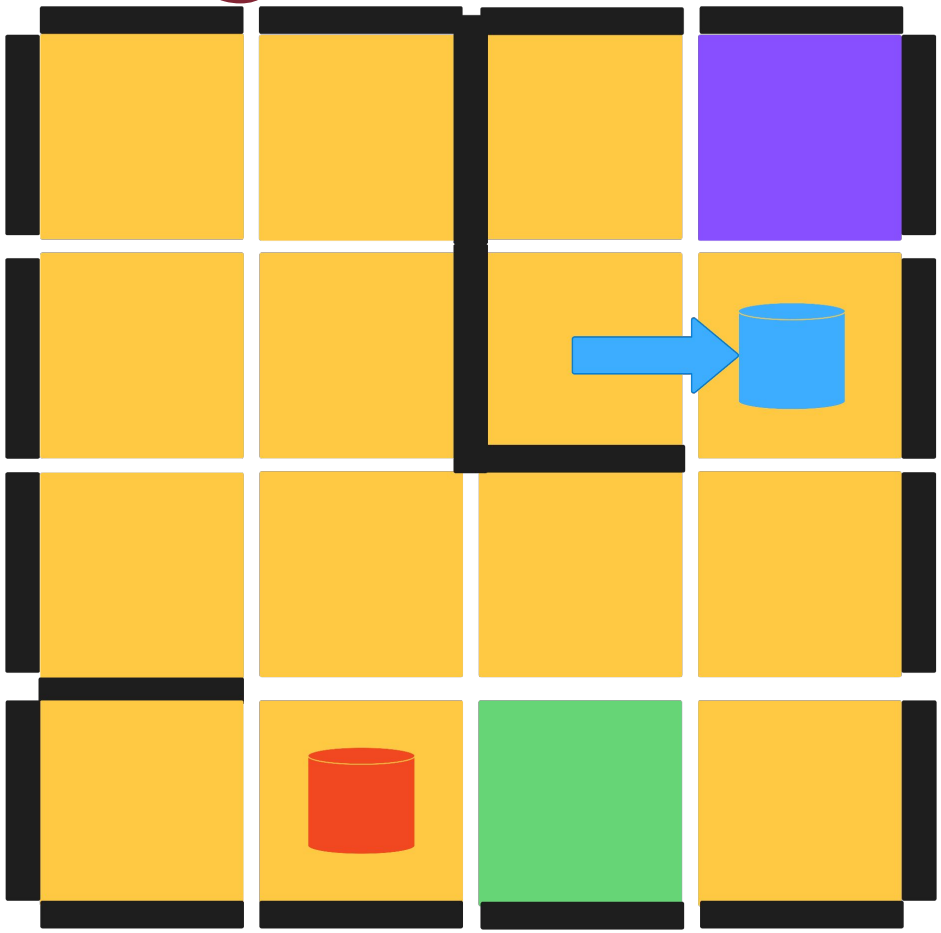


move_right blue c23 c24

Legend

CHARGING STATION

GOAL

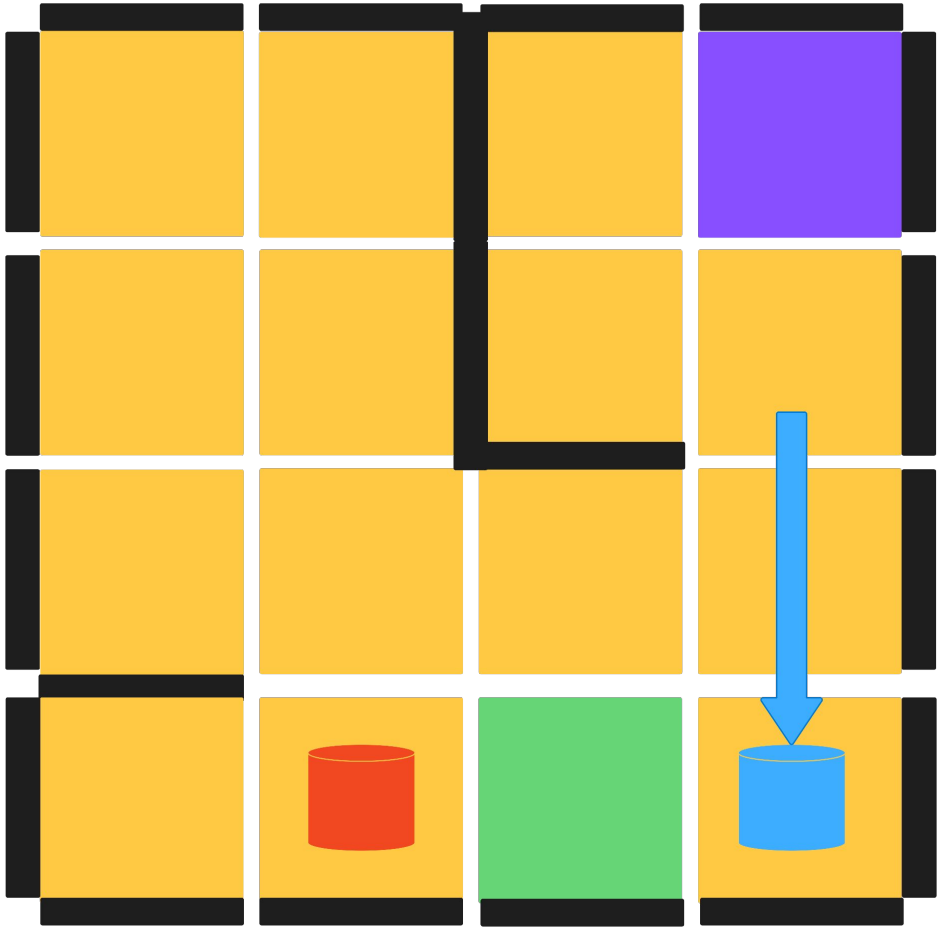


move_down blue c24 c44

Legend

CHARGING STATION

GOAL

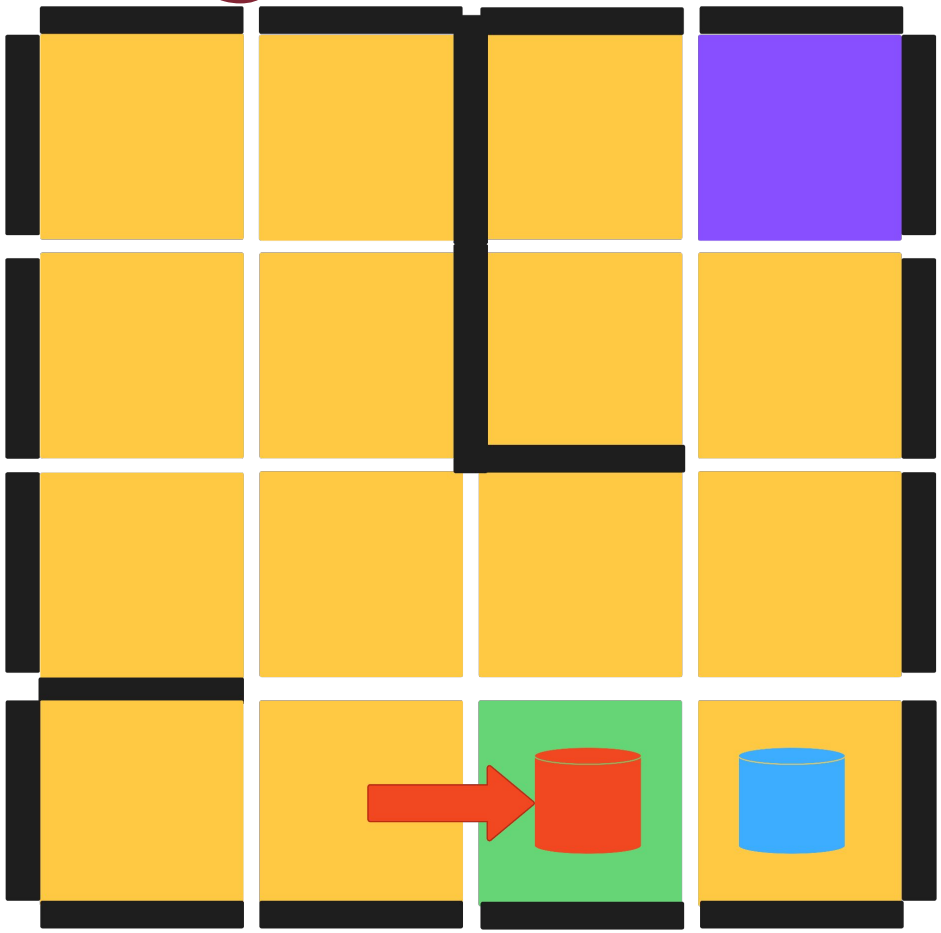


move_right red c42 c43

Legend

CHARGING STATION

GOAL



Thank you for your attention!